Automated Device Compliance Management Portal

DevOps Mini Project - Step-by-step Deliverables and Outputs

Atharva Anand Damale | Roll/ID 23102C0004

Department of Computer Engineering - BE CMPN C

Academic Year 2026-27 (ODD Semester)

Repository: github.com/Athdam05/device-compliance-portal

How to read this report

The report follows the 14 steps given on the course sheet. Each step starts with the task and the deliverable exactly as given, then a checklist that points to where every deliverable is shown, followed by the evidence: screenshots of the running application, Jenkins and GitHub, rendered command/log output, code listings and tables. Terminal and log figures are rendered from the real command output and Jenkins/Ansible logs saved during the work; long logs are shown as excerpts (lines were selected, never altered). The complete source, scripts and raw logs are in the repository.

Step	Title
1	Problem Definition and Scope
2	Agile Planning and DevOps Workflow
3	Requirements, Architecture and Technology Setup
4	Git and GitHub Repository Initialization
5	Feature Development with Branching
6	MVP Completion and Git Collaboration
7	Jenkins Installation and Continuous Integration Job
8	Pipeline as Code and Server Deployment
9	Selenium Test Design and Local Execution
10	Continuous Testing in Jenkins
11	Docker Image and Container Lifecycle
12	Jenkins-Docker Continuous Deployment
13	Configuration Management Script
14	Automated Provisioning and Reliability Validation

Technology used

Java 11, Spring Boot 2.7, Thymeleaf, H2 | Maven 3.9.9 | Git/GitHub | Jenkins 2.580 (JDK 21) | Apache Tomcat 9.0.98 | Selenium 4.21 + Microsoft Edge 155 | Docker 29 + local registry (registry:2) | Ansible core 2.17 (control node in a container), Ubuntu 22.04 managed node.

STEP 1

Problem Definition and Scope

> **Task given:** Study the real-time need for the Device Compliance Management Portal. Identify target users, existing pain points, stakeholders, constraints and measurable success criteria. Freeze a small 15-Task MVP scope.

> **Deliverable required:** Problem statement, stakeholder list, objectives, constraints and approved MVP scope.

Document written for the Device Compliance Management Portal: problem, users, objectives, constraints and the frozen 15-task MVP. Source: `docs/01-problem-definition-and-scope.md` in the repository.

Deliverable required by the sheet	Shown in this report
Problem statement	Section 1 below
Stakeholder list	Section 2 table
Objectives	Section 4
Constraints	Section 5 (and success criteria, section 6)
Approved MVP scope	Section 7 - 15 tasks, out-of-scope list

1. Problem Statement

Organisations (college labs, small IT teams) track laptops, desktops and servers in spreadsheets. Checks such as OS patch level, antivirus status, disk encryption and firewall state are done manually, so non-compliant devices go unnoticed, audits are slow, and there is no single place to see who is responsible for fixing what.

The portal centralises device records, evaluates each device against compliance rules, and drives a role-based workflow (Reported -> Under Review -> Remediated -> Closed).

2. Target Users & Stakeholders

Stakeholder	Role in system	Need
IT Administrator	ADMIN	Register devices, manage rules, close findings
Compliance Officer	AUDITOR	Review non-compliant devices, approve remediation
Technician	TECHNICIAN	Fix issues, update device status
Department Head / Management	Viewer (dashboard)	Summary of compliance posture
Course Teacher / Evaluator	External	Verify DevOps pipeline end to end

3. Existing Pain Points

- Spreadsheet records are stale, duplicated and not searchable.
- No ownership/accountability trail for remediation.
- No instant summary (compliant vs non-compliant counts).
- Manual deployment of internal tools is error prone.

4. Objectives

1. Create, view, update and search device records.
2. Role-based status workflow for compliance findings.
3. Summary dashboard (total / compliant / non-compliant / under review).
4. Fully automated build, test, containerise and deploy pipeline (CI/CD).

5. Constraints

- Individual student project; one semester timeline.
- Java 11, Maven, Spring Boot, embedded H2 DB; Tomcat/Docker targets.
- Free tools only (GitHub, Jenkins, Docker Hub, Ansible).

- Runs on a single local node (Windows 11 + Docker).

6. Measurable Success Criteria

- All 15 MVP tasks done and demonstrable.
- ≥ 4 Selenium journeys pass automatically in Jenkins.
- Commit-to-running-container pipeline completes in < 10 min.
- Ansible re-run reports 0 changed tasks (idempotent); rollback < 2 min.

7. Frozen 15-Task MVP Scope

#	Task
1	Project skeleton (Spring Boot, Maven)
2	Device entity & repository (H2)
3	Create device form
4	List/view devices
5	View device details
6	Update device
7	Search/filter by name, owner, status
8	Compliance rule evaluation (OS, AV, encryption, firewall)
9	User roles (ADMIN, AUDITOR, TECHNICIAN) & login
10	Status workflow transitions with role checks
11	Status history log
12	Summary dashboard
13	Input validation & error pages
14	Health endpoint
15	README & API documentation

Out of scope: agent-based auto-scanning, email alerts, SSO, multi-tenant.

Approved by: Course teacher (for the course teacher's sign-off)

STEP 2

Agile Planning and DevOps Workflow

> **Task given:** Create user stories and acceptance criteria for the Device Compliance Management Portal. Prepare a product backlog, 15-Task Kanban/Scrum plan, Definition of Done and a DevOps lifecycle diagram from development to operations.

> **Deliverable required:** Backlog, task board, sprint plan, Definition of Done and DevOps workflow diagram.

Agile artefacts for the portal. The Kanban board and backlog are maintained as tables in the repository (`docs/02-agile-planning-and-devops-workflow.md`); the final state of the backlog is in step 6.

Deliverable required by the sheet	Shown in this report
Backlog	10 user stories with acceptance criteria + prioritised 15-task backlog
Task board	Kanban table (state at planning time); final board in step 6
Sprint plan	3 sprints with goals
Definition of Done	Section 5
DevOps workflow diagram	Figure: DevOps lifecycle

1. User Stories & Acceptance Criteria

ID	User Story	Acceptance Criteria
US1	As an Admin, I want to register a device so it is tracked.	Form validates name, owner, OS, serial; saved device appears in list.
US2	As a user, I want to view all devices and details.	List shows name, owner, status; clicking opens detail page.
US3	As an Admin/Technician, I want to update a device.	Edits persist; validation errors shown inline.
US4	As a user, I want to search devices.	Search by name/owner/status returns matching rows; empty result message shown.
US5	As an Auditor, I want devices auto-evaluated against rules.	Device with outdated OS / no AV / no encryption / firewall off is flagged NON_COMPLIANT.
US6	As a user, I want to log in with a role.	Wrong credentials rejected; menus depend on role.
US7	As staff, I want a status workflow.	Only allowed transitions per role (Technician: Reported->Under Review->Remediated; Auditor/Admin: ->Closed).
US8	As an Auditor, I want status history.	Each transition stores who, when, from, to.
US9	As a manager, I want a dashboard.	Shows totals of compliant, non-compliant, under review devices.
US10	As an Ops engineer, I want a health endpoint.	`/actuator/health` returns UP.

2. Product Backlog (prioritised, 15 tasks)

#	Task	Story	Pts	Sprint
1	Project skeleton (Spring Boot, Maven)	-	2	1
2	Device entity & repository	US1	3	1
3	Create device form	US1	3	1
4	List/view devices	US2	2	1
5	Device details page	US2	2	1
6	Update device	US3	3	2
7	Search/filter	US4	3	2

#	Task	Story	Pts	Sprint
8	Compliance rule engine	US5	5	2
9	Roles & login	US6	5	2
10	Status workflow w/ role checks	US7	5	3
11	Status history log	US8	3	3
12	Summary dashboard	US9	3	3
13	Validation & error pages	US1,3	2	3
14	Health endpoint	US10	1	3
15	README & API docs	-	2	3

3. Sprint Plan (3 sprints x ~1 week)

- **Sprint 1 – Foundation & Feature 1:** tasks 1-5 (device create/list/view). Goal: first working workflow.
- **Sprint 2 – Core MVP:** tasks 6-9 (update, search, rules, login).
- **Sprint 3 – Workflow & Release:** tasks 10-15, release tag v1.0.0. DevOps work (Jenkins, Docker, Ansible) runs in parallel as a pipeline track.

4. Kanban Board (current state)

Backlog	To Do	In Progress	Review	Done
6-15	2,3,4,5	1	-	Step 1 scope, Step 2 planning

Columns WIP limit: In Progress <= 2, Review <= 2. Board maintained as GitHub Project / Issues.

5. Definition of Done

- Code compiled, merged to `develop` via reviewed pull request.
- Unit tests pass; Selenium journey for the feature passes.
- Jenkins build is green.
- Acceptance criteria demonstrated.
- README/docs updated; no critical warnings.
- Deployable as WAR and Docker image.

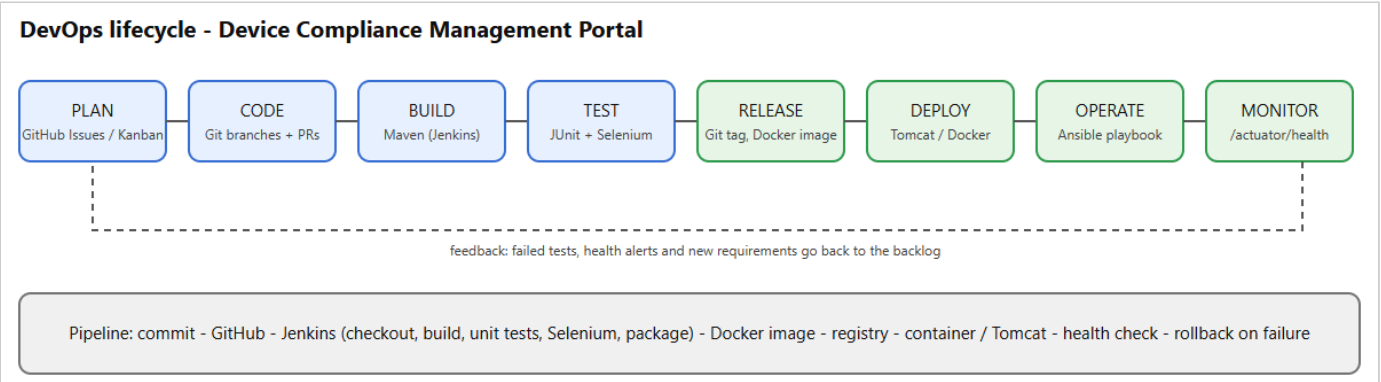


Figure 2.1 - DevOps lifecycle from development to operations (as implemented in this project)

STEP 3

Requirements, Architecture and Technology Setup

> **Task given:** Design the minimum application architecture for the Device Compliance Management Portal. The MVP should cover create, view, update and search records; role-based status workflow; summary dashboard. Select the programming stack, Maven/Gradle/Ant build tool, database if required, Tomcat/Nginx deployment target and local development setup.

> **Deliverable required:** SRS summary, use-case diagram, architecture diagram, data model/API list and working local setup.

Architecture and technology choices, plus a working local setup (build, tests, run). Source: `docs/03-requirements-architecture-setup.md`.

Deliverable required by the sheet	Shown in this report
SRS summary	Section 1
Use-case diagram	Figure 3.1
Architecture diagram	Figure 3.2
Data model / API list	Sections 5 and 6
Working local setup	Figures 3.3-3.4: build output and running application

1. SRS Summary

Functional: FR1 create device · FR2 view list/details · FR3 update · FR4 search (name/owner/status) · FR5 compliance evaluation · FR6 login with roles · FR7 role-based status workflow · FR8 status history · FR9 dashboard · FR10 health endpoint. **Non-functional:** NFR1 page response < 2 s · NFR2 passwords hashed · NFR3 deployable as WAR (Tomcat) and Docker image · NFR4 automated build/test/deploy · NFR5 idempotent provisioning.

2. Technology Stack

Layer	Choice
Language / Framework	Java 11, Spring Boot 2.7 (MVC, Data JPA, Validation, Actuator)
UI	Thymeleaf server-side templates
Build tool	Apache Maven 3.9.9
Database	H2 (file mode)
Deployment target	Apache Tomcat 9 (WAR) and Docker container
CI/CD	Git/GitHub, Jenkins, Docker Hub
Testing	JUnit 5, Selenium WebDriver
Config management	Ansible

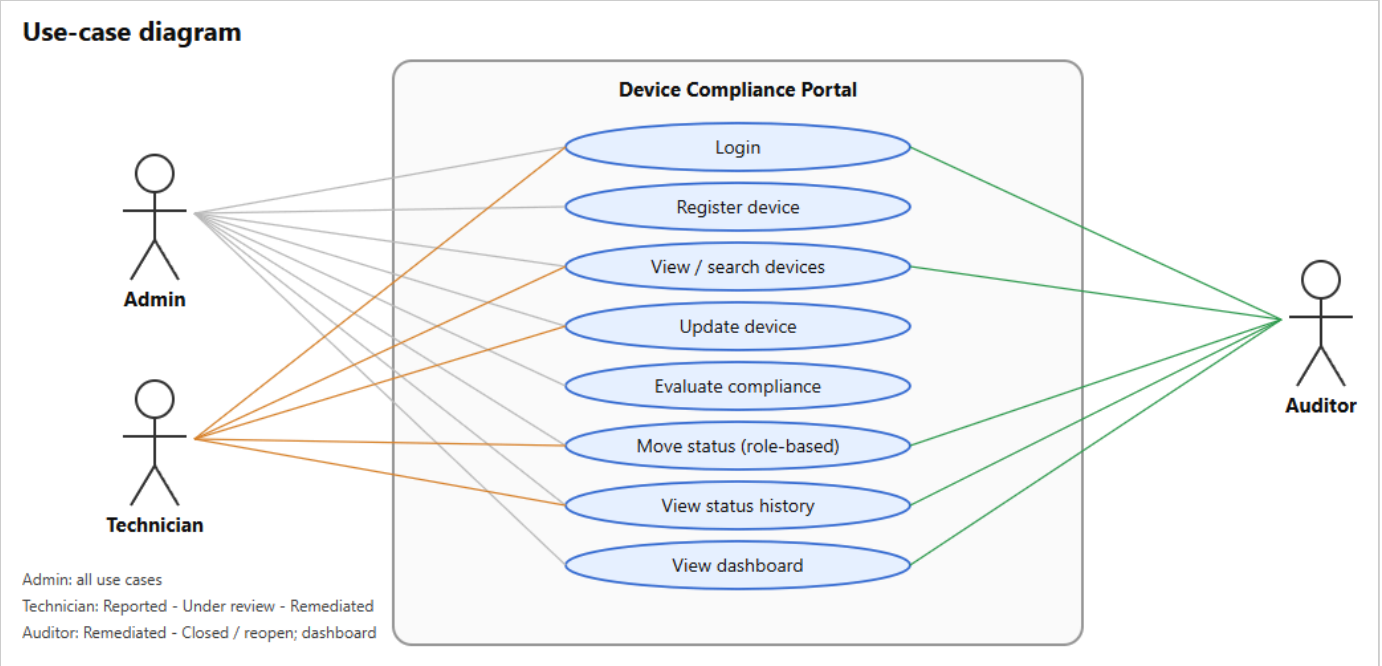


Figure 3.1 - Use-case diagram

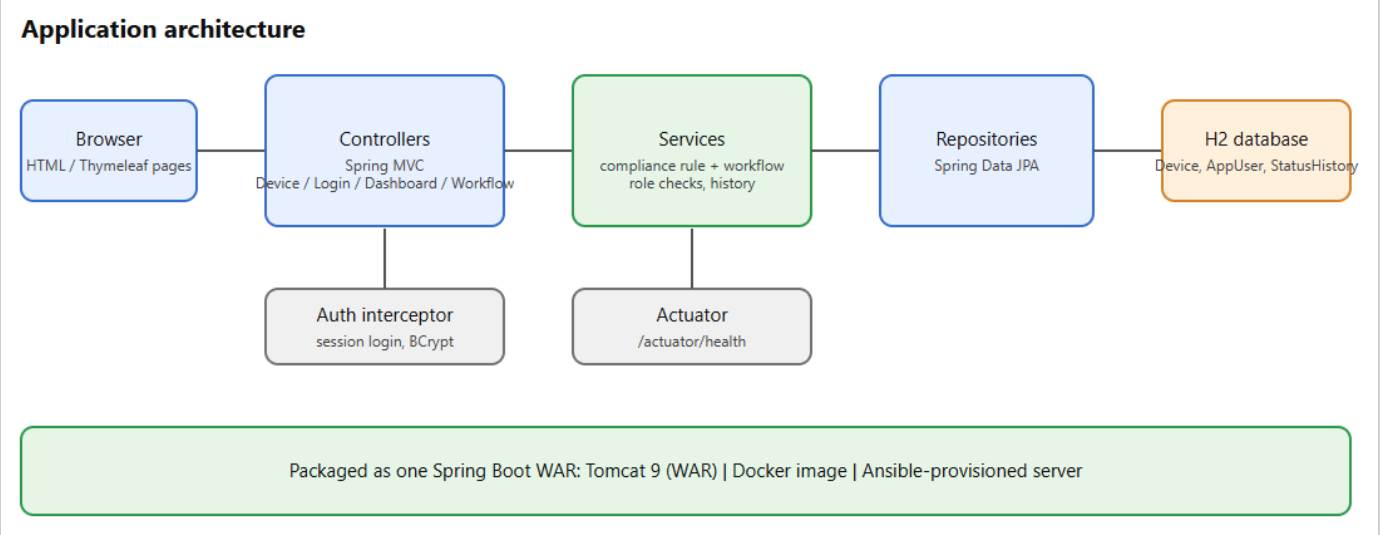


Figure 3.2 - Architecture diagram

5. Data Model

Device(id, name, owner, serialNo, os, osVersion, antivirusEnabled, diskEncrypted, firewallEnabled, complianceState, workflowStatus, createdAt, updatedAt) **AppUser**(id, username, passwordHash, role) **StatusHistory**(id, deviceId, fromStatus, toStatus, changedBy, changedAt) Relations: Device 1—* StatusHistory.

6. API / Route List

Method	Path	Purpose
GET	/login	login page
GET	/devices	list + search (`?q=&status=`)
GET	/devices/new · POST /devices	create
GET	/devices/{id}	details
GET	/devices/{id}/edit · POST /devices/{id}	update
POST	/devices/{id}/status	workflow transition
GET	/dashboard	summary
GET	/actuator/health	health check

```
Windows PowerShell / Git Bash - local build and tests

$ java -version
openjdk version "11.0.32.1" 2026-08-18

$ mvn -v
Apache Maven 3.9.9 (8e8579a9e76f7d015ee5ec7bfcdc97d260186937)
Maven home: C:\Users\damal\tools\apache-maven-3.9.9
Java version: 11.0.32.1, vendor: Eclipse Adoptium, runtime: C:\Program Files\Eclipse Adoptium\jdk-11.0.32.101-hotspot

$ mvn clean package
[INFO] Building Device Compliance Management Portal 0.1.0-SNAPSHOT
[INFO] --------------------------------[ war ]---------------------------------
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 25.622 s - in com.dcmp.portal.DeviceControllerTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.737 s - in com.dcmp.portal.PortalApplicationTests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.314 s - in com.dcmp.portal.SearchUpdateDashboardTests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.015 s - in com.dcmp.portal.WorkflowTests
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- war:3.3.2:war (default-war) @ device-compliance-portal ---
[INFO] Processing war project
[INFO] Building war: C:\Users\damal\Downloads\device-compliance-portal\target\device-compliance-portal-0.1.0-SNAPSHOT.war
[INFO] BUILD SUCCESS
[INFO] Total time:  53.026 s

$ ls -l target/*.war
44390667 target/device-compliance-portal-0.1.0-SNAPSHOT.war
```

Figure 3.3 - Local setup: JDK, Maven, `mvn clean package` (11 tests pass, WAR produced)

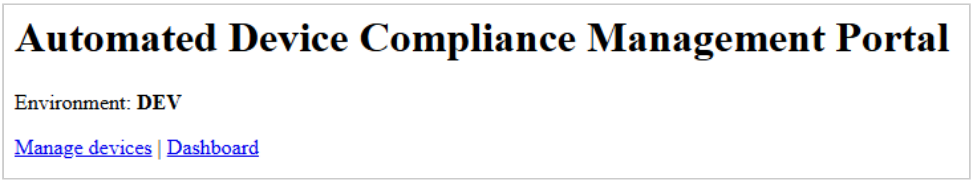


Figure 3.4 - Application running (home page) and health endpoint below



Figure 3.5 - /actuator/health returns UP

STEP 4

Git and GitHub Repository Initialization

Task given: Create a GitHub repository for the Device Compliance Management Portal. Add README, .gitignore, folder structure, issue templates and branch naming rules. Commit the initial application skeleton using meaningful commit messages.

Deliverable required: Repository URL, README, initial commits, issues and branch policy.

Public repository: github.com/Athdam05/device-compliance-portal.

Deliverable required by the sheet	Shown in this report
Repository URL	above and Figure 4.1
README	Figure 4.2 and text below
Initial commits	Figures 4.3-4.4
Issues	Figures 4.5-4.6 (issue templates and created issues)
Branch policy	README section below, Figure 4.7 (branches)

device-compliance-portal

Public

Notifications Fork 0 Star 0

main 13 Branches 1 Tag Go to file Code

Athdam05 Merge pull request #5 from Athdam05/release/v1.0.0 a1bdd41 · 4 days ago 22 Commits

.github/ISSUE_TEMPLATE	docs: add README, issue templates and branch policy	4 days ago
docs	docs: update backlog and record MVP completion for v1....	4 days ago
src	merge: resolve list.html conflict between search UI and lo...	4 days ago
.gitignore	chore: add Maven project skeleton with Spring Boot app	4 days ago
README.md	docs: update backlog and record MVP completion for v1....	4 days ago
pom.xml	feat: add users, BCrypt login and session auth interceptor	4 days ago

README

Automated Device Compliance Management Portal

DevOps mini project – **Atharva Anand Damale (23102C0004)**, BE CMPN C, Academic Year 2026-27.

A web portal to register devices, evaluate them against compliance rules (OS, antivirus, disk encryption, firewall), and manage remediation through a role-based status workflow with a summary dashboard.

Tech stack

Java 11 · Spring Boot 2.7 · Thymeleaf · H2 · Maven · Tomcat/Docker · Jenkins · Selenium · Ansible

Run locally

```
mvn clean package
mvn spring-boot:run
# http://localhost:8080  |  health: /actuator/health
```

Repository structure

```
src/main/java      application code
src/main/resources templates + config
src/test           unit / Selenium tests
docs/              step-wise project documentation
.github/           issue templates
```

Branch policy

Branch	Purpose
`main`	stable, tagged releases only
`develop`	integration branch; PRs merge here
`feature/<issue#>-<short-name>`	new features (e.g. `feature/3-create-device`)
`bugfix/<issue#>-<short-name>`	bug fixes
`hotfix/<short-name>`	urgent fixes from `main`

Rules: no direct pushes to `main` / `develop`; every change via pull request with review; commit messages follow `type: summary` (`feat`, `fix`, `docs`, `test`, `chore`, `ci`).

Documentation

1. Problem definition & scope
2. Agile planning & DevOps workflow

About

DevOps mini project

Readme
Activity
0 stars
0 watching
0 forks
Report repository

Releases

1 tag

Contributors 1

Athdam05 Atharva Damale

Languages

Java 79.9% HTML 20.1%

3. Requirements, architecture & setup
4. MVP completion & Git collaboration

Figure 4.1 - GitHub repository (files, branches, README)

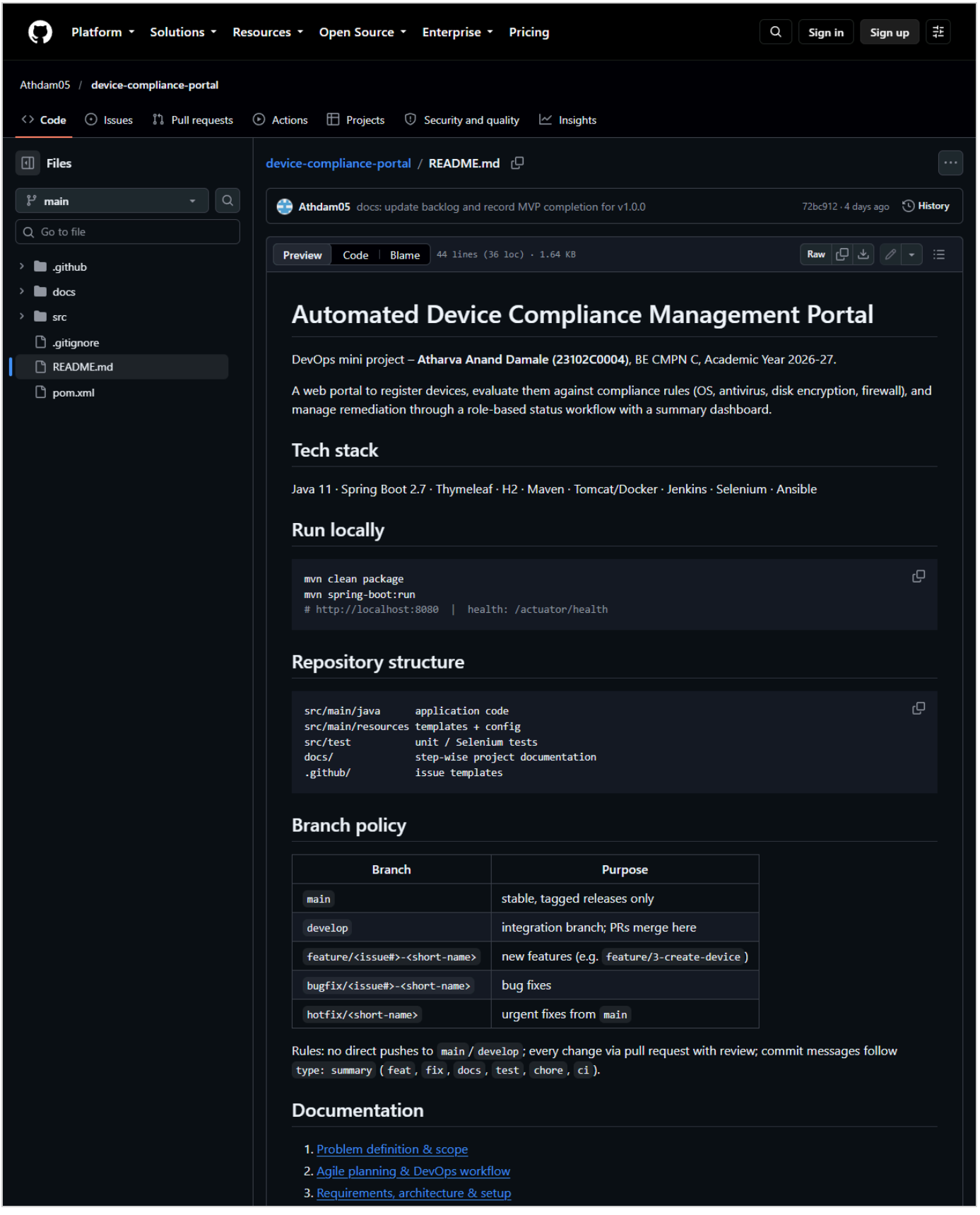


Figure 4.2 - README rendered on GitHub

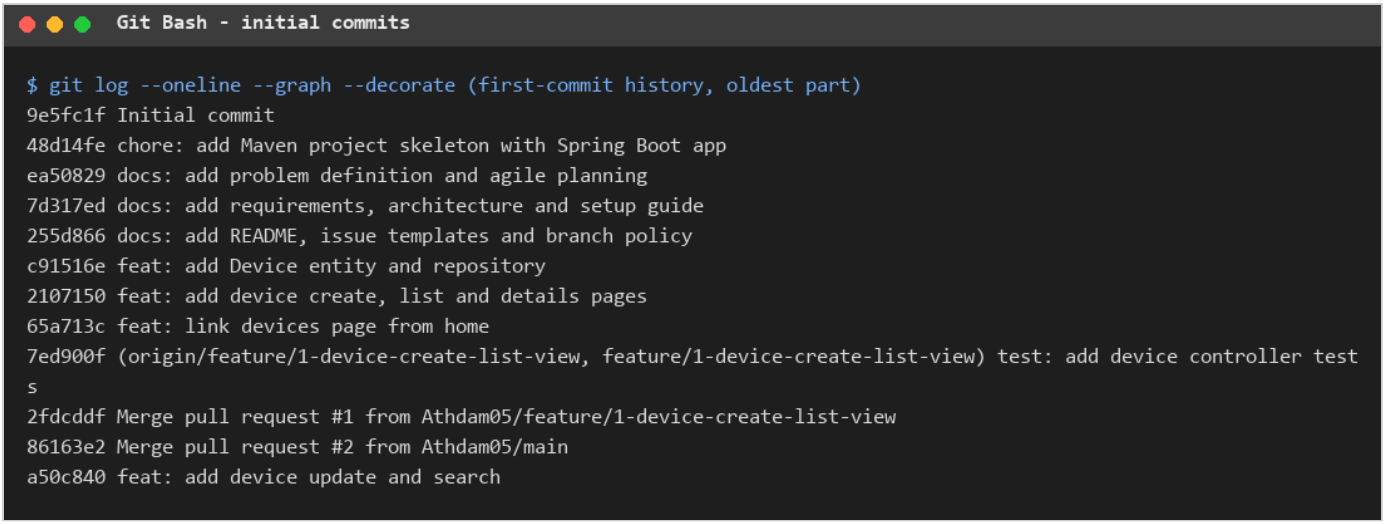


Figure 4.3 - Initial commits with meaningful messages

Commits

main All users All time

Commits on Oct 17, 2026

Merge pull request #5 from Athdam05/release/v1.0.0 Athdam05 authored 4 days ago	Verified	a1bdd41
docs: update backlog and record MVP completion for v1.0.0 Athdam05 committed 4 days ago		72bc912
Merge pull request #4 from Athdam05/feature/3-roles-workflow Athdam05 authored 4 days ago	Verified	2c07e8f
merge: resolve list.html conflict between search UI and login header Athdam05 committed 4 days ago		bb8f57f
Merge pull request #3 from Athdam05/feature/2-update-search-dashboard Athdam05 authored 4 days ago	Verified	57ccaf1
feat: add move-status UI, history table and login-aware list header; update tests Athdam05 committed 4 days ago		8469906
feat: add status workflow service with role rules and history Athdam05 committed 4 days ago		e2f237b
feat: add users, BCrypt login and session auth interceptor Athdam05 committed 4 days ago		a6a5149
feat: add search form and edit link to device list; add tests Athdam05 committed 4 days ago		4a12723
feat: add compliance summary dashboard Athdam05 committed 4 days ago		10ceee0
feat: add device update and search Athdam05 committed 4 days ago		a50c840
Merge pull request #2 from Athdam05/main Athdam05 authored 4 days ago	Verified	86163e2
Merge pull request #1 from Athdam05/feature/1-device-create-list-view Athdam05 authored 4 days ago	Verified	2fdcddf
test: add device controller tests Athdam05 committed 4 days ago		7ed900f
feat: link devices page from home Athdam05 committed 4 days ago		65a713c
feat: add device create, list and details pages Athdam05 committed 4 days ago		2107150
feat: add Device entity and repository Athdam05 committed 4 days ago		c91516e
docs: add README, issue templates and branch policy Athdam05 committed 4 days ago		255d866
docs: add requirements, architecture and setup guide Athdam05 committed 4 days ago		7d317ed
docs: add problem definition and agile planning Athdam05 committed 4 days ago		ea50829
chore: add Maven project skeleton with Spring Boot app Athdam05 committed 4 days ago		48d14fe
Initial commit Athdam05 authored 4 days ago	Verified	9e5fc1f

Figure 4.4 - Commit history on GitHub

Issue templates (.github/ISSUE_TEMPLATE)

feature_request.md

```
---
name: Feature / user story
about: Propose a feature or MVP task
labels: enhancement
---
**User story**
As a <role>, I want <goal> so that <benefit>.

**Acceptance criteria**
- [ ]

**Branch:** `feature/<issue#>-<name>`
```

bug_report.md

```
---
name: Bug report
about: Report a defect
labels: bug
---
**Steps to reproduce**

**Expected result**

**Actual result**

**Environment / logs**
```

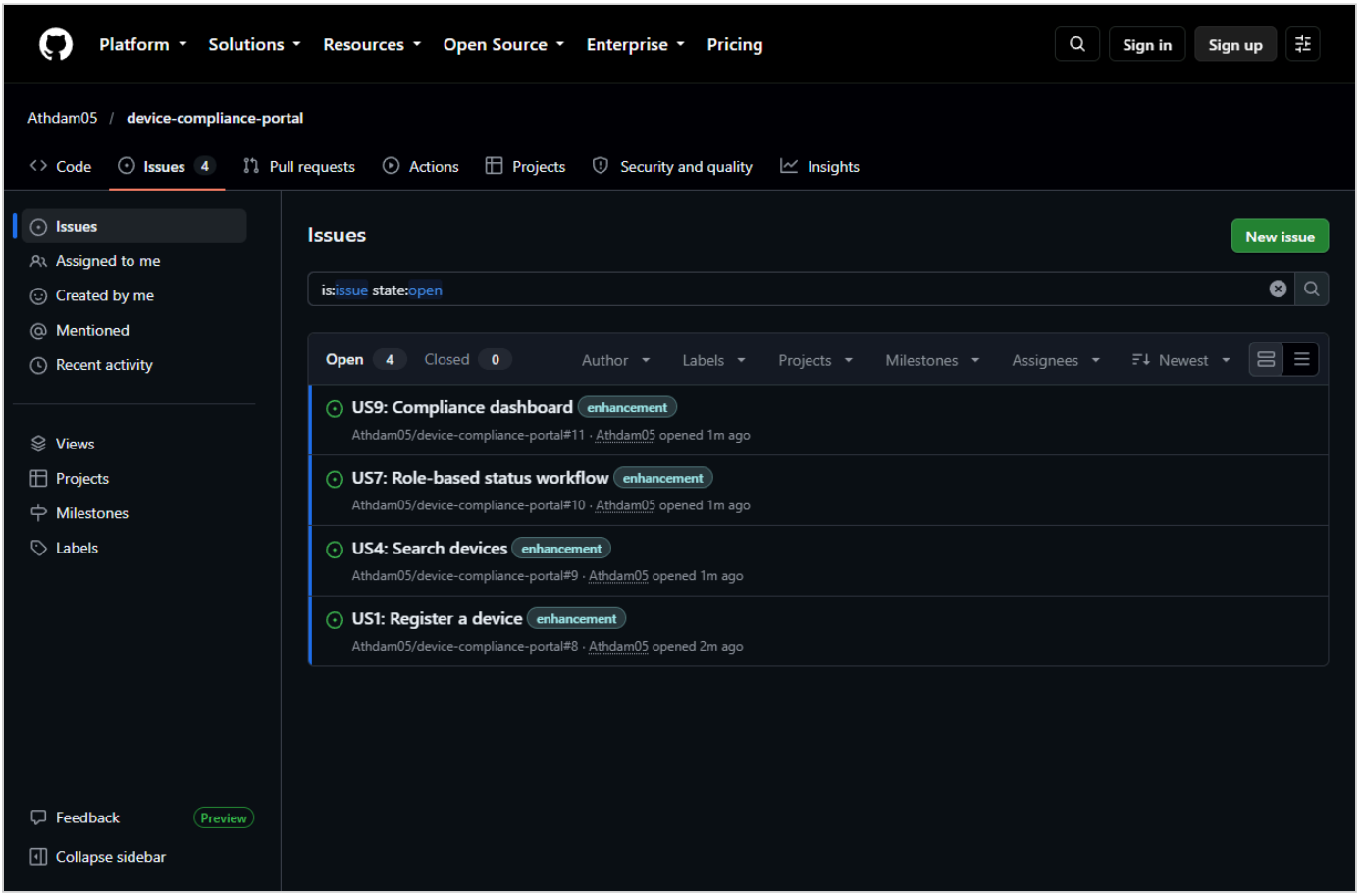


Figure 4.5 - Issues created from the user stories

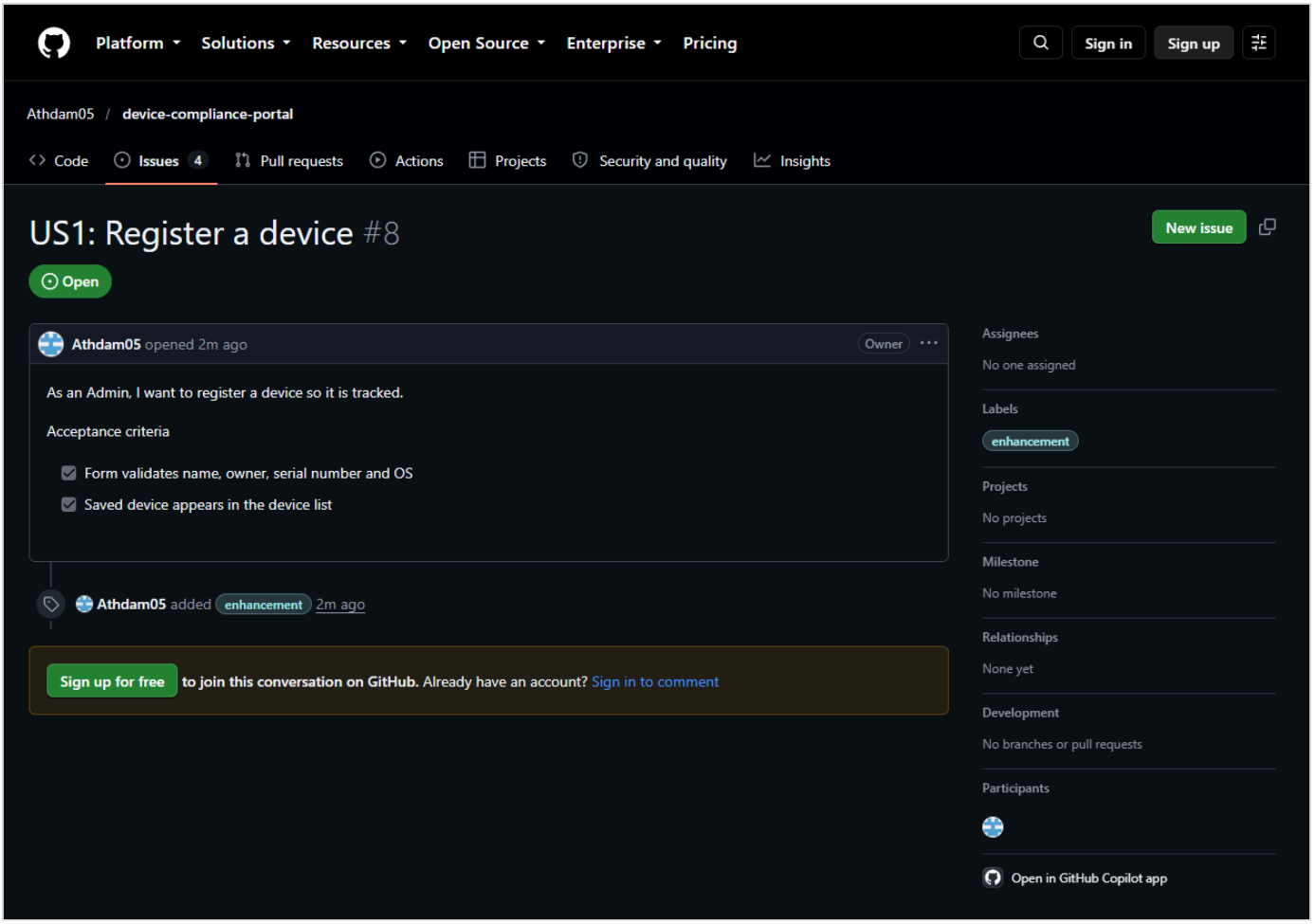


Figure 4.6 - Example issue with acceptance criteria

Branch naming and policy (from README)

Branch policy

Branch	Purpose
`main`	stable, tagged releases only
`develop`	integration branch; PRs merge here
`feature/<issue#>-<short-name>`	new features (e.g. `feature/3-create-device`)
`bugfix/<issue#>-<short-name>`	bug fixes
`hotfix/<short-name>`	urgent fixes from `main`

Rules: no direct pushes to `main` / `develop`; every change via pull request with review; commit messages follow `type: summary` (`feat`, `fix`, `docs`, `test`, `chore`, `ci`).

Git Bash - branches and tags

```
$ git branch -a
  develop
  feature/1-device-create-list-view
* feature/10-provisioning-rollback
  feature/2-update-search-dashboard
  feature/3-roles-workflow
  feature/4-jenkinsfile-tomcat-deploy
  feature/5-selenium-tests
  feature/6-selenium-in-jenkins
  feature/7-docker
  feature/8-docker-cd
  feature/9-ansible-config
  main
  release/v1.0.0
  remotes/origin/HEAD -> origin/main
  remotes/origin/develop
  remotes/origin/feature/1-device-create-list-view
  remotes/origin/feature/10-provisioning-rollback
  remotes/origin/feature/2-update-search-dashboard
  remotes/origin/feature/3-roles-workflow
  remotes/origin/feature/4-jenkinsfile-tomcat-deploy
  remotes/origin/feature/5-selenium-tests
  remotes/origin/feature/6-selenium-in-jenkins
  remotes/origin/feature/7-docker
  remotes/origin/feature/8-docker-cd
  remotes/origin/feature/9-ansible-config
  remotes/origin/main
  remotes/origin/release/v1.0.0

$ git tag -n
v1.0.0          v1.0.0 - MVP baseline: device CRUD, search, dashboard, roles, status workflow
```

Figure 4.7 - Branches and tag in the repository

STEP 5

Feature Development with Branching

Task given: Implement the first core workflow of the Device Compliance Management Portal on a feature branch. Use add, commit, push, pull and logs. Raise a pull request, review changes and merge into the development branch.

Deliverable required: Working feature 1, feature branch, pull request, review comments and merge evidence.

Feature 1 (Device entity, create, list and details) was built on branch `feature/1-device-create-list-view`, reviewed and merged through pull requests.

Deliverable required by the sheet	Shown in this report
Working feature 1	Figures 5.1-5.4 (screens of the running feature)
Feature branch	Figure 5.5 (git log / branch), 4 commits
Pull request	Figures 5.6-5.7 (PR #1 into develop/main, PR #2)
Review comments	Figure 5.8
Merge evidence	Figures 5.6-5.7 (Merged badge, merge commit)

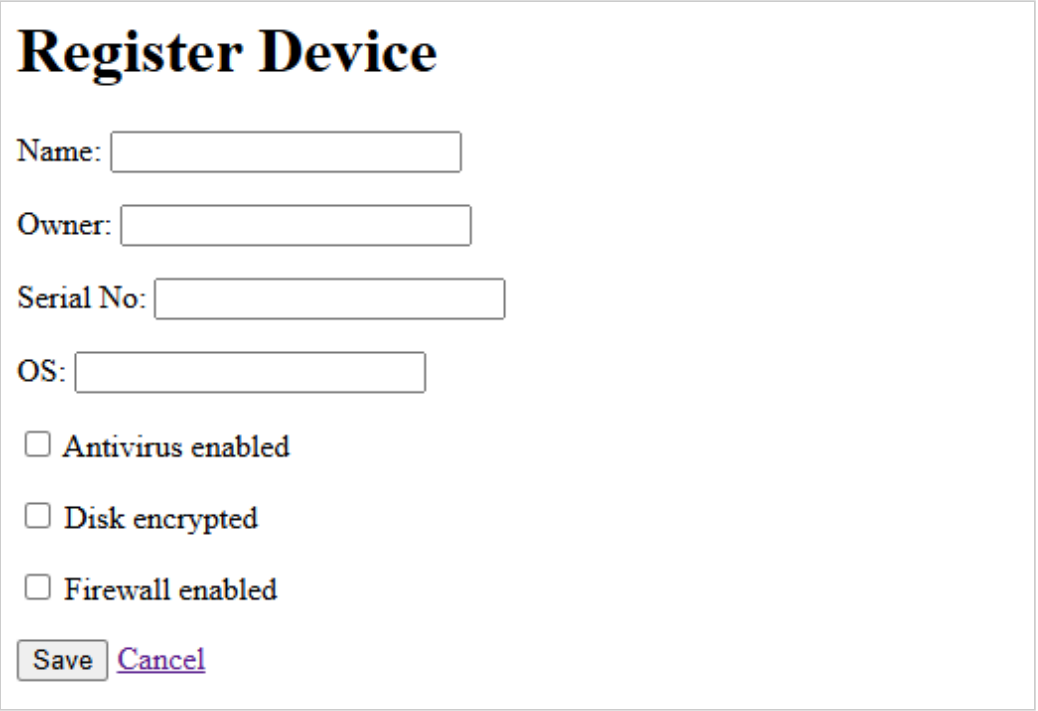


Figure 5.1 - Create device form

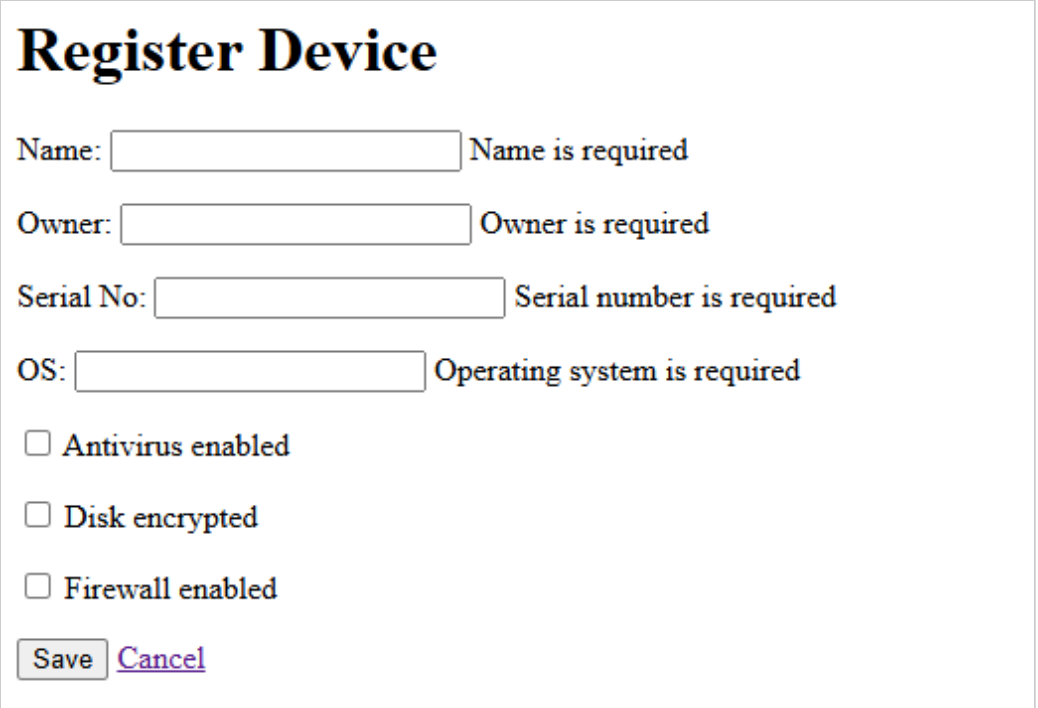


Figure 5.2 - Validation errors on an empty form

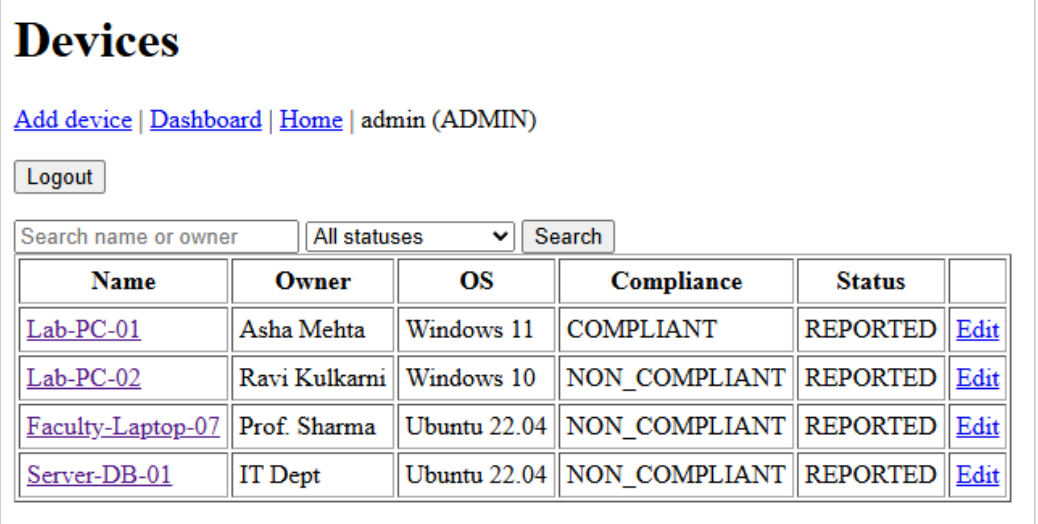


Figure 5.3 - Device list (4 devices; compliance computed from the 3 controls)



Figure 5.4 - Device details page

```
Git Bash - feature branch commits, pull/merge activity, remote

$ git log --oneline --graph feature/1-device-create-list-view   (add / commit / push on the feature branch)
* 7ed900f (origin/feature/1-device-create-list-view, feature/1-device-create-list-view) test: add device contr
* 65a713c feat: link devices page from home
* 2107150 feat: add device create, list and details pages
* c91516e feat: add Device entity and repository
* 255d866 docs: add README, issue templates and branch policy
* 7d317ed docs: add requirements, architecture and setup guide
* ea50829 docs: add problem definition and agile planning

$ git reflog | grep -E "pull|merge"   (pull / merge activity)
8da98f0 HEAD@{2026-10-17}: pull -q origin develop: Fast-forward
a1bdd41 HEAD@{2026-10-17}: pull -q origin main: Fast-forward
2c07e8f HEAD@{2026-10-17}: pull -q origin develop: Fast-forward
bb8f57f HEAD@{2026-10-17}: commit (merge): merge: resolve list.html conflict between search UI and login header
86163e2 HEAD@{2026-10-17}: pull -q origin develop: Fast-forward

$ git remote -v
origin  https://github.com/Athdam05/device-compliance-portal.git (fetch)
origin  https://github.com/Athdam05/device-compliance-portal.git (push)
```

Figure 5.5 - Feature branch commits, pull/merge activity and remote

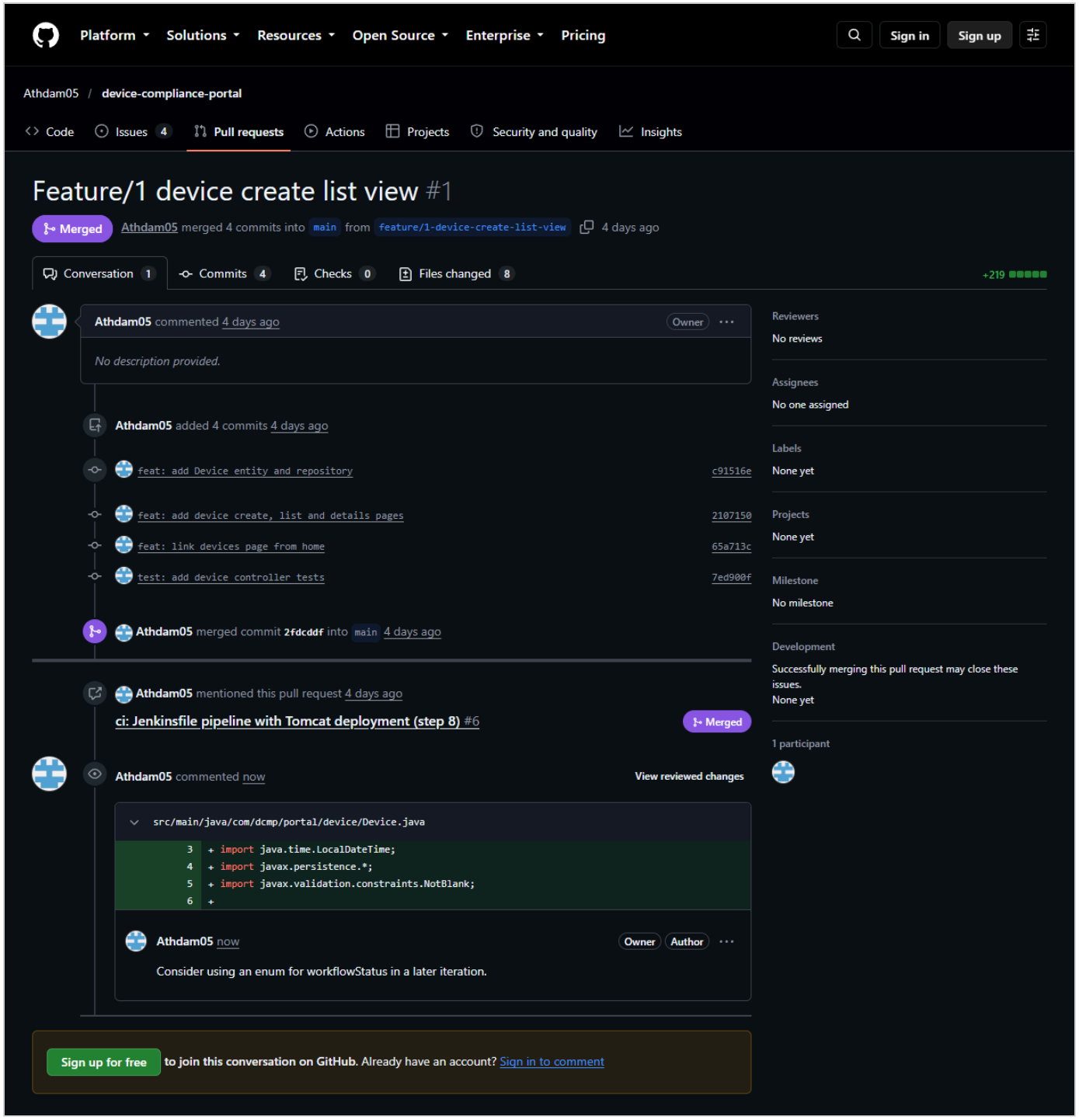


Figure 5.6 - Pull request #1 (feature branch) - merged

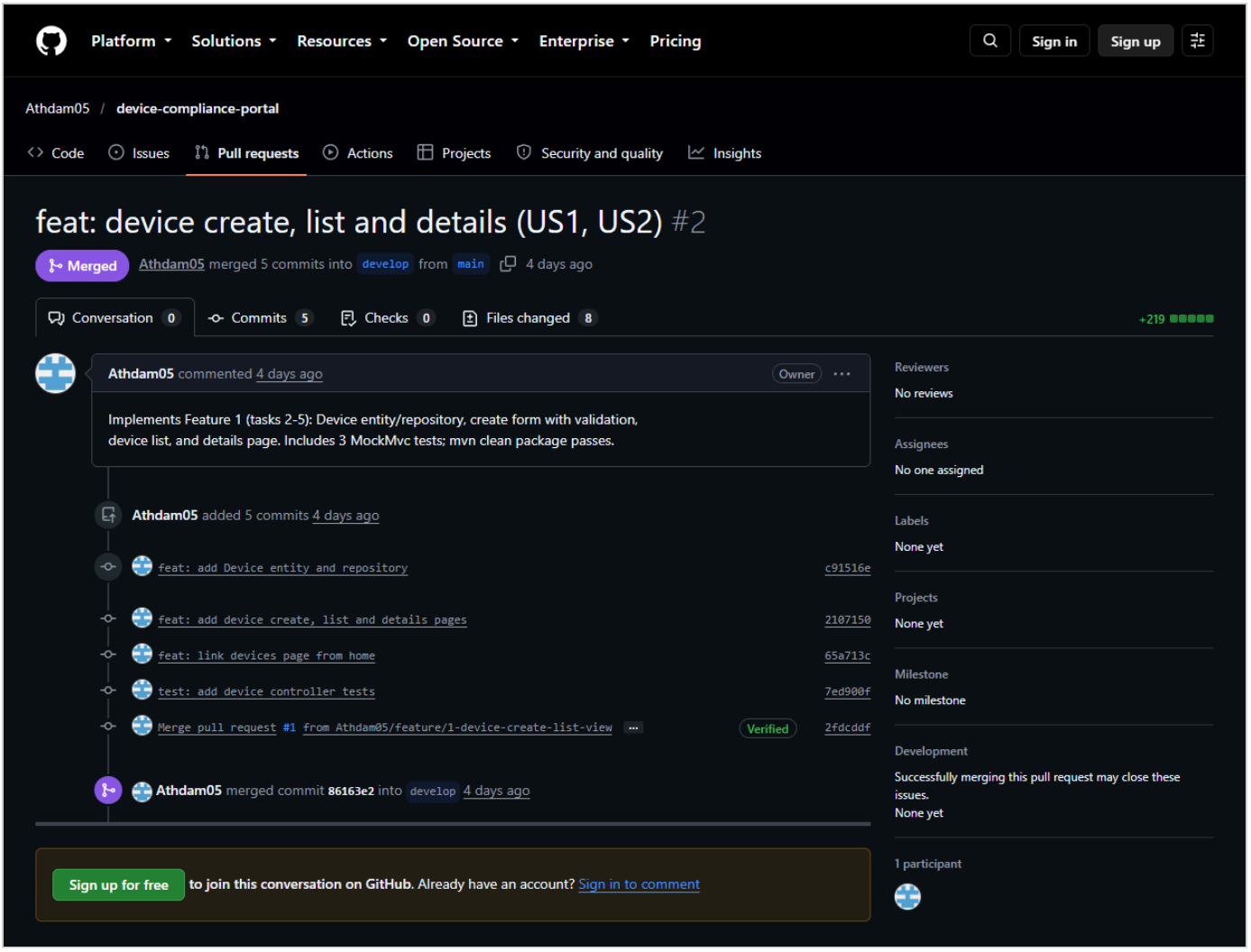


Figure 5.7 - Pull request #2 into develop - merged

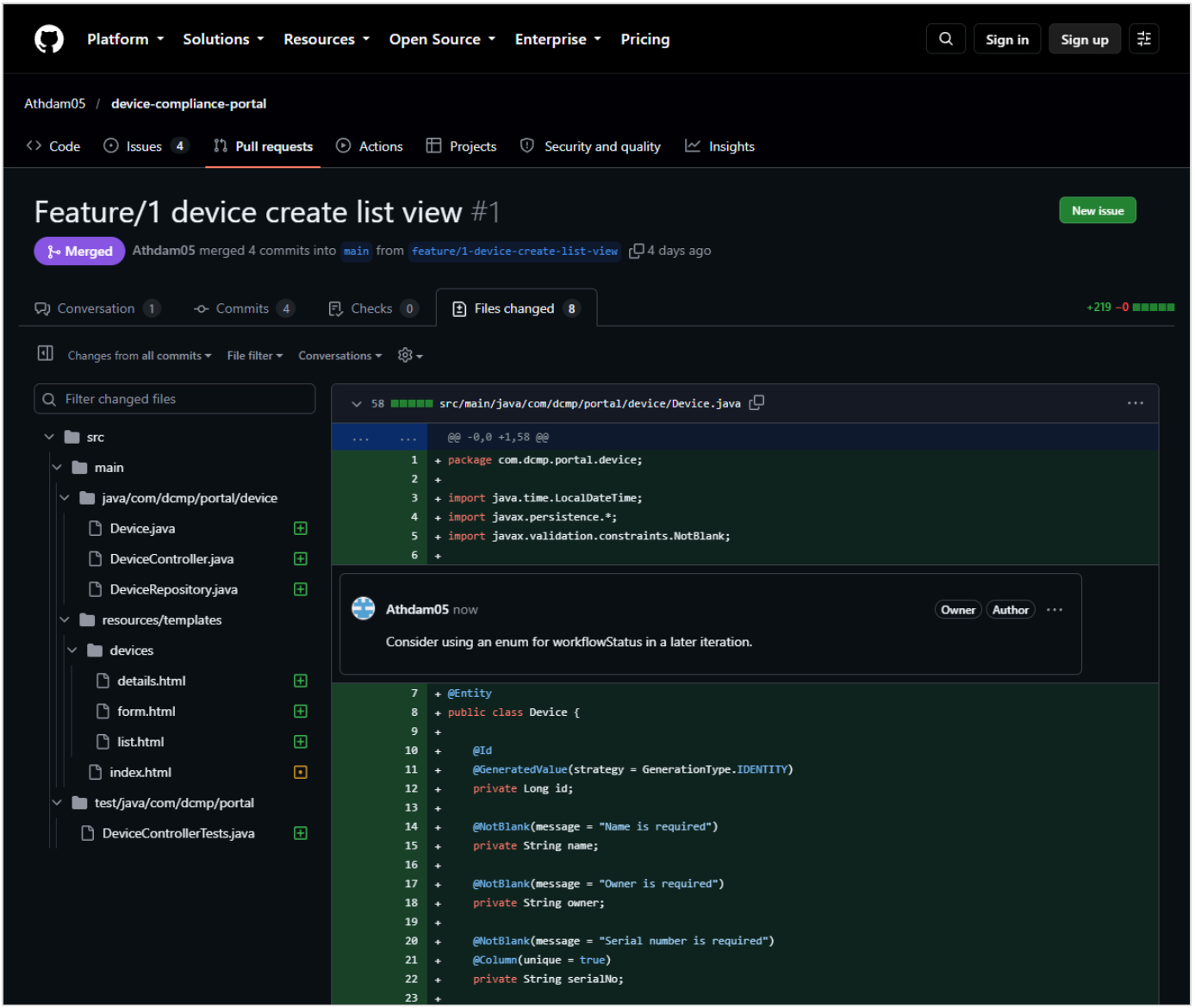


Figure 5.8 - Review comment on the feature-1 pull request

STEP 6

MVP Completion and Git Collaboration

> **Task given:** Implement the remaining essential MVP functions for the Device Compliance Management Portal: create, view, update and search records; role-based status workflow; summary dashboard. Demonstrate a second feature branch, conflict creation/resolution, tagging and a release-ready source baseline.

> **Deliverable required:** Functional MVP, resolved merge conflict, tagged version and updated backlog.

Remaining MVP functions were built on two further branches: `feature/2-update-search-dashboard` and `feature/3-roles-workflow`. A real merge conflict between them was created and resolved, and release **v1.0.0** was tagged.

Deliverable required by the sheet	Shown in this report
Functional MVP	Figures 6.1-6.8 (update, search, roles, workflow, history, dashboard)
Second feature branch	Figure 6.9 (PR #3 and PR #4)
Resolved merge conflict	Figures 6.10-6.11 (git log, resolution commit, PR #4)
Tagged version	Figures 6.12-6.13 (tag v1.0.0, release PR #5)
Updated backlog	Table below (docs/06)

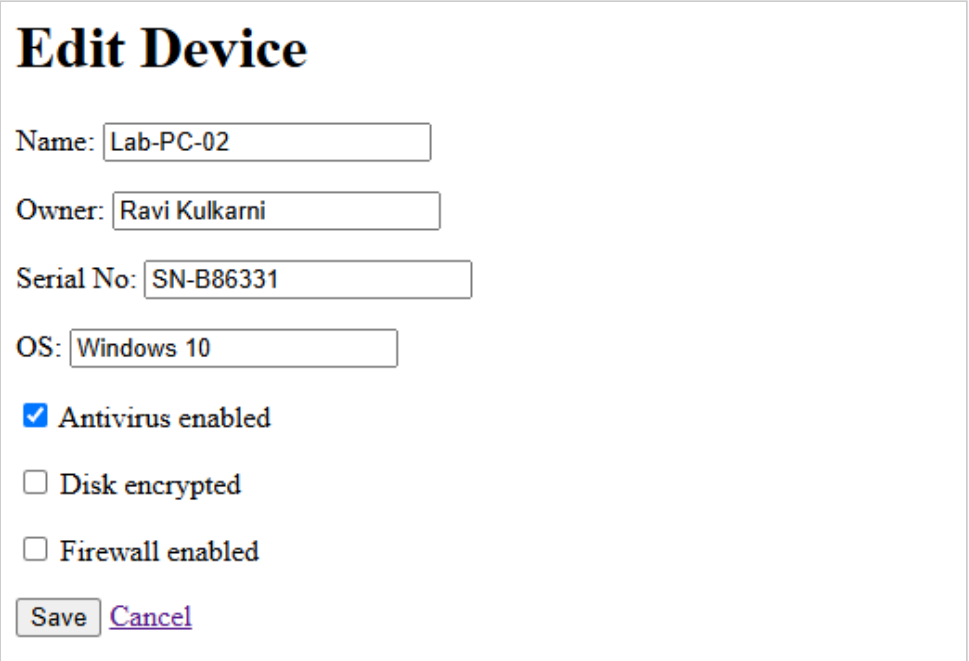


Figure 6.1 - Update device (edit form)

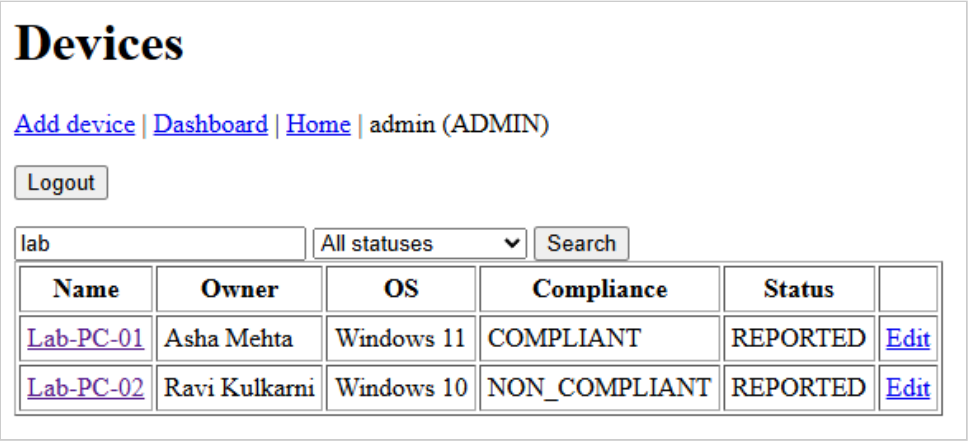


Figure 6.2 - Search for 'lab' (name/owner/status filter)

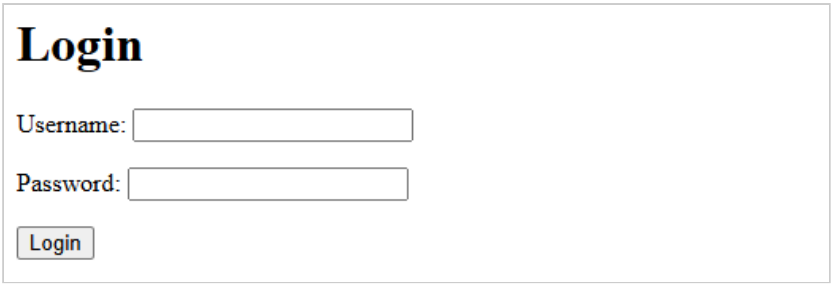


Figure 6.3 - Login page (roles ADMIN / AUDITOR / TECHNICIAN)

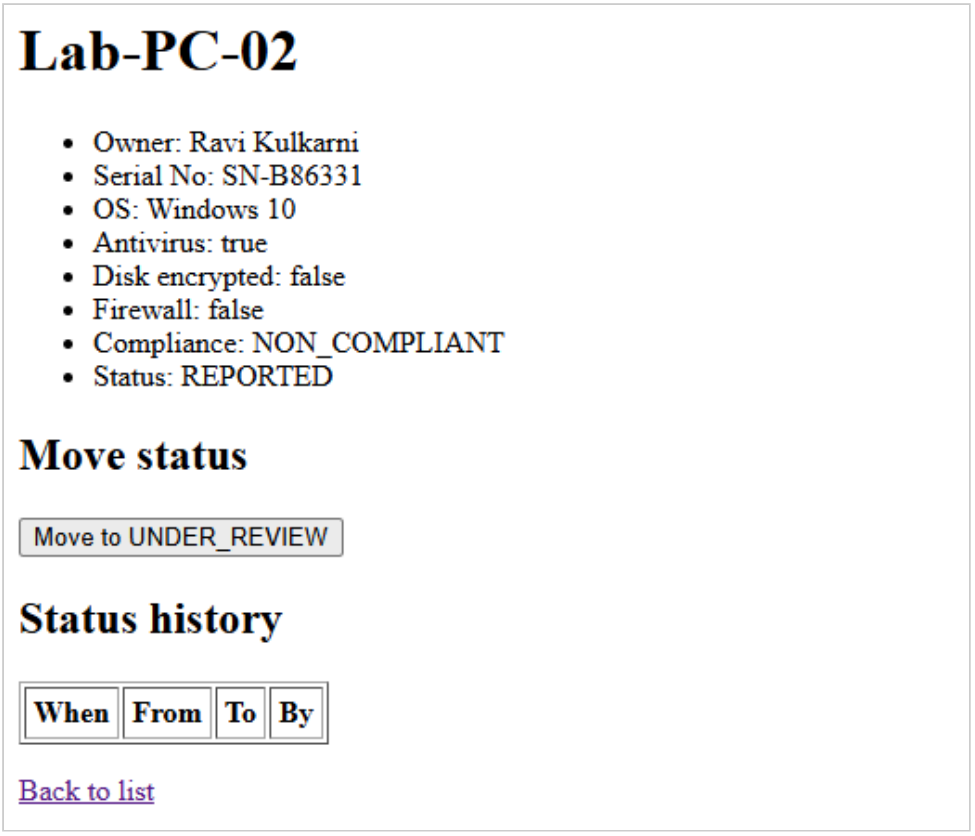


Figure 6.4 - Admin view of a device before any workflow action

Lab-PC-02

- Owner: Ravi Kulkarni
- Serial No: SN-B86331
- OS: Windows 10
- Antivirus: true
- Disk encrypted: false
- Firewall: false
- Compliance: NON_COMPLIANT
- Status: UNDER_REVIEW

Move status

Move to REMEDIATED

Status history

When	From	To	By
2026-10-21 12:38	REPORTED	UNDER_REVIEW	tech

Back to list

Figure 6.5 - Technician moves REPORTED -> UNDER_REVIEW

Lab-PC-02

- Owner: Ravi Kulkarni
- Serial No: SN-B86331
- OS: Windows 10
- Antivirus: true
- Disk encrypted: false
- Firewall: false
- Compliance: NON_COMPLIANT
- Status: REMEDIATED

Move status

No transitions available for your role.

Status history

When	From	To	By
2026-10-21 12:38	UNDER_REVIEW	REMEDIATED	tech
2026-10-21 12:38	REPORTED	UNDER_REVIEW	tech

Back to list

Figure 6.6 - Technician moves to REMEDIATED; history is recorded

Lab-PC-02

- Owner: Ravi Kulkarni
- Serial No: SN-B86331
- OS: Windows 10
- Antivirus: true
- Disk encrypted: false
- Firewall: false
- Compliance: NON_COMPLIANT
- Status: REMEDIATED

Move status

Move to UNDER_REVIEW | Move to CLOSED

Status history

When	From	To	By
2026-10-21 12:38	UNDER_REVIEW	REMEDIATED	tech
2026-10-21 12:38	REPORTED	UNDER_REVIEW	tech

Back to list

Figure 6.7 - Auditor sees the only allowed action: close

Compliance Dashboard

Devices | Home

- Total devices: 4
- Compliant: 1
- Non-compliant: 3

By workflow status

- Reported: 3
- Under review: 0
- Remediated: 0
- Closed: 1

Figure 6.8 - Summary dashboard

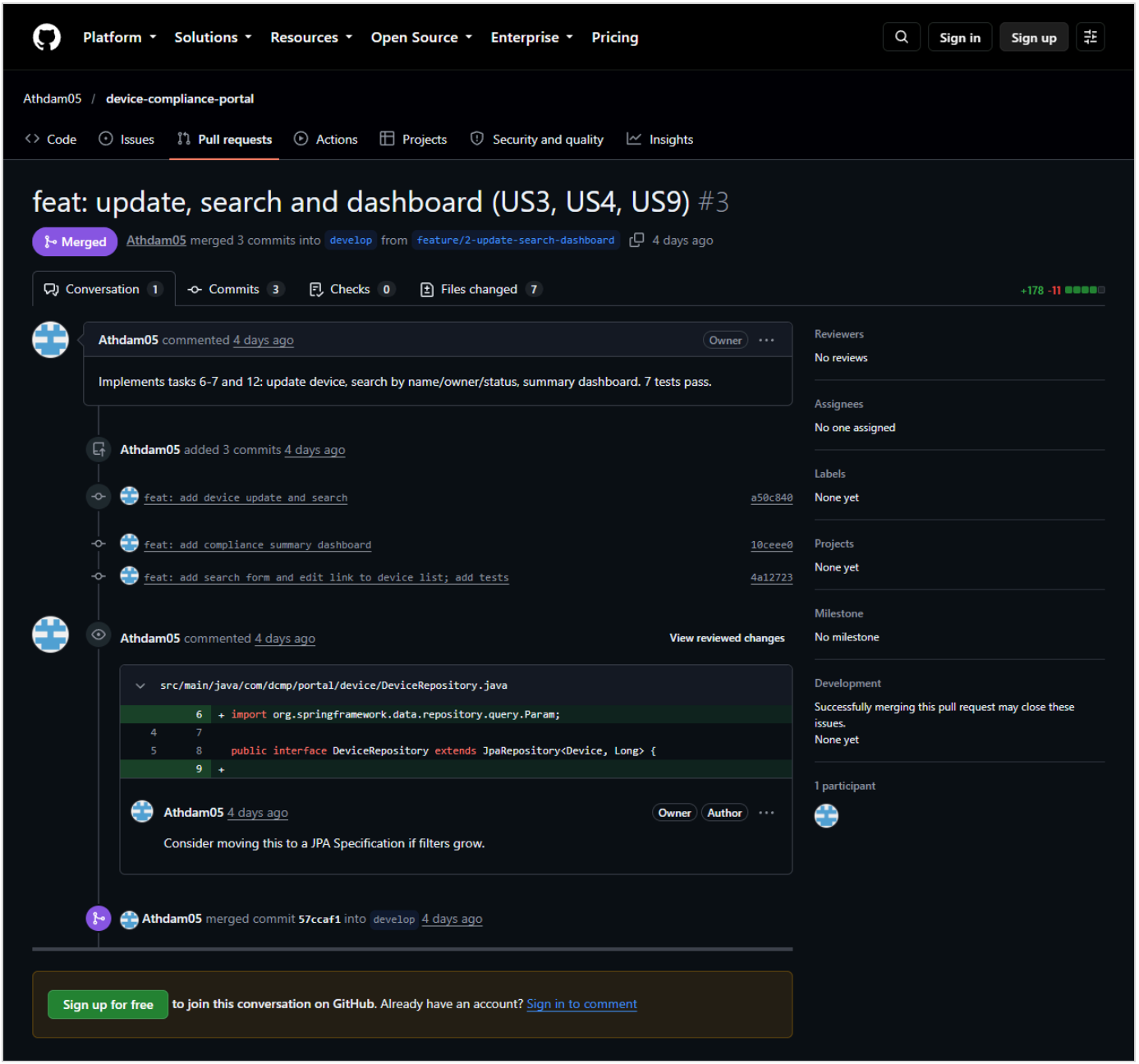


Figure 6.9a - PR #3 (feature/2) merged into develop

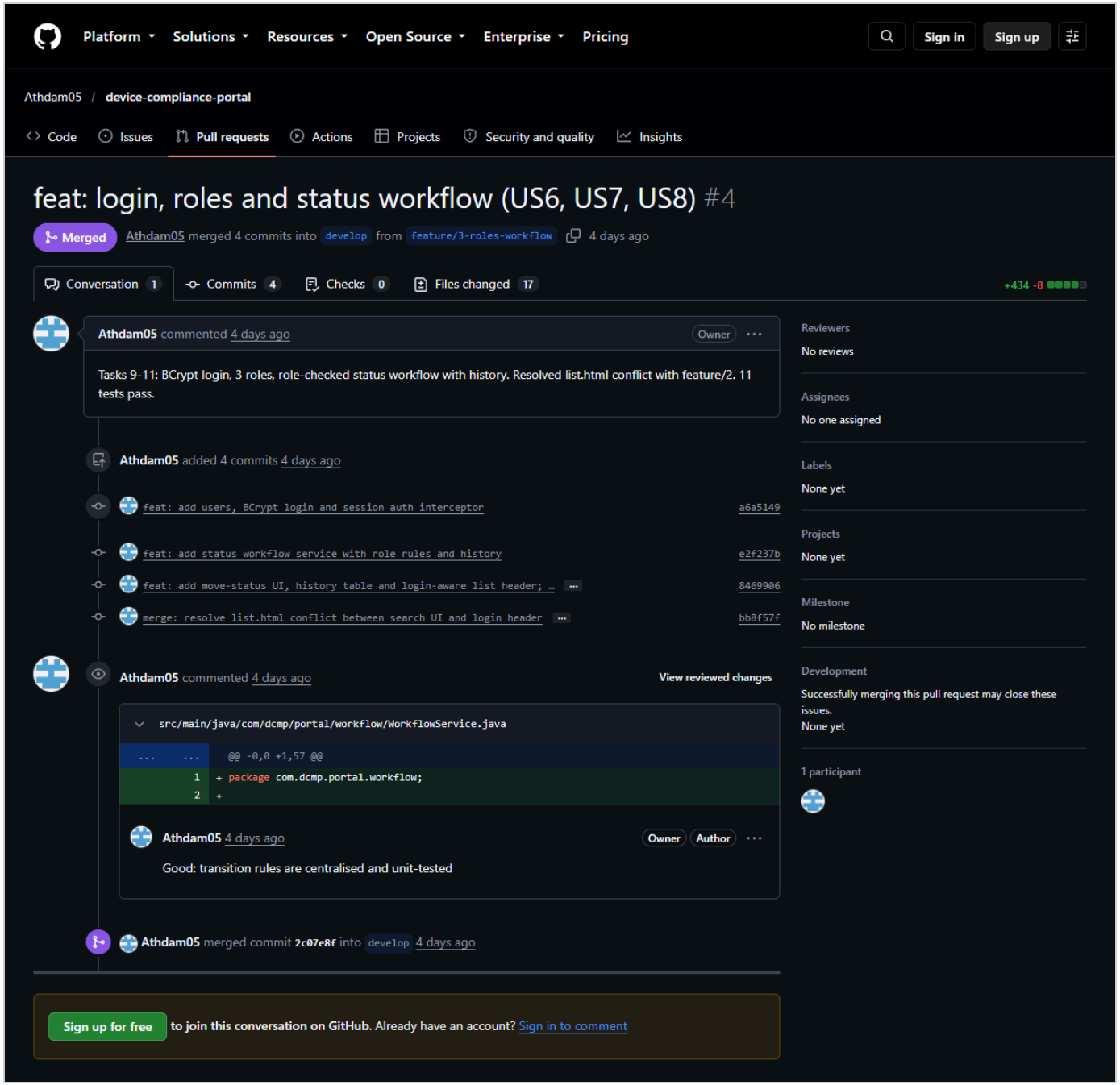


Figure 6.9b - PR #4 (feature/3) merged after conflict resolution

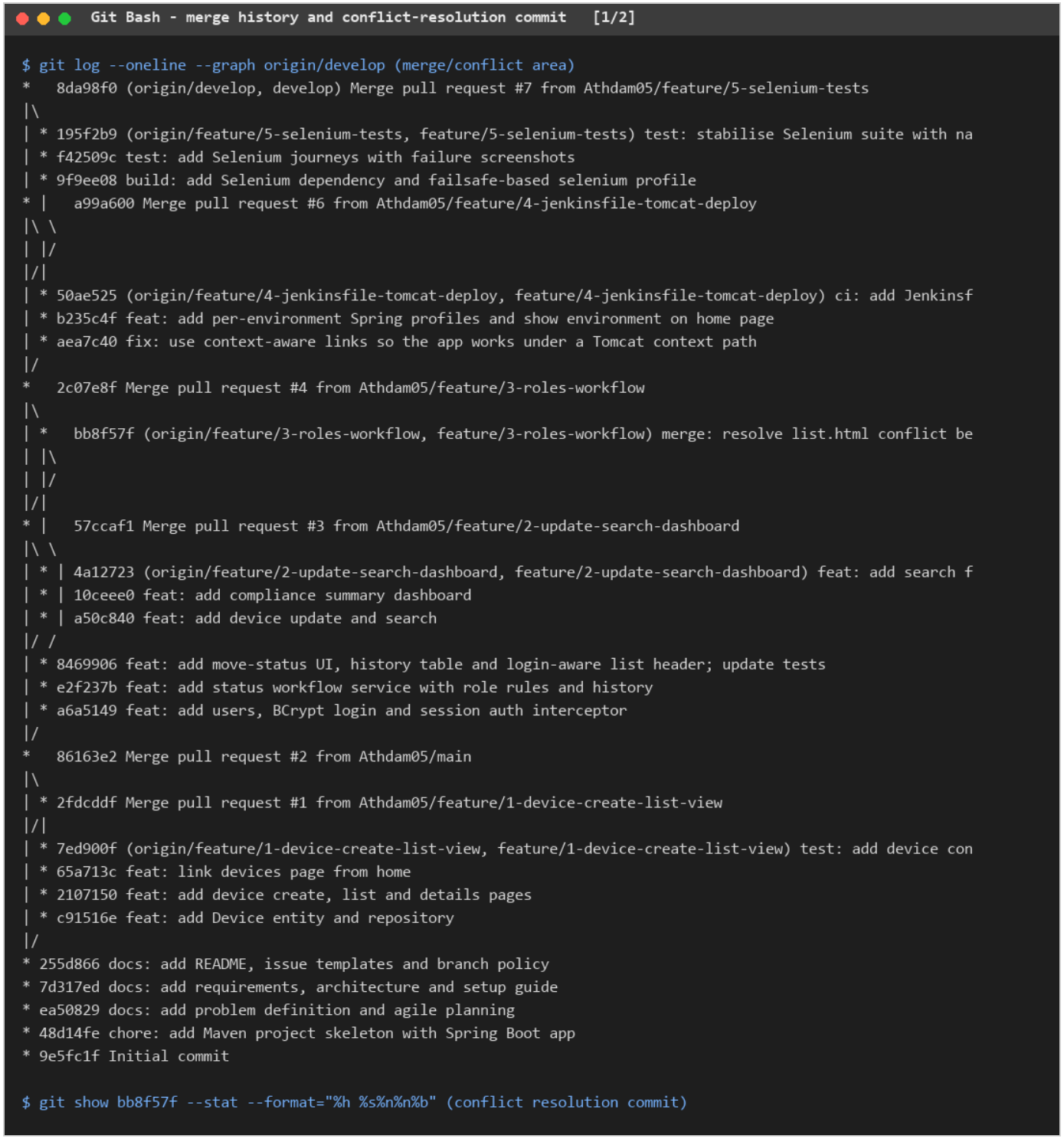


Figure 6.10 - Merge history and the conflict-resolution commit bb8f57f (part 1)

```
Git Bash - merge history and conflict-resolution commit   [2/2]

bb8f57f merge: resolve list.html conflict between search UI and login header

Keep the Dashboard link and search form from feature/2 together with the
logged-in user and logout button from feature/3. Update feature/2 tests to
authenticate now that the login interceptor is in place.


 .../java/com/dcmp/portal/DashboardController.java  | 32 ++++++++++
 .../com/dcmp/portal/device/DeviceController.java   | 46 ++++++++++++++--
 .../com/dcmp/portal/device/DeviceRepository.java   | 11 ++++
 src/main/resources/templates/dashboard.html        | 20 +++++++
 src/main/resources/templates/devices/form.html     |  7 ++-
 src/main/resources/templates/devices/list.html     | 16 ++++++-
 .../dcmp/portal/SearchUpdateDashboardTests.java    | 65 ++++++++++++++++++++++
```

Figure 6.10 - Merge history and the conflict-resolution commit bb8f57f (part 2)

The conflict

Both branches edited the header line of `src/main/resources/templates/devices/list.html` (feature/2 added the Dashboard link and search form; feature/3 added the logged-in user and Logout). Merging `origin/develop` into feature/3 reported `CONFLICT (content): Merge conflict in src/main/resources/templates/devices/list.html`; the resolution kept both changes and the feature/2 tests were updated to log in. 11 tests passed afterwards.

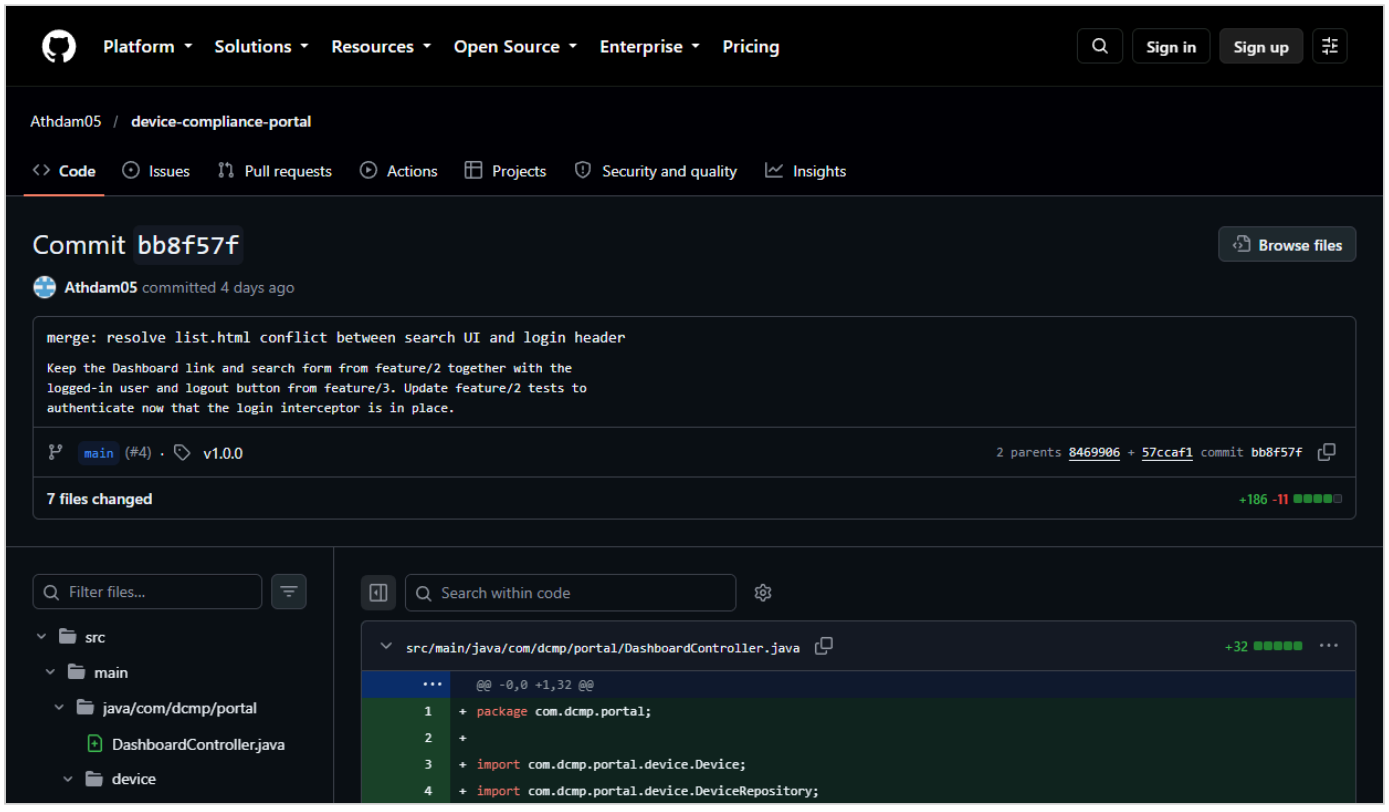


Figure 6.11 - Resolution commit on GitHub

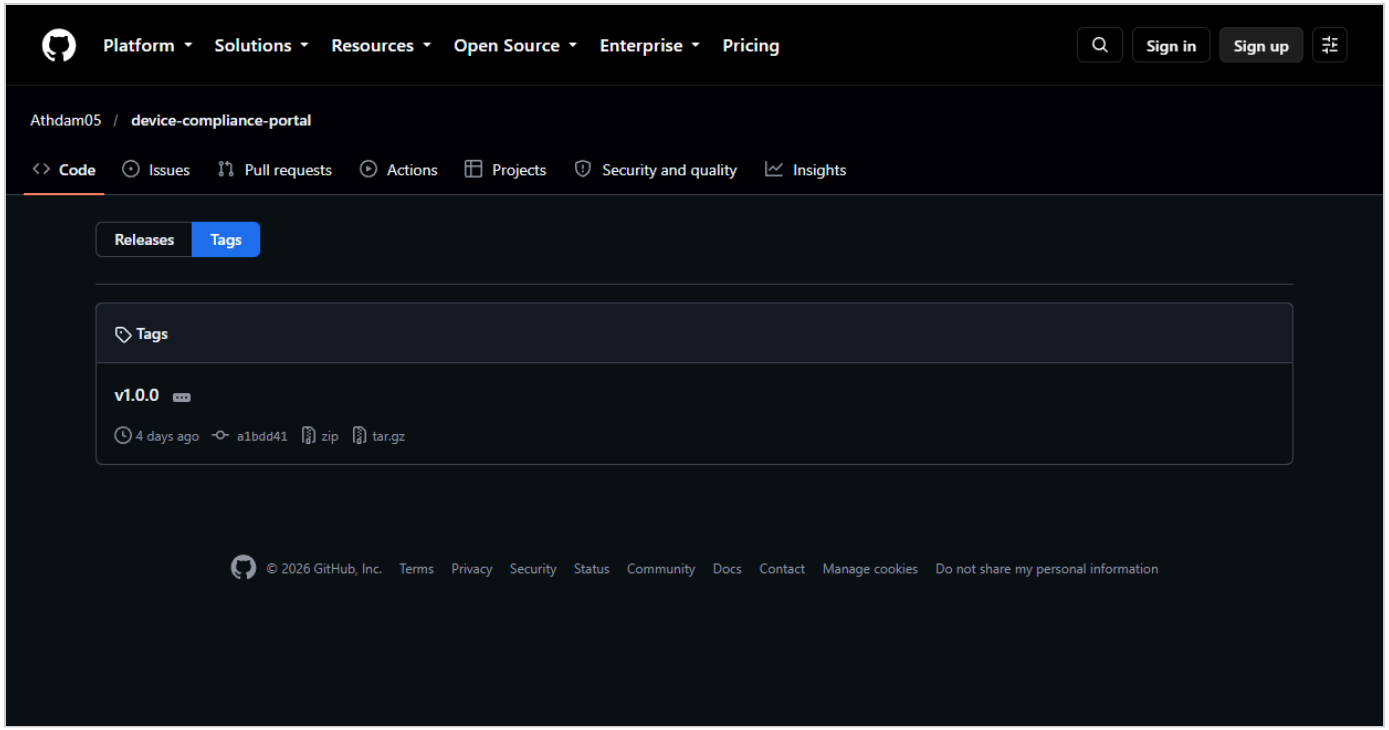


Figure 6.12 - Tag v1.0.0

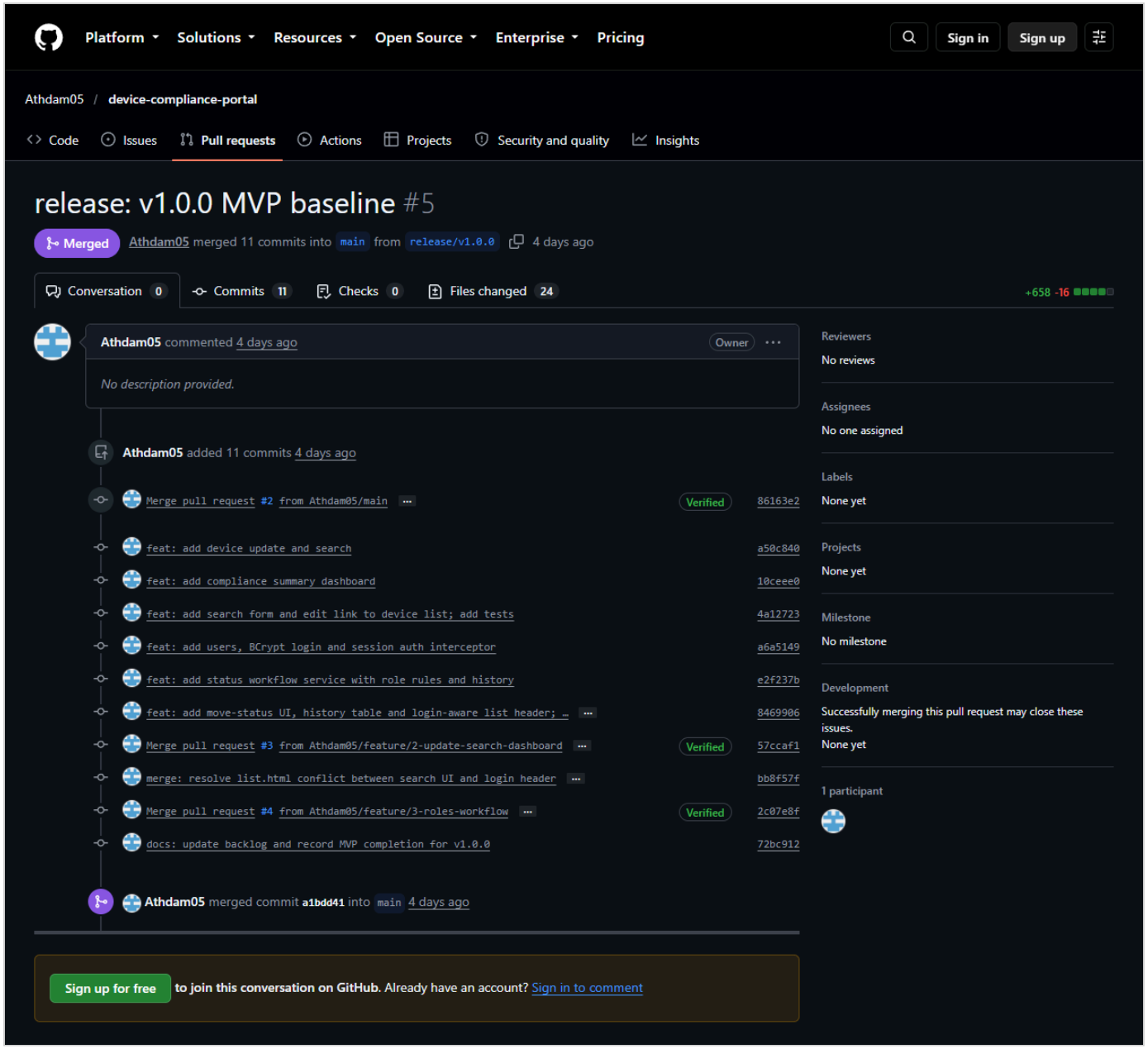


Figure 6.13 - Release PR #5 into main

Updated backlog (end of sprint 3)

1. Updated Backlog (end of Sprint 3)

#	Task	Status	Delivered in
1	Project skeleton (Spring Boot, Maven)	Done	main (initial commits)
2	Device entity & repository	Done	PR #1 `feature/1-device-create-list-view`
3	Create device form	Done	PR #1
4	List/view devices	Done	PR #1
5	Device details page	Done	PR #1
6	Update device	Done	PR #3 `feature/2-update-search-dashboard`
7	Search/filter (name, owner, status)	Done	PR #3
8	Compliance rule evaluation	Done (simplified)	PR #1 – antivirus + disk encryption + firewall. **OS-version rule deferred** to backlog

#	Task	Status	Delivered in
9	User roles & login (BCrypt, session)	Done	PR #4 `feature/3-roles-workflow`
10	Status workflow with role checks	Done	PR #4
11	Status history log	Done	PR #4
12	Summary dashboard	Done	PR #3
13	Input validation & error pages	Done (validation); custom error pages deferred	PR #1
14	Health endpoint	Done	`/actuator/health`
15	README & documentation	Done (docs 01-06, README)	main

Carried over to next backlog: OS-version compliance rule, custom error pages, REST/JSON API docs.

2. Role matrix (implemented)

Role	Allowed status moves
TECHNICIAN	REPORTED → UNDER_REVIEW → REMEDIATED
AUDITOR	REMEDIATED → CLOSED, REMEDIATED → UNDER_REVIEW (reopen)
ADMIN	all of the above

Demo users (seeded, passwords stored as BCrypt hashes): `admin/admin123`, `auditor/auditor123`, `tech/tech123`. These are demo credentials for the coursework only.

STEP 7

Jenkins Installation and Continuous Integration Job

Task given: Install and configure Jenkins. Connect the Device Compliance Management Portal GitHub repository and create a Maven/Gradle/Ant build job triggered by commit or scheduled polling. Archive the build artefact.

Deliverable required: Configured Jenkins job, successful build log, trigger evidence and archived artefact.

Jenkins 2.580 (LTS) runs from the WAR on port 9090 using JDK 21; the project itself is built with JDK 11 and Maven 3.9.9. Freestyle job `dcmp-ci` polls the repository (`H/2 * * * *`) and builds on every change.

Deliverable required by the sheet	Shown in this report
Configured Jenkins job	Figures 7.1-7.2
Successful build log	Figures 7.3-7.4
Trigger evidence	Figure 7.5: 'Started by an SCM change' + polling log
Archived artefact	Figure 7.6

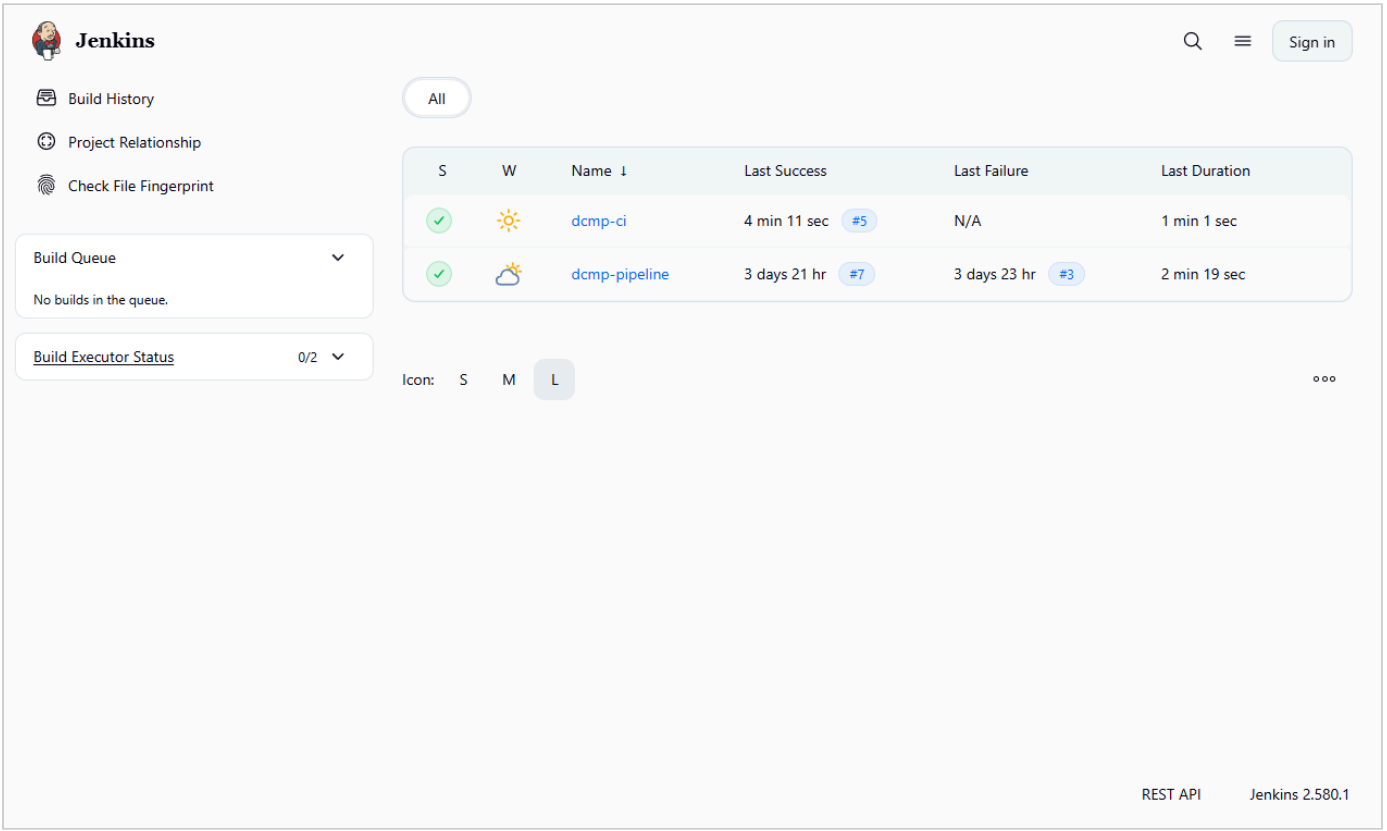


Figure 7.1 - Jenkins dashboard with the jobs

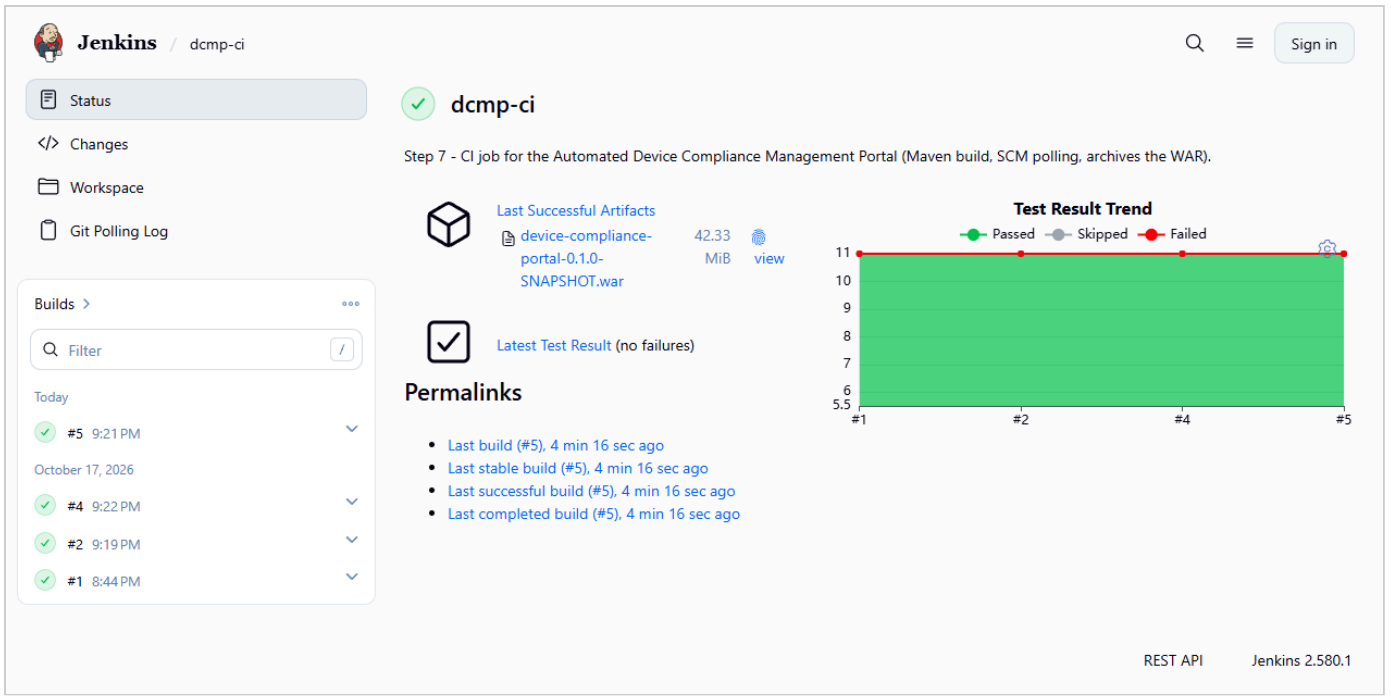


Figure 7.2 - Job dcmp-ci: status, last build, archived WAR, test result trend

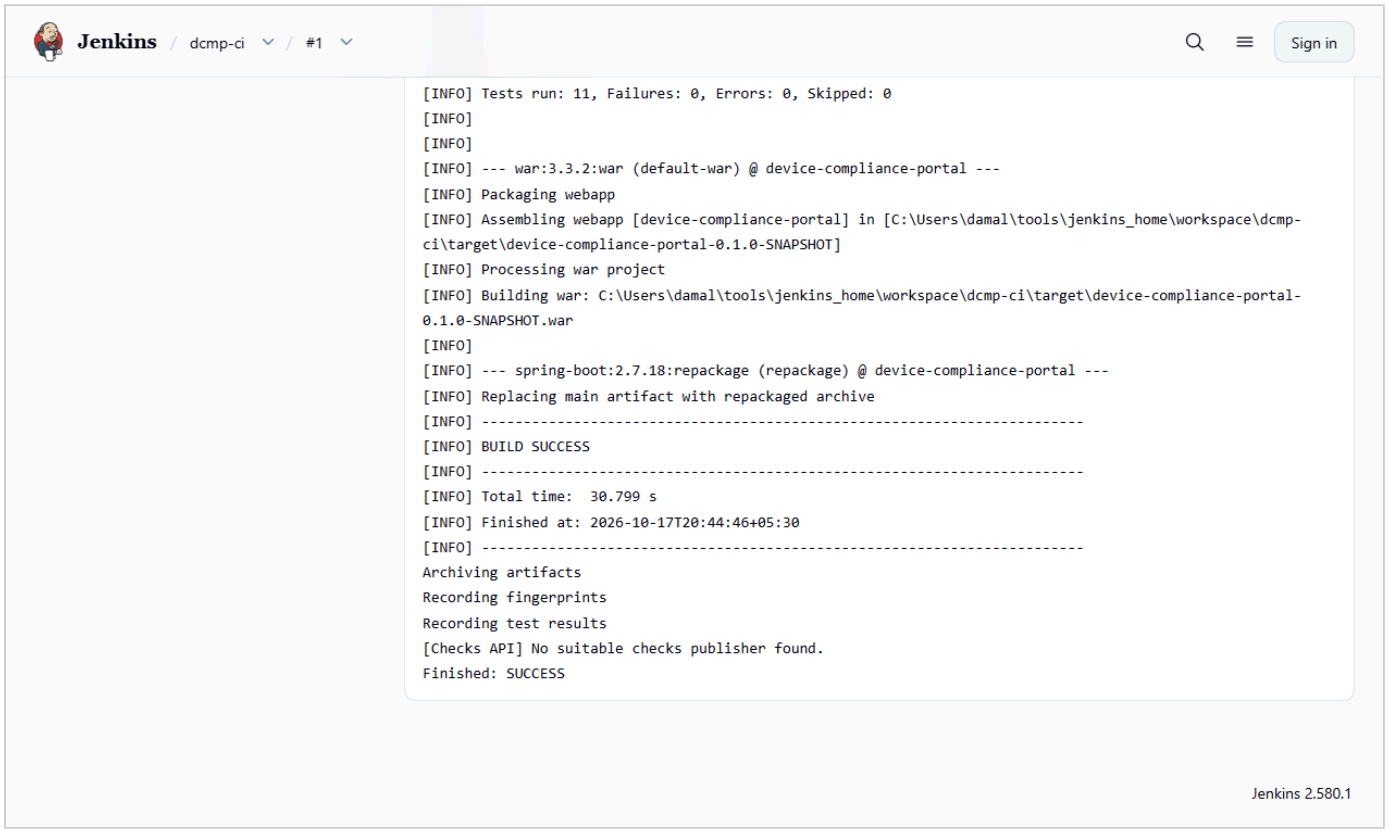


Figure 7.3 - Console output of a successful build

```
Jenkins - job dcmp-ci console / artefact (from build log)

== Jenkins job: dcmp-ci  (build #1, triggered by SCM polling)
Started by an SCM change
Cloning repository https://github.com/Athdam05/device-compliance-portal.git
Checking out Revision 2c07e8fb09ad3378f7b6a4126ce90221a9a5af58 (refs/remotes/origin/develop)
Commit message: "Merge pull request #4 from Athdam05/feature/3-roles-workflow"
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building war: C:\Users\damal\tools\jenkins_home\workspace\dcmp-ci\target\device-compliance-portal-0.1.0-SNAPSHO
T.war
[INFO] BUILD SUCCESS
Archiving artifacts
Finished: SUCCESS

== Archived artefact
44389689 device-compliance-portal-0.1.0-SNAPSHOT.war

== SCM polling (trigger configuration: H/2 * * * *)
<spec>H/2 * * * *</spec>
```

Figure 7.4 - Build log excerpt: SCM trigger, Git checkout, 11 tests, BUILD SUCCESS, archiving

```
Jenkins - dcmp-ci Git polling log

== dcmp-ci Git Polling Log (excerpt)
Started on 21-Oct-2026, 6:09:00 pm
[poll] Last Built Revision: Revision 8da98f0a9d43651d0b4fc279be0eac740120e8e3 (refs/remotes/origin/develop)
[poll] Latest remote head revision on refs/heads/develop is: 8da98f0a9d43651d0b4fc279be0eac740120e8e3 - already built
by 4
Done. Took 1 sec
No changes
```

Figure 7.5 - Git polling log (trigger evidence)

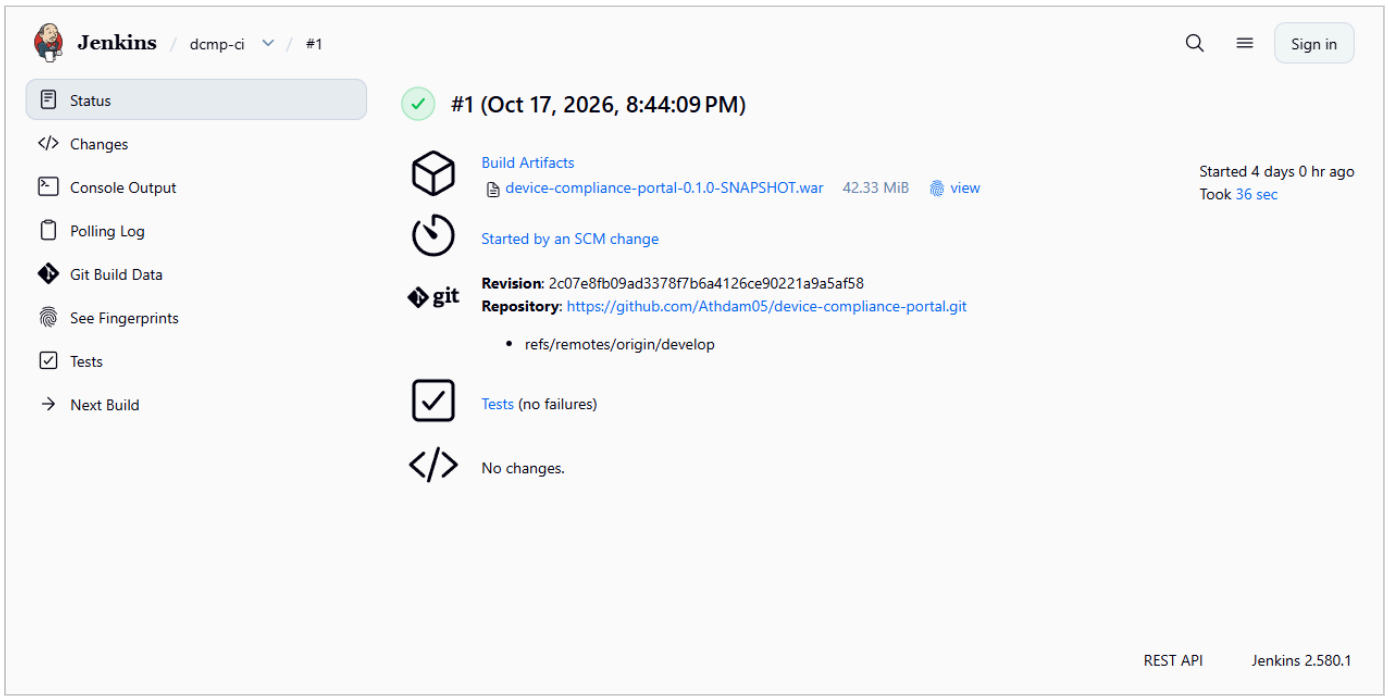


Figure 7.6 - Archived build artefact (WAR)

Job configuration (config.xml excerpt)

```
Source: https://github.com/Athdam05/device-compliance-portal.git   branch: */develop
Trigger: Poll SCM  H/2 * * * *
Build step: set JAVA_HOME=JDK 11; mvn -B clean package
Post-build: archive target/*.war ; publish JUnit results target/surefire-reports/*.xml
```

STEP 8

Pipeline as Code and Server Deployment

> **Task given:** Create a Jenkinsfile for the Device Compliance Management Portal with checkout, build, package and deploy stages. Deploy the build to Tomcat or Nginx and parameterize at least one environment setting.

> **Deliverable required:** Jenkinsfile, successful pipeline run, deployed application URL/screenshot and configuration evidence.

`Jenkinsfile` in the repository root defines the pipeline. Apache Tomcat 9.0.98 (port 8081) is the deployment target. The environment is parameterised: Jenkins parameter `DEPLOY_ENV` (dev / qa) selects the Spring profile (`application-dev.properties`, `application-qa.properties`), and `APP_CONTEXT` sets the context path.

Deliverable required by the sheet	Shown in this report
Jenkinsfile	Listing below (current version; stages Checkout, Build, Package, Deploy, Verify)
Successful pipeline run	Figures 8.1-8.2
Deployed application URL/screenshot	Figures 8.3-8.4
Configuration evidence (parameter)	Figures 8.5-8.6

Jenkinsfile (repository root)

```
// Step 8 - Pipeline as Code: checkout -> build -> package -> deploy to Tomcat.
// Runs on the local Jenkins node (Windows). Tool locations are parameters/env so they can be changed in one place.
pipeline {
    agent any

    parameters {
        // Parameterised environment setting: selects the Spring profile (application-<env>.properties)
        choice(name: 'DEPLOY_ENV', choices: ['dev', 'qa'], description: 'Target environment / Spring profile')
        string(name: 'APP_CONTEXT', defaultValue: 'dcmp', description: 'Tomcat context path for the WAR')
    }

    environment {
        JAVA_HOME   = 'C:\\Program Files\\Eclipse Adoptium\\jdk-11.0.32.101-hotspot'
        MVN         = 'C:\\Users\\damal\\tools\\apache-maven-3.9.9\\bin\\mvn.cmd'
        TOMCAT_HOME = 'C:\\Users\\damal\\tools\\apache-tomcat-9.0.98'
        EDGE_DRIVER = 'C:\\Users\\damal\\tools\\edgedriver\\msedgedriver.exe'
        TEST_PORT   = '8082'
        REGISTRY       = 'localhost:5000'
        IMAGE_NAME     = 'dcmp'
        APP_VERSION    = "1.0.${BUILD_NUMBER}"
        CONTAINER_NAME = 'dcmp-app'
        DOCKER_PORT    = '8092'
        CATALINA_HOME = "${TOMCAT_HOME}"
        CATALINA_PID_DIR = "${TOMCAT_HOME}\\temp"
    }

    options {
        timestamps()
        disableConcurrentBuilds()
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
                bat 'git log --oneline -1'
            }
        }

        stage('Build') {
            steps {
                bat '"%MVN%" -B clean package'
            }
            post {
                always {
                    junit 'target/surefire-reports/*.xml'
                }
            }
        }

        stage('Package') {
            steps {
```

```
                // Version the artefact with the Jenkins build number
                bat 'copy /Y target\\device-compliance-portal-0.1.0-SNAPSHOT.war target\\%APP_CONTEXT%.war'
                archiveArtifacts artifacts: 'target/*.war', fingerprint: true
            }
        }

        stage('Selenium Tests') {
            // Start the freshly built app on a private port with a throw-away in-memory database,
            // run the browser tests against it and ALWAYS stop it again. A failure here stops the
            // pipeline, so the Deploy stage below is never reached with a broken build.
            steps {
                bat '''
                    if exist selenium.pid del selenium.pid
                    powershell -NoProfile -Command "$p = Start-Process -FilePath '%JAVA_HOME%\\bin\\java.exe' -ArgumentList '-ja
r','target\\device-compliance-portal-0.1.0-SNAPSHOT.war','--server.port=%TEST_PORT%','--spring.datasource.url=jdbc:h2:mem:seleniumd
b' -PassThru -WindowStyle Hidden; $p.Id | Out-File -Encoding ascii selenium.pid"
                '''
                bat '''
                    @echo off
                    set /a tries=0
                    :wait
                    curl -s -f http://localhost:%TEST_PORT%/actuator/health && goto up
                    set /a tries+=1
                    if %tries% GEQ 30 exit /b 1
                    ping -n 3 127.0.0.1 > nul
                    goto wait
                    :up
                    echo.
                    echo Test instance is up on port %TEST_PORT%
                '''
                bat '"%MVN%" -B failsafe:integration-test failsafe:verify -Pselenium -Dbase.url=http://localhost:%TEST_PORT% -Dwebd
river.edge.driver="%EDGE_DRIVER%"'
            }
            post {
                always {
                    junit testResults: 'target/failsafe-reports/*.xml', allowEmptyResults: true
                    archiveArtifacts artifacts: 'target/screenshots/*.png', allowEmptyArchive: true
                    bat '''
                        if exist selenium.pid (
                          for /f %%p in (selenium.pid) do taskkill /F /PID %%p
                          del selenium.pid
                        )
                        exit /b 0
                    '''
                }
            }
        }

        stage('Deploy') {
            steps {
                // 1. Tell Tomcat which environment (Spring profile) to run with
                bat '''
                    > "%TOMCAT_HOME%\\bin\\setenv.bat" echo set "JAVA_HOME=%JAVA_HOME%"
                    >> "%TOMCAT_HOME%\\bin\\setenv.bat" echo set "JAVA_OPTS=-Dspring.profiles.active=%DEPLOY_ENV%"
                '''
                // 2. Stop Tomcat if it is running (ignore errors when it is already stopped)
                bat '''
                    call "%TOMCAT_HOME%\\bin\\shutdown.bat"
                    ping -n 8 127.0.0.1 > nul
                    exit /b 0
                '''
                // 3. Replace the deployed application
                bat '''
                    if exist "%TOMCAT_HOME%\\webapps\\%APP_CONTEXT%" rmdir /S /Q "%TOMCAT_HOME%\\webapps\\%APP_CONTEXT%"
                    copy /Y target\\%APP_CONTEXT%.war "%TOMCAT_HOME%\\webapps\\%APP_CONTEXT%.war"
                '''
                // 4. Start Tomcat detached so the Jenkins build can finish
                bat '''
                    set "BUILD_ID=dontKillMe"
                    set "JENKINS_NODE_COOKIE=dontKillMe"
                    start "tomcat" /B cmd /c ""%TOMCAT_HOME%\\bin\\catalina.bat" run > "%TOMCAT_HOME%\\logs\\jenkins-run.log" 2>&1"
                    exit /b 0
                '''
            }
        }

        stage('Verify') {
            steps {
                // Wait for the app to come up, then check health
                bat '''
                    @echo off
                    set /a tries=0
                    :wait
                    curl -s -f http://localhost:8081/%APP_CONTEXT%/actuator/health && goto ok
                    set /a tries+=1
                    if %tries% GEQ 30 exit /b 1
                    ping -n 4 127.0.0.1 > nul
```

```
                    goto wait
                    :ok
                    echo.
                    echo Deployed: http://localhost:8081/%APP_CONTEXT%/ ^(env=%DEPLOY_ENV%^)
                '''
            }
        }

        // ---- Step 12: continuous deployment with Docker (only reached when every stage above passed) ----
        stage('Docker Build') {
            steps {
                // Versioned image: 1.0.<build number>; "latest" always points at the newest good build
                bat '''
                    docker build --label git.commit=%GIT_COMMIT% -t %REGISTRY%/%IMAGE_NAME%:%APP_VERSION% -t %REGISTRY%/%IMAGE_NAM
E%:latest .
                    docker image ls %REGISTRY%/%IMAGE_NAME%
                '''
            }
        }

        stage('Docker Push') {
            steps {
                bat '''
                    docker push %REGISTRY%/%IMAGE_NAME%:%APP_VERSION%
                    docker push %REGISTRY%/%IMAGE_NAME%:latest
                    echo Tags now in the registry:
                    curl -s http://%REGISTRY%/v2/%IMAGE_NAME%/tags/list
                '''
            }
        }

        stage('Docker Deploy') {
            steps {
                // Replace the running container with a fresh one created from the image pulled back out of the registry
                bat '''
                    docker rm -f %CONTAINER_NAME% 2>nul
                    docker pull %REGISTRY%/%IMAGE_NAME%:%APP_VERSION%
                    docker run -d --name %CONTAINER_NAME% --restart unless-stopped -p %DOCKER_PORT%:8080 -v dcmp-app-data:/data -e
SPRING_PROFILES_ACTIVE=%DEPLOY_ENV% %REGISTRY%/%IMAGE_NAME%:%APP_VERSION%
                    exit /b 0
                '''
                bat '''
                    @echo off
                    set /a tries=0
                    :wait
                    curl -s -f http://localhost:%DOCKER_PORT%/actuator/health && goto ok
                    set /a tries+=1
                    if %tries% GEQ 30 exit /b 1
                    ping -n 4 127.0.0.1 > nul
                    goto wait
                    :ok
                    echo.
                    echo Container %CONTAINER_NAME% is healthy: http://localhost:%DOCKER_PORT%/ ^(image %APP_VERSION%, env=%DEPLOY_
ENV%^)
                    docker ps --filter name=%CONTAINER_NAME%
                '''
            }
        }
    }

    post {
        success { echo "Deployed to http://localhost:8081/${params.APP_CONTEXT}/ (env=${params.DEPLOY_ENV})" }
        failure { echo 'Pipeline failed - see stage logs' }
    }
}
```

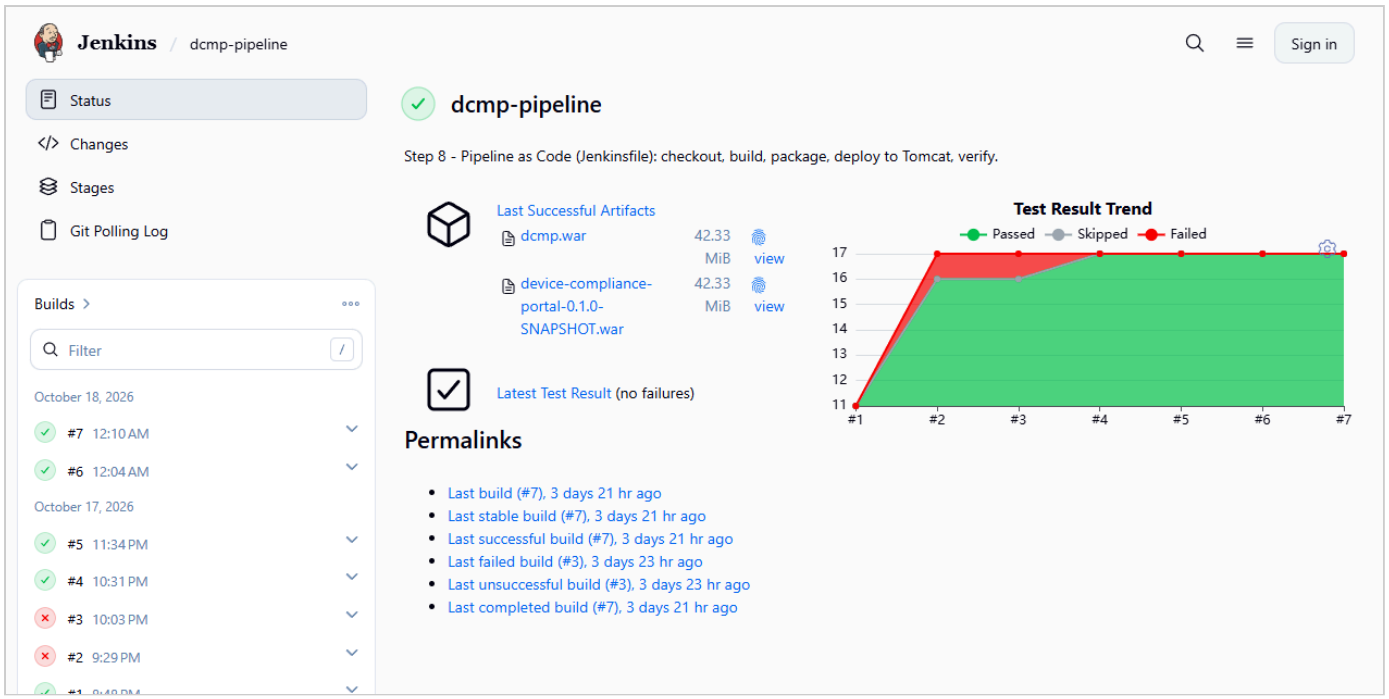


Figure 8.1 - Pipeline job: stage view of successful runs

```
Jenkins - dcmp-pipeline build #1 (console log excerpt)

== Pipeline job dcmp-pipeline - build #1 (step 8: checkout, build, package, deploy to Tomcat, verify)
Started by an SCM change
Checking out Revision 50ae525a1c58132cdcbbffd93a13c8f05e144db7 (refs/remotes/origin/feature/4-jenkinsfile-tomcat-deploy)
Commit message: "ci: add Jenkinsfile with checkout, build, package, deploy and verify stages"
[Pipeline] { (Checkout)
[Pipeline] { (Build)
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[Pipeline] { (Package)
[Pipeline] { (Deploy)
[Pipeline] { (Verify)
{"status":"UP"}
Deployed: http://localhost:8081/dcmp/ (env=dev)
Finished: SUCCESS
```

Figure 8.2 - Console excerpt of pipeline build #1: all stages passed, deployed to Tomcat

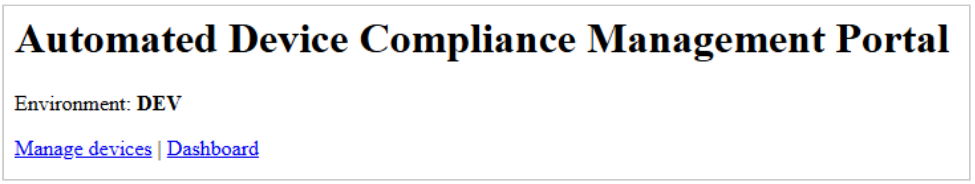


Figure 8.3 - Deployed application at http://localhost:8081/dcmp/ (Environment = DEV)



Figure 8.4 - Health check of the Tomcat deployment

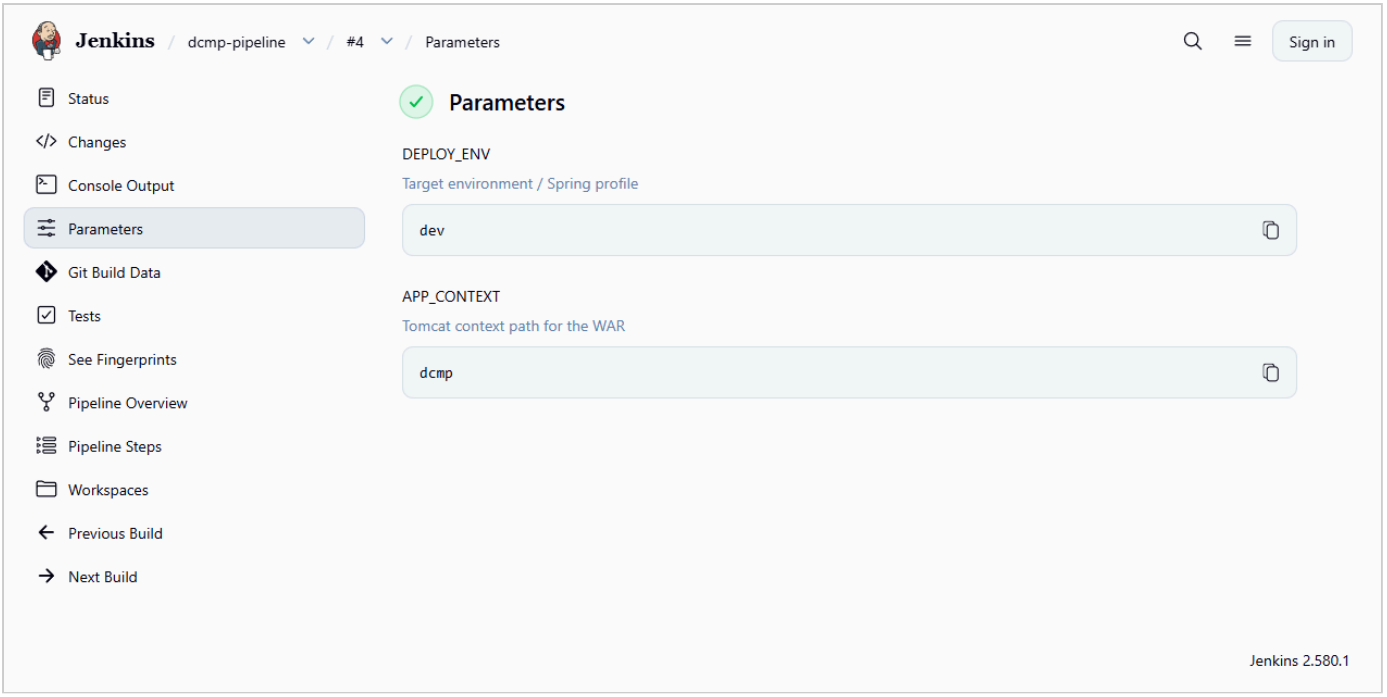


Figure 8.5 - Parameters used by a pipeline run (DEPLOY_ENV = dev, APP_CONTEXT = dcmp) - the parameterised environment setting

application-dev.properties (qa profile sets app.environment=QA)

```
app.environment=DEV
```

STEP 9

Selenium Test Design and Local Execution

> **Task given:** Identify three to five critical user journeys in the Device Compliance Management Portal. Write Selenium WebDriver test cases with assertions, test data and screenshots for failures. Execute the suite locally through Maven.

> **Deliverable required:** Test plan, Selenium scripts, local test report and failure screenshot mechanism.

Five critical user journeys (six tests) are automated with Selenium WebDriver 4.21 in Microsoft Edge, run through Maven Failsafe (`mvn verify -Pselenium`). Detailed plan: `docs/09-selenium-test-plan.md`.

Deliverable required by the sheet	Shown in this report
Test plan	Tables below
Selenium scripts	Listings below
Local test report	Figure 9.1
Failure screenshot mechanism	Listing + Figure 9.2 (real failure screenshot)

1. Critical user journeys

ID	Journey	Why it is critical
J1	Invalid login is rejected; valid login reaches the device list	Gate to the whole application
J2	Register a device; details page shows COMPLIANT / NON_COMPLIANT	Core feature + compliance rule
J3	Search finds a registered device and shows "No devices found" for a miss	Core feature (US4)
J4	Technician moves a device to UNDER_REVIEW; history row recorded; no "Close" option	Role-based workflow (US7, US8)
J5	Dashboard totals are consistent (total = compliant + non-compliant)	Management summary (US9)

2. Test data

Each test creates its own device with a unique serial (`SEL-<nanoTime>`), so tests are independent and repeatable. Demo users (seeded): `admin/admin123`, `tech/tech123`.

3. Test cases

Test	Steps	Assertion
`j1_invalidLoginIsRejected…`	login with wrong password; login admin/admin123	`#login-error` contains "Invalid"; URL ends with `/devices`; `#who` shows ADMIN
`j2_registerCompliantDevice…`	create device with all 3 controls ticked	`#device-name` equals name; `#compliance` = COMPLIANT
`j2_registerDeviceWithMissingControls…`	create device with only antivirus	`#compliance` = NON_COMPLIANT
`j3_searchFinds…`	create device; search by name; search for nonsense	table contains name; table contains "No devices found"
`j4_technicianCanMove…`	admin creates device; logout; tech moves to UNDER_REVIEW	`#status` = UNDER_REVIEW; history row by `tech`; no `#to-CLOSED`
`j5_dashboardShowsSummaryCounts`	create non-compliant device; open dashboard	total ≥ 1; total = compliant + non-compliant

4. Failure screenshot mechanism

`BaseSeleniumIT` registers a JUnit 5 `TestExecutionExceptionHandler`. When any test throws, it saves `target/screenshots/<TestClass>_<testMethod>.png` **before** the browser is closed, then rethrows the failure. (An earlier version used `TestWatcher`, which runs after `@AfterEach` has closed the browser and produced no screenshots — this was found and fixed during development.)

UserJourneysIT.java - the test cases

```
package com.dcmp.portal.selenium;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;

/** Critical user journeys (see docs/09-selenium-test-plan.md). */
class UserJourneysIT extends BaseSeleniumIT {

    // J1 - authentication
    @Test
    void j1_invalidLoginIsRejectedAndValidLoginReachesDeviceList() {
        login("admin", "wrong-password");
        assertTrue(driver.findElement(By.id("login-error")).getText().contains("Invalid"));

        login("admin", "admin123");
        assertTrue(driver.getCurrentUrl().endsWith("/devices"), "expected device list, was " + driver.getCurrentUrl());
        assertTrue(driver.findElement(By.id("who")).getText().contains("ADMIN"));
    }

    // J2 - register a device and see its compliance result
    @Test
    void j2_registerCompliantDeviceShowsCompliantOnDetails() {
        login("admin", "admin123");
        String name = createDevice("J2", true, true, true);
        assertEquals(name, driver.findElement(By.id("device-name")).getText());
        assertEquals("COMPLIANT", driver.findElement(By.id("compliance")).getText());
    }

    @Test
    void j2_registerDeviceWithMissingControlsIsNonCompliant() {
        login("admin", "admin123");
        createDevice("J2b", true, false, false);
        assertEquals("NON_COMPLIANT", driver.findElement(By.id("compliance")).getText());
    }

    // J3 - search
    @Test
    void j3_searchFindsRegisteredDeviceAndHidesOthers() {
        login("admin", "admin123");
        String name = createDevice("J3", true, true, true);

        search(name);
        String table = driver.findElement(By.tagName("table")).getText();
        assertTrue(table.contains(name));

        search("no-such-device-xyz");
        assertTrue(driver.findElement(By.tagName("table")).getText().contains("No devices found"));
    }

    // J4 - role-based workflow
    @Test
    void j4_technicianCanMoveToUnderReviewAndHistoryIsRecorded() {
        login("admin", "admin123");
        createDevice("J4", true, true, true);
        String detailsUrl = driver.getCurrentUrl();
        open("/devices");
        clickAndWait(By.id("logout"));

        login("tech", "tech123");
        driver.get(detailsUrl);
        clickAndWait(By.id("to-UNDER_REVIEW"));
        assertEquals("UNDER_REVIEW", driver.findElement(By.id("status")).getText());
        assertTrue(driver.findElement(By.xpath("//table//td[text()='tech']")).isDisplayed());
        assertTrue(driver.findElements(By.id("to-CLOSED")).isEmpty(), "technician must not see Close");
    }

    // J5 - dashboard
    @Test
    void j5_dashboardShowsSummaryCounts() {
        login("admin", "admin123");
        createDevice("J5", false, false, false);
        open("/dashboard");
        int total = Integer.parseInt(driver.findElement(By.id("total")).getText());
        int compliant = Integer.parseInt(driver.findElement(By.id("compliant")).getText());
        int non = Integer.parseInt(driver.findElement(By.id("non-compliant")).getText());
        assertTrue(total >= 1);
        assertEquals(total, compliant + non);
    }
}
```

BaseSeleniumIT.java - browser setup, waits and failure screenshot

```
package com.dcmp.portal.selenium;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.extension.RegisterExtension;
import org.junit.jupiter.api.extension.TestExecutionExceptionHandler;
import org.junit.jupiter.api.extension.ExtensionContext;
import org.openqa.selenium.By;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.StaleElementReferenceException;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriverException;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.edge.EdgeOptions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;

/**
 * Base class for browser tests. The application under test is addressed by the
 * {@code base.url} system property (default: the Tomcat deployment from step 8).
 * A screenshot is written to target/screenshots/ whenever a test fails.
 */
abstract class BaseSeleniumIT {

    static final String BASE_URL = System.getProperty("base.url", "http://localhost:8081/dcmp");

    protected WebDriver driver;

    /** Runs while the browser is still open (before @AfterEach), so the failure screenshot shows the real page. */
    @RegisterExtension
    final TestExecutionExceptionHandler screenshotOnFailure = new TestExecutionExceptionHandler() {
        @Override
        public void handleTestExecutionException(ExtensionContext context, Throwable throwable) throws Throwable {
            saveScreenshot(context.getRequiredTestClass().getSimpleName() + "_" + context.getRequiredTestMethod().getName());
            throw throwable;
        }
    };

    @BeforeEach
    void startBrowser() {
        EdgeOptions options = new EdgeOptions();
        if (!"false".equals(System.getProperty("headless", "true"))) {
            options.addArguments("--headless=new");
        }
        options.addArguments("--window-size=1280,900", "--no-sandbox", "--remote-allow-origins=*");
        driver = new EdgeDriver(options);
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
    }

    @AfterEach
    void stopBrowser() {
        if (driver != null) {
            driver.quit();
        }
    }

    protected void open(String path) {
        driver.get(BASE_URL + path);
    }

    /** Clicks an element and waits until the resulting page navigation has replaced it. */
    protected void clickAndWait(By locator) {
        WebElement element = driver.findElement(locator);
        element.click();
        new WebDriverWait(driver, Duration.ofSeconds(10)).until(d -> isDetached(element));
    }

    /**
```

Maven - Selenium suite run locally

```
$ mvn test-compile failsafe:integration-test failsafe:verify -Pselenium -Dbase.url=http://localhost:8081/dcmp -Dwebdri
ver.edge.driver=...msedgedriver.exe
[INFO]  T E S T S
[INFO] Running com.dcmp.portal.selenium.UserJourneysIT
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 23.485 s - in com.dcmp.portal.selenium.UserJour
neysIT
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  30.041 s

$ cat target/failsafe-reports/com.dcmp.portal.selenium.UserJourneysIT.txt
-------------------------------------------------------------------------------
Test set: com.dcmp.portal.selenium.UserJourneysIT
-------------------------------------------------------------------------------
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 23.485 s - in com.dcmp.portal.selenium.UserJourneysIT

== individual test results (failsafe XML)
  PASS  j2_registerDeviceWithMissingControlsIsNonCompliant  (3.562 s)
  PASS  j5_dashboardShowsSummaryCounts  (1.643 s)
  PASS  j2_registerCompliantDeviceShowsCompliantOnDetails  (1.518 s)
  PASS  j3_searchFindsRegisteredDeviceAndHidesOthers  (1.885 s)
  PASS  j1_invalidLoginIsRejectedAndValidLoginReachesDeviceList  (1.679 s)
  PASS  j4_technicianCanMoveToUnderReviewAndHistoryIsRecorded  (13.162 s)
```

Figure 9.1 - Local execution through Maven: 6 tests, 0 failures, plus per-test results



Figure 9.2 - Screenshot saved automatically by the framework when a test failed (J2, during step 10)

STEP 10

Continuous Testing in Jenkins

Task given: Integrate the Device Compliance Management Portal Selenium suite into Jenkins. Publish test reports and configure the pipeline so failed tests stop deployment. Correct at least one deliberately introduced defect and rerun the pipeline.

Deliverable required: Jenkins test report, failed pipeline evidence, defect correction commit and successful rerun.

The Selenium suite runs inside the pipeline between Package and Deploy: the build is started on port 8082 with a throw-away database, the tests run against it, results and failure screenshots are published, and a failure stops the pipeline so **Deploy is skipped**. A defect was introduced on purpose (compliance rule changed from AND to OR); it passes all unit tests, and only Selenium catches it.

Deliverable required by the sheet	Shown in this report
Jenkins test report	Figures 10.1, 10.4
Failed pipeline evidence	Figures 10.2-10.3
Defect correction commit	Figure 10.5 (diff of both commits)
Successful rerun	Figures 10.6-10.7

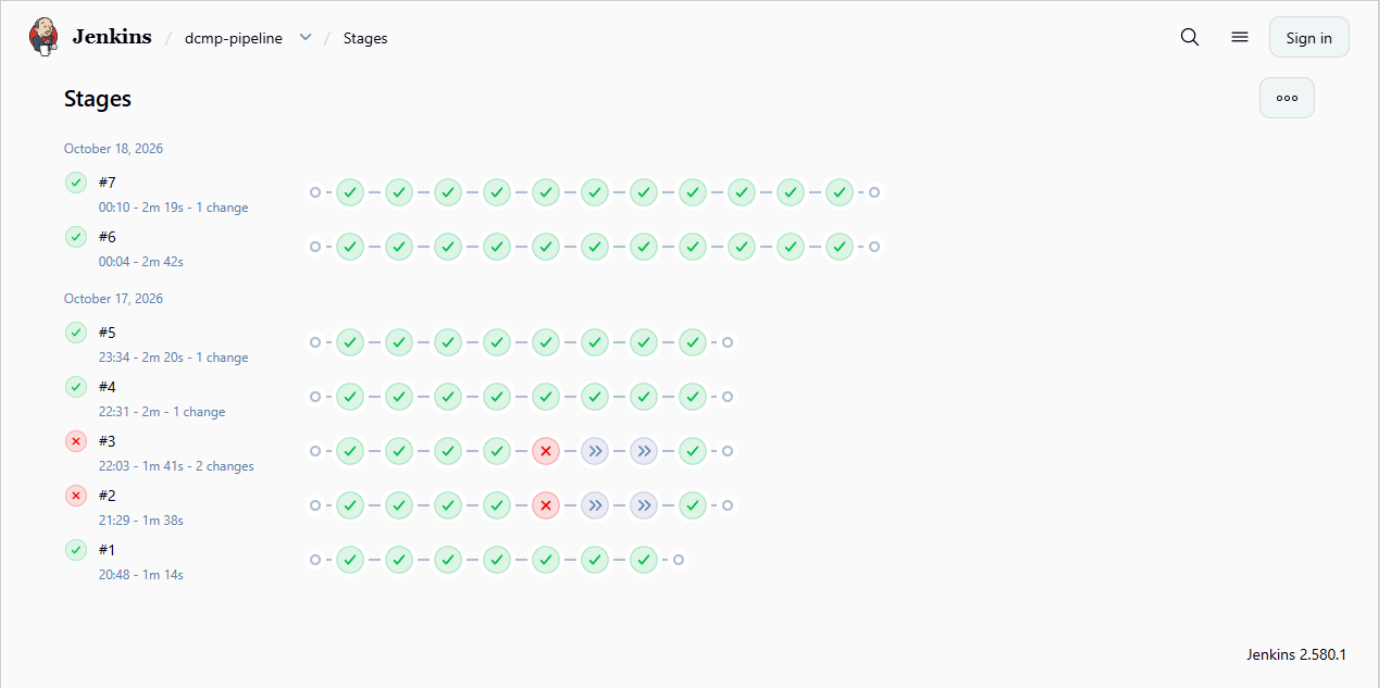


Figure 10.1 - Stage view of all pipeline runs. #3 is the deliberate defect (fails at Selenium); #2 was an earlier unplanned failure from a flaky test wait that was fixed; #4 and later are green

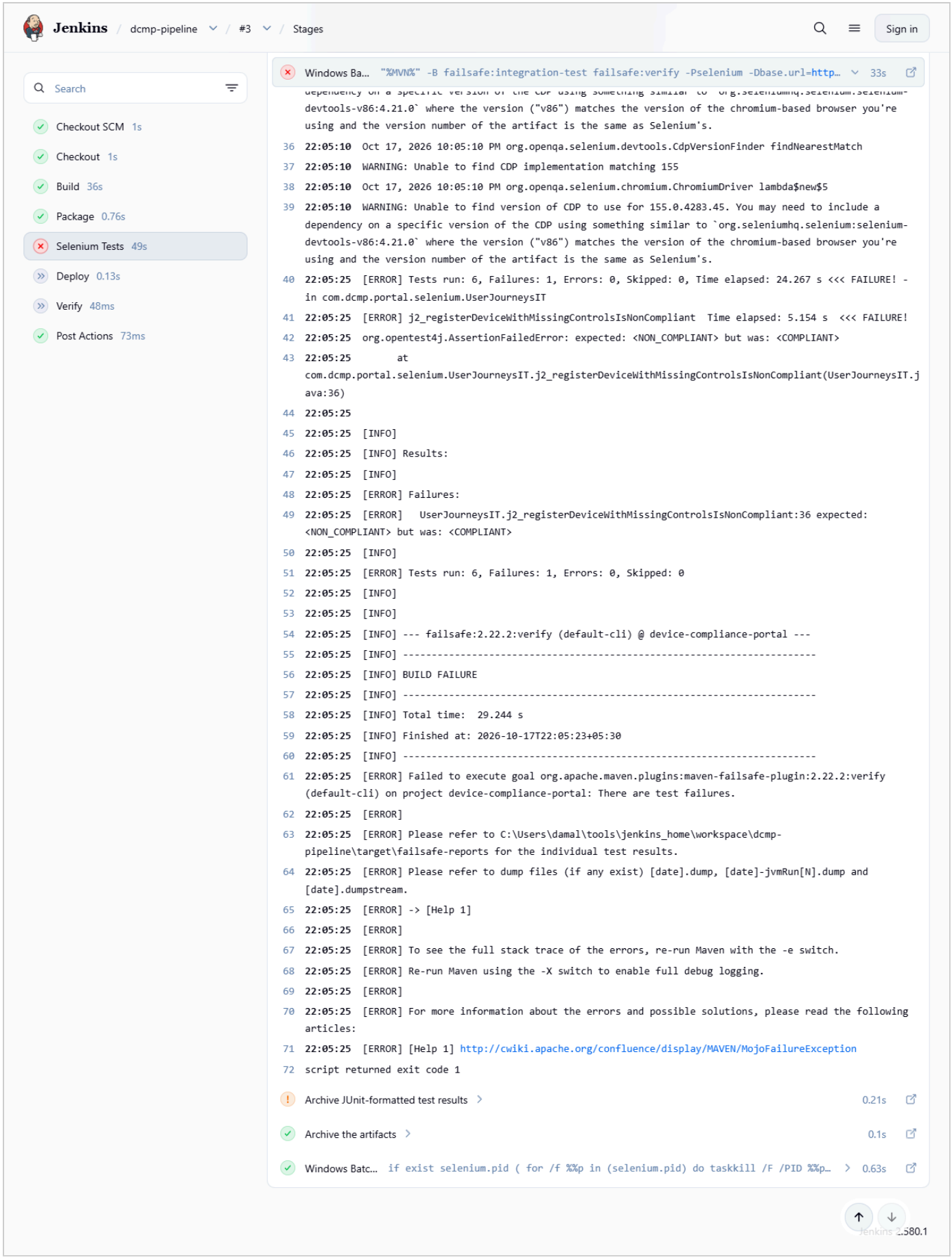


Figure 10.1b - Build #3 stages: red at Selenium Tests, Deploy and Verify skipped

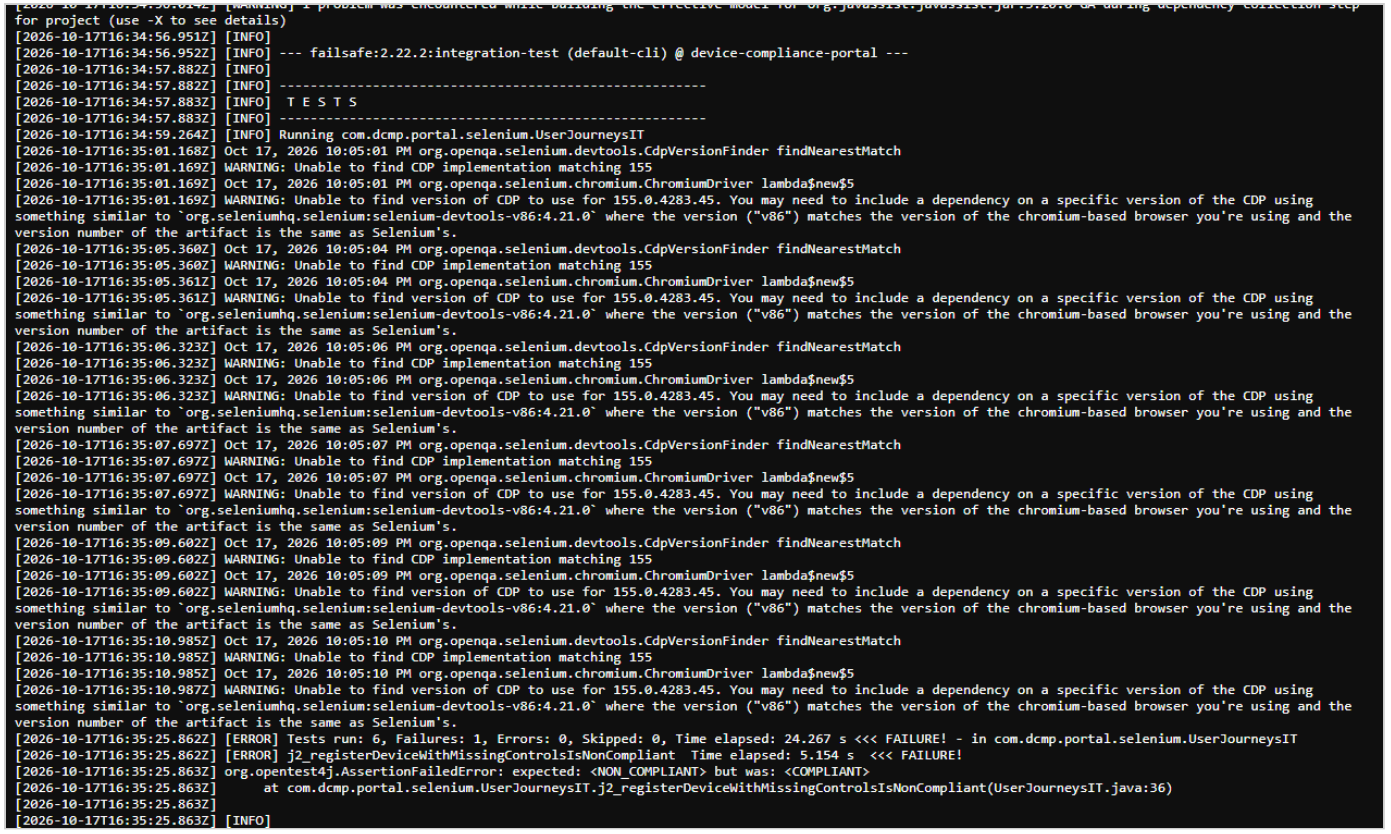


Figure 10.2 - Console of failed build #3: the AssertionFailedError (expected NON_COMPLIANT but was COMPLIANT)

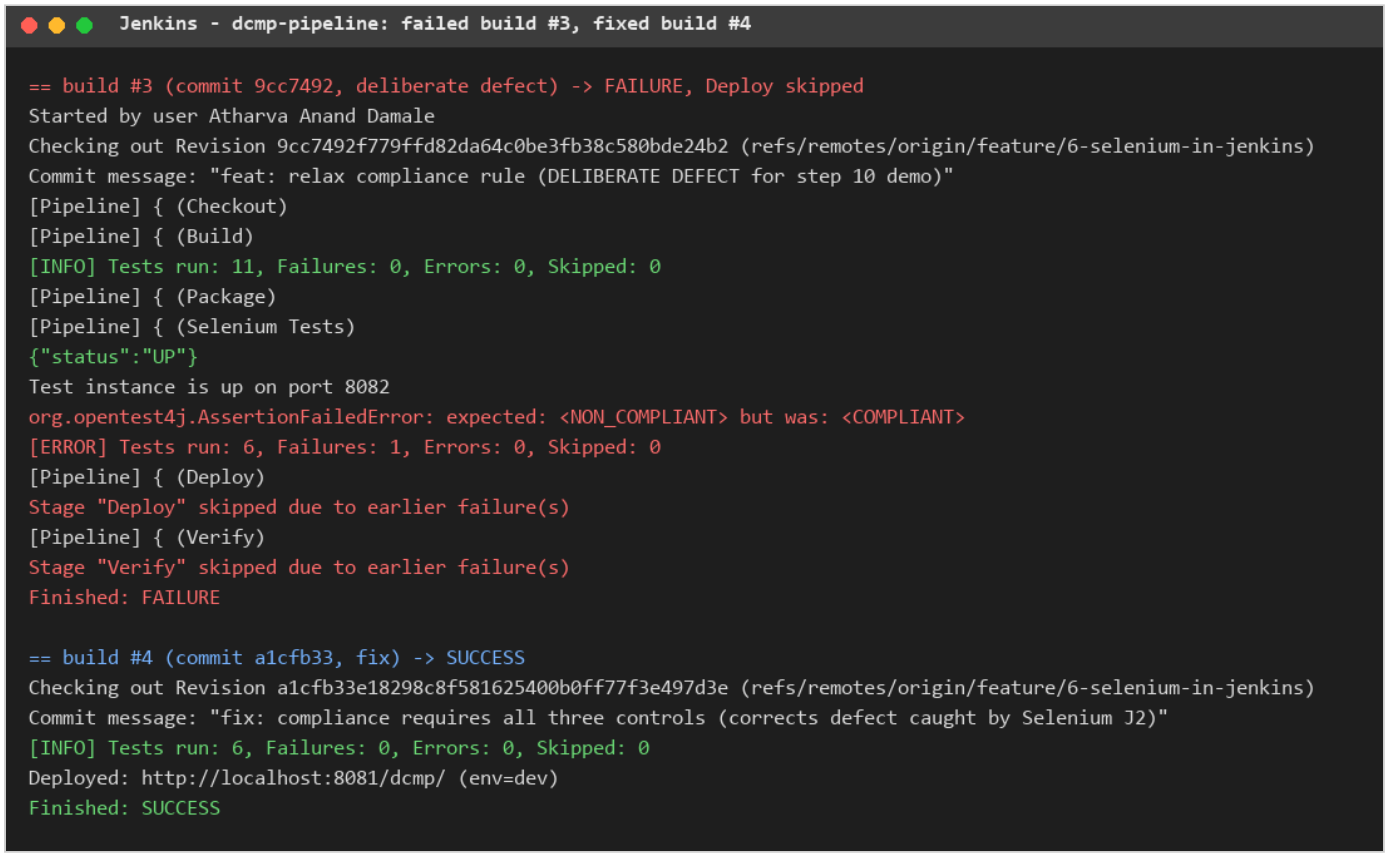


Figure 10.3 - Failed build #3 and successful rerun #4 (log excerpts)

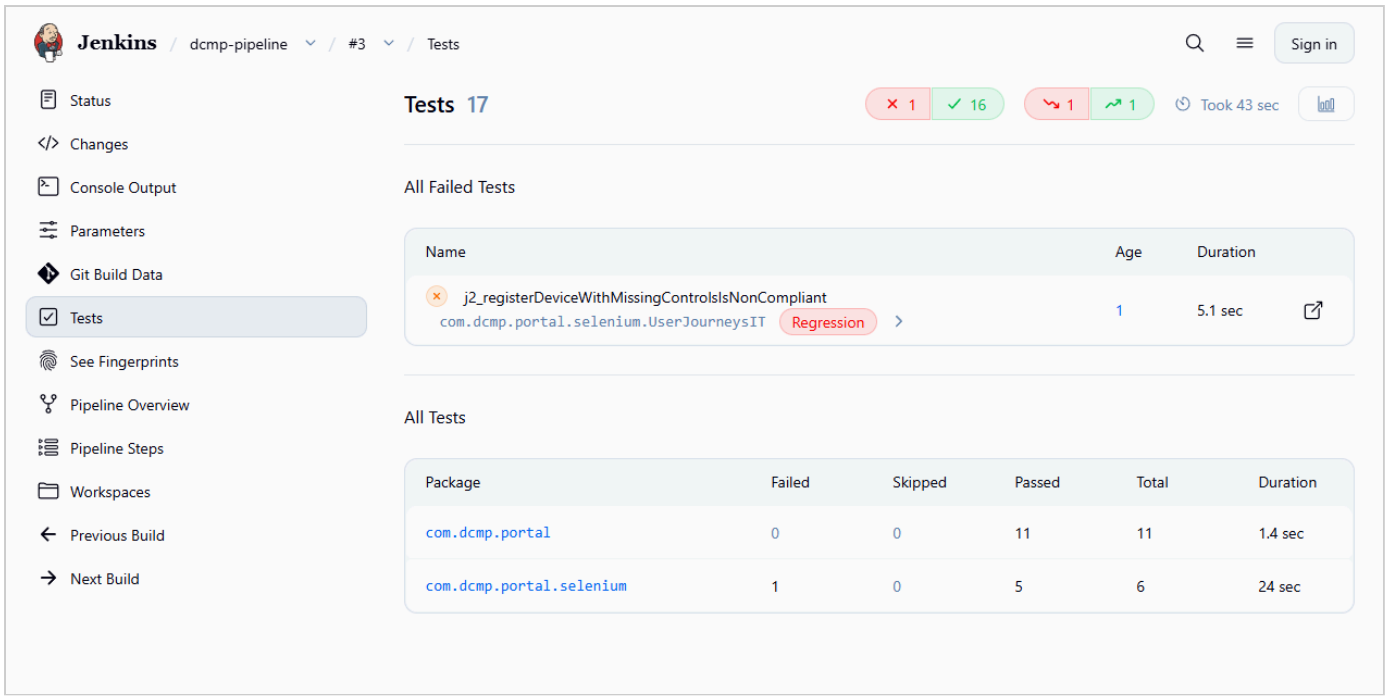


Figure 10.4 - Jenkins test report of the failed build: 1 failure (j2_registerDeviceWithMissingControls...)

Git Bash - the deliberate defect and its correction

```
$ git show 9cc7492 -- src/main/java/com/dcmp/portal/device/Device.java   (the deliberately introduced defect)
9cc7492 feat: relax compliance rule (DELIBERATE DEFECT for step 10 demo)

--- a/src/main/java/com/dcmp/portal/device/Device.java
+++ b/src/main/java/com/dcmp/portal/device/Device.java
@@ -53,6 +53,6 @@ public class Device {
     public void setCreatedAt(LocalDateTime t) { this.createdAt = t; }

     public boolean isCompliant() {
-        return antivirusEnabled && diskEncrypted && firewallEnabled;
+        return antivirusEnabled || diskEncrypted || firewallEnabled;
     }
 }

$ git show a1cfb33 -- src/main/java/com/dcmp/portal/device/Device.java   (the correction)
a1cfb33 fix: compliance requires all three controls (corrects defect caught by Selenium J2)

--- a/src/main/java/com/dcmp/portal/device/Device.java
+++ b/src/main/java/com/dcmp/portal/device/Device.java
@@ -53,6 +53,6 @@ public class Device {
     public void setCreatedAt(LocalDateTime t) { this.createdAt = t; }

     public boolean isCompliant() {
-        return antivirusEnabled || diskEncrypted || firewallEnabled;
+        return antivirusEnabled && diskEncrypted && firewallEnabled;
     }
 }
```

Figure 10.5 - Defect commit 9cc7492 and the correction commit a1cfb33



Figure 10.6 - Failure screenshot archived by Jenkins for build #3

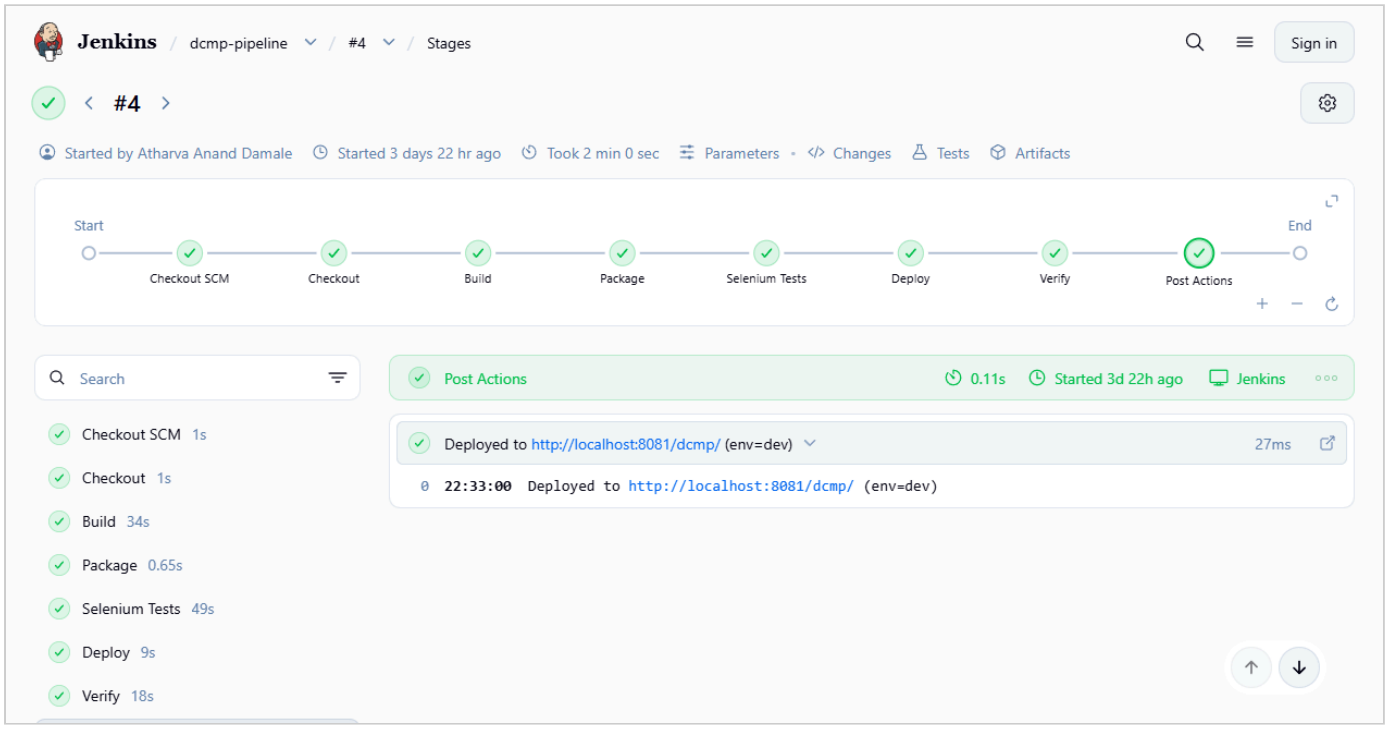


Figure 10.7 - Build #4 after the fix: all stages green, 6/6 Selenium tests passed

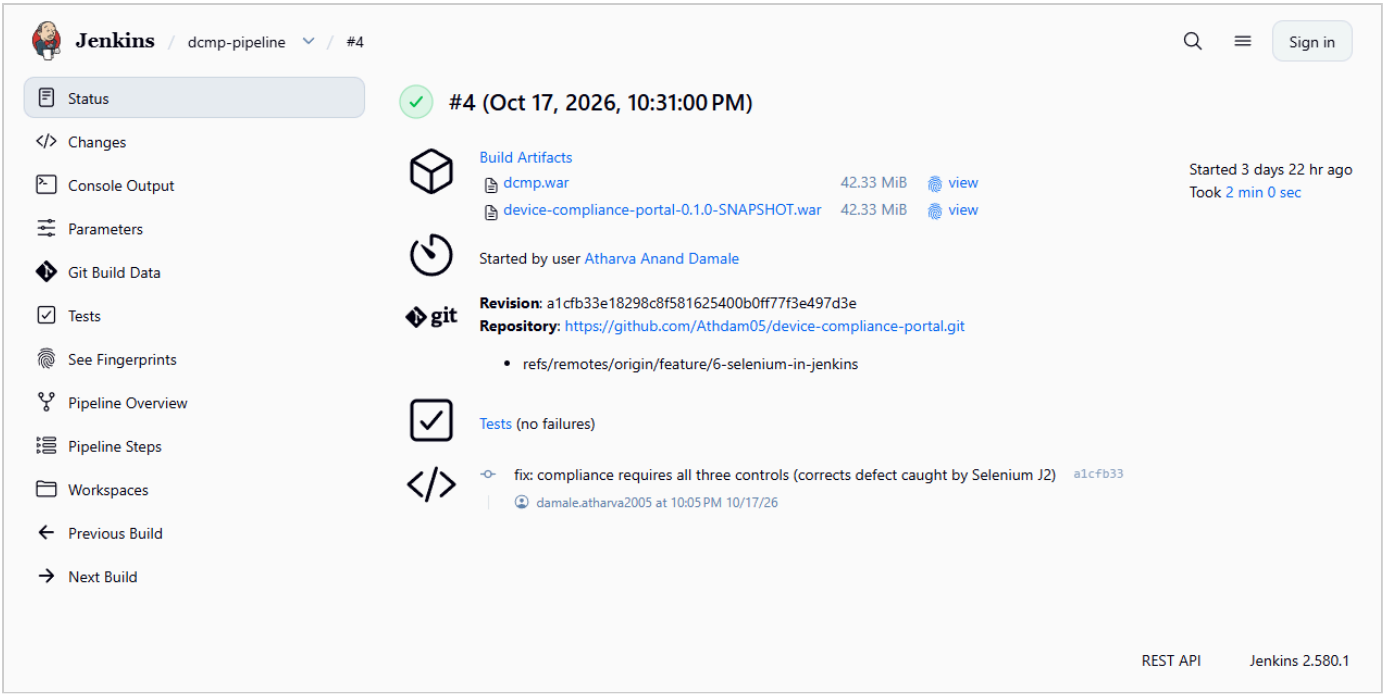


Figure 10.8 - Build #4 status page (commit a1cfb33, test result: no failures)

3. Other findings during integration

Run	Problem	Resolution
#2	J3 failed in `login` with `WebDriverException: Node with given id does not belong to the document`	Edge reports a replaced page this way instead of `StaleElementReferenceException`. `clickAndWait` now treats both as "page changed" (commit `eb4e69e`). Suite then passed 2/2 local runs and Jenkins #4.

STEP 11

Docker Image and Container Lifecycle

Task given: Create a Dockerfile for the Device Compliance Management Portal. Build and tag the image, run the container, map ports, inspect logs, stop/restart/remove containers and document the complete container lifecycle.

Deliverable required: Dockerfile, image details, Docker command log and running container evidence.

Multi-stage `Dockerfile` (Maven build stage, then Eclipse Temurin 11 JRE runtime stage, non-root user, health check). Image tagged `device-compliance-portal:1.0.0` and `:latest`.

Deliverable required by the sheet	Shown in this report
Dockerfile	Listing below
Image details	Figure 11.1
Docker command log (run, port map, logs, stop/restart/remove)	Figure 11.2
Running container evidence	Figures 11.3-11.4
Documented lifecycle	Table below

Dockerfile

```
# Step 11 - container image for the Device Compliance Management Portal.
# Two stages: Maven builds the WAR, a slim JRE image runs it (Spring Boot executable WAR, embedded Tomcat).

# ---- build stage ----
FROM maven:3.9.9-eclipse-temurin-11 AS build
WORKDIR /build
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q clean package -DskipTests

# ---- runtime stage ----
FROM eclipse-temurin:11-jre
LABEL org.opencontainers.image.title="device-compliance-portal" \
      org.opencontainers.image.authors="Atharva Anand Damale (23102C0004)"

# run as an unprivileged user; /data holds the H2 database so it can be mounted as a volume
RUN useradd --system --create-home dcmp && mkdir /data && chown dcmp /data
USER dcmp
WORKDIR /app
COPY --from=build /build/target/device-compliance-portal-*.war app.war

ENV PORT=8080 \
    SPRING_PROFILES_ACTIVE=dev \
    SPRING_DATASOURCE_URL=jdbc:h2:file:/data/dcmp;AUTO_SERVER=TRUE
EXPOSE 8080
VOLUME /data

HEALTHCHECK --interval=30s --timeout=5s --start-period=40s --retries=3 \
  CMD curl -fs http://localhost:${PORT}/actuator/health || exit 1

ENTRYPOINT ["java", "-jar", "app.war"]
```

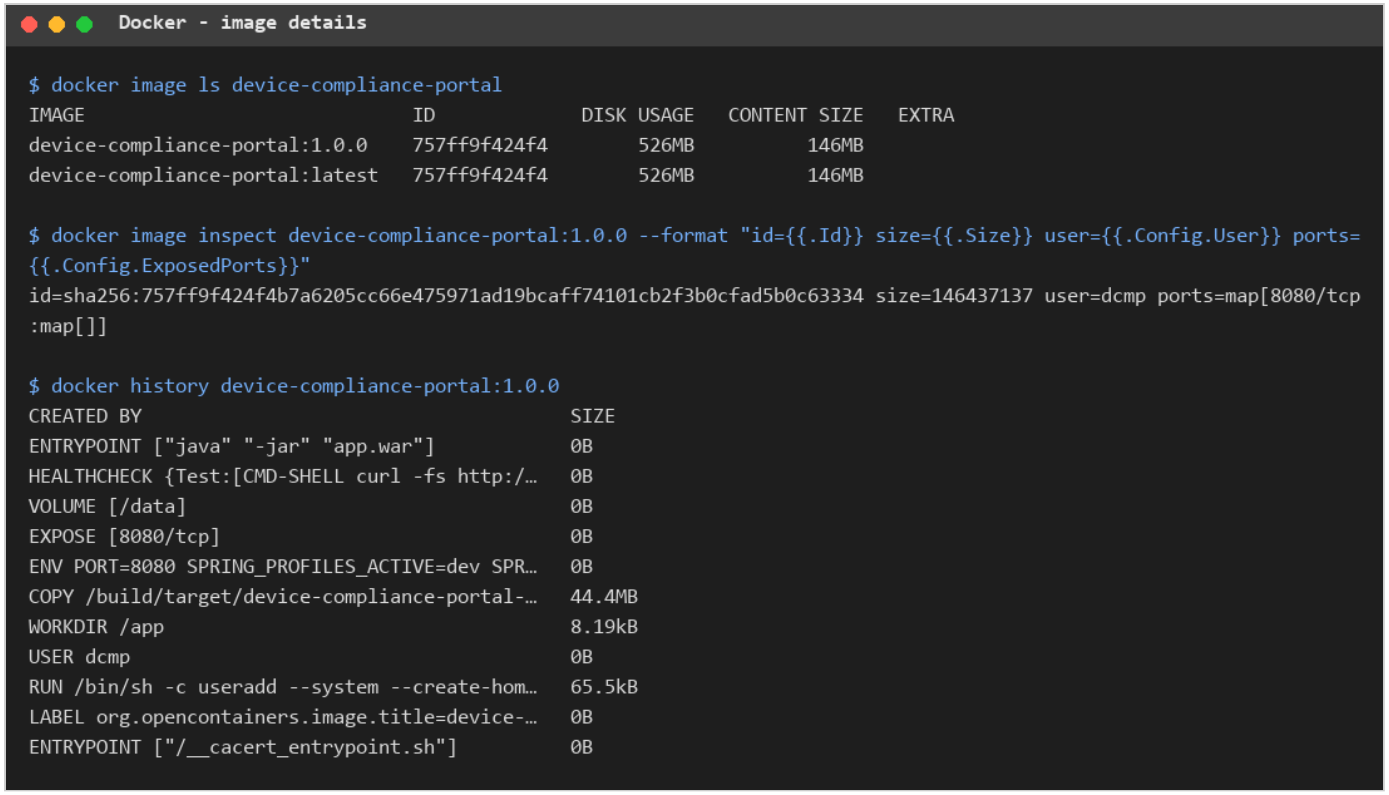


Figure 11.1 - Image details (id, size, user, exposed port, layers)

```
Docker - container lifecycle command log   [1/3]

$ docker image ls device-compliance-portal
IMAGE                            ID             DISK USAGE   CONTENT SIZE   EXTRA
device-compliance-portal:1.0.0   757ff9f424f4        526MB          146MB
device-compliance-portal:latest  757ff9f424f4        526MB          146MB

$ docker image inspect device-compliance-portal:1.0.0 --format id={{.Id}} size={{.Size}} os={{.Os}}/{{.Architecture}}
user={{.Config.User}} exposed={{.Config.ExposedPorts}}
id=sha256:757ff9f424f4b7a6205cc66e475971ad19bcaff74101cb2f3b0cfad5b0c63334 size=146437137 os=linux/arm64 user=dcmp exp
osed=map[8080/tcp:map[]]

$ docker history --no-trunc=false device-compliance-portal:1.0.0
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
757ff9f424f4   8 minutes ago   ENTRYPOINT ["java" "-jar" "app.war"]            0B        buildkit.dockerfile.v0
<missing>      8 minutes ago   HEALTHCHECK {Test:[CMD-SHELL curl -fs http:/…   0B        buildkit.dockerfile.v0
<missing>      8 minutes ago   VOLUME [/data]                                  0B        buildkit.dockerfile.v0
<missing>      8 minutes ago   EXPOSE [8080/tcp]                               0B        buildkit.dockerfile.v0
<missing>      8 minutes ago   ENV PORT=8080 SPRING_PROFILES_ACTIVE=dev SPR…   0B        buildkit.dockerfile.v0
<missing>      8 minutes ago   COPY /build/target/device-compliance-portal-…   44.4MB    buildkit.dockerfile.v0
<missing>      9 minutes ago   WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      9 minutes ago   USER dcmp                                       0B        buildkit.dockerfile.v0
<missing>      9 minutes ago   RUN /bin/sh -c useradd --system --create-hom…   65.5kB    buildkit.dockerfile.v0
<missing>      9 minutes ago   LABEL org.opencontainers.image.title=device-…   0B        buildkit.dockerfile.v0
<missing>      18 hours ago    ENTRYPOINT ["/__cacert_entrypoint.sh"]          0B        buildkit.dockerfile.v0
<missing>      18 hours ago    COPY --chmod=755 entrypoint.sh /__cacert_ent…   12.3kB    buildkit.dockerfile.v0
<missing>      18 hours ago    RUN /bin/sh -c set -eux;     echo "Verifying…   12.3kB    buildkit.dockerfile.v0
<missing>      18 hours ago    RUN /bin/sh -c set -eux;     ARCH="$(dpkg --…   138MB     buildkit.dockerfile.v0
<missing>      18 hours ago    ENV JAVA_VERSION=jdk-11.0.32.1+1                0B        buildkit.dockerfile.v0
<missing>      18 hours ago    RUN /bin/sh -c set -eux;     apt-get update;…   60.5MB    buildkit.dockerfile.v0
<missing>      18 hours ago    ENV LANG=en_US.UTF-8 LANGUAGE=en_US:en LC_AL…   0B        buildkit.dockerfile.v0
<missing>      18 hours ago    ENV PATH=/opt/java/openjdk/bin:/usr/local/sb…   0B        buildkit.dockerfile.v0
<missing>      18 hours ago    ENV JAVA_HOME=/opt/java/openjdk                 0B        buildkit.dockerfile.v0
<missing>      10 days ago     umoci raw add-layer --image /home/buildd/roc…   12.3kB    Add rock control metadata
<missing>      10 days ago     umoci config --image /home/buildd/rockcraft-…   0B        Set annotations
<missing>      10 days ago     umoci config --image /home/buildd/rockcraft-…   0B        Set labels
<missing>      10 days ago     umoci config --image /home/buildd/rockcraft-…   0B        Set default PATH for bare-bas
ed rock
<missing>      10 days ago     umoci config --image /home/buildd/rockcraft-…   0B        Set default commands
<missing>      10 days ago     umoci config --image /home/buildd/rockcraft-…   0B        Set entrypoint
<missing>      10 days ago     umoci raw add-layer --image /home/buildd/roc…   136MB

== 1. create + start (map host 8090 -> container 8080, named volume for the database)

$ docker run -d --name dcmp-demo -p 8090:8080 -v dcmp-data:/data device-compliance-portal:1.0.0
01ca3f273499606847e4f7504e7d2a081b15cd77004e057cab774f01511aeb35

== 2. wait for the application

$ curl -s http://localhost:8090/actuator/health
```

Figure 11.2 - Complete container lifecycle: run, health, inspect, logs, stop, start, restart, remove (part 1)

Docker - container lifecycle command log [2/3]

```
{"status":"UP"}

$ docker ps --filter name=dcmp-demo
CONTAINER ID   IMAGE                          COMMAND              CREATED         STATUS                   PORTS
                                NAMES
01ca3f273499   device-compliance-portal:1.0.0   "java -jar app.war"   7 seconds ago   Up 6 seconds (healthy)   0.0.0.0
:8090->8080/tcp, [::]:8090->8080/tcp   dcmp-demo

== 3. inspect logs / state / resource use

$ docker logs --tail 5 dcmp-demo
2026-10-17 18:16:17.274  INFO 1 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on por
t(s): 8080 (http) with context path ''
2026-10-17 18:16:17.284  INFO 1 --- [           main] com.dcmp.portal.PortalApplication        : Started PortalApplica
tion in 3.789 seconds (JVM running for 4.241)
2026-10-17 18:16:18.155  INFO 1 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring D
ispatcherServlet 'dispatcherServlet'
2026-10-17 18:16:18.156  INFO 1 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing Servlet
'dispatcherServlet'
2026-10-17 18:16:18.157  INFO 1 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initializat
ion in 1 ms

$ docker inspect dcmp-demo --format status={{.State.Status}} pid={{.State.Pid}} ports={{.NetworkSettings.Ports}} healt
h={{.State.Health.Status}}
status=running pid=956 ports=map[8080/tcp:[{0.0.0.0 8090} {:: 8090}]] health=healthy

$ docker stats --no-stream dcmp-demo
CONTAINER ID   NAME        CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O     PIDS
01ca3f273499   dcmp-demo   0.82%     468.1MiB / 7.547GiB   6.06%     2.53kB / 1.33kB   0B / 8.19kB   48

$ docker exec dcmp-demo sh -c id && ls -l /data
uid=999(dcmp) gid=999(dcmp) groups=999(dcmp)
total 32
-rw-r--r-- 1 dcmp dcmp   146 Oct 17 18:16 dcmp.lock.db
-rw-r--r-- 1 dcmp dcmp 28672 Oct 17 18:16 dcmp.mv.db

== 4. stop

$ docker stop dcmp-demo
dcmp-demo

$ docker ps -a --filter name=dcmp-demo
CONTAINER ID   IMAGE                          COMMAND              CREATED         STATUS
       PORTS     NAMES
01ca3f273499   device-compliance-portal:1.0.0   "java -jar app.war"   10 seconds ago   Exited (143) Less than a second
 ago             dcmp-demo

== 5. restart (data volume survives)

$ docker start dcmp-demo
dcmp-demo

$ curl -s http://localhost:8090/actuator/health
{"status":"UP"}

$ docker restart dcmp-demo
```

Figure 11.2 - Complete container lifecycle: run, health, inspect, logs, stop, start, restart, remove (part 2)

```
Docker - container lifecycle command log   [3/3]

dcmp-demo

== 6. remove container (volume kept), then clean up

$ docker rm -f dcmp-demo
dcmp-demo

$ docker ps -a --filter name=dcmp-demo
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

$ docker volume ls --filter name=dcmp-data
DRIVER    VOLUME NAME
local     dcmp-data

$ docker volume rm dcmp-data
dcmp-data
```

Figure 11.2 - Complete container lifecycle: run, health, inspect, logs, stop, start, restart, remove (part 3)

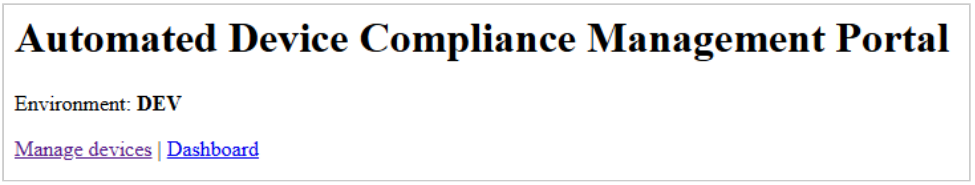


Figure 11.3 - Application served from the container (host port mapped to container 8080)



Figure 11.4 - Container health endpoint

3. Container lifecycle (real run – full output in 11-docker-command-log.txt)

Reproducible with `bash scripts/docker-lifecycle-demo.sh`.

Phase	Command	Observed result
Create + start, map ports, mount volume	`docker run -d --name dcmp-demo -p 8090:8080 -v dcmp-data:/data device-compliance-portal:1.0.0`	container started
Health	`curl http://localhost:8090/actuator/health`	`{"status":"UP"}`; `docker ps` shows `Up … (healthy)`, `0.0.0.0:8090->8080/tcp`
Inspect	`docker inspect`, `docker stats --no-stream`, `docker exec dcmp-demo id`	`status=running`, `health=healthy`, `uid=999(dcmp)`
Logs	`docker logs --tail 5 dcmp-demo`	Spring Boot startup lines
Stop	`docker stop dcmp-demo`	status `Exited (143)` (SIGTERM, graceful)
Start / restart	`docker start dcmp-demo`, `docker restart dcmp-demo`	health UP again
Remove	`docker rm -f dcmp-demo`	container gone from `docker ps -a`
Clean up volume	`docker volume rm dcmp-data`	volume removed

Data persistence check

Created a device in a container with volume `dcmp-persist-data`, destroyed the container (`docker rm -f`), started a **new** container on the same volume and listed devices: the device `Persist-Check` was still present. (Test containers/volume removed afterwards.)

STEP 12

Jenkins-Docker Continuous Deployment

Task given: Extend the Device Compliance Management Portal Jenkins pipeline to build a versioned Docker image, publish it to Docker Hub or a local registry and deploy a fresh container automatically after successful tests.

Deliverable required: Versioned image, registry evidence and end-to-end commit-to-container pipeline.

Three Docker stages were added to the pipeline: **Docker Build** (tag 1.0.<build number>), **Docker Push** to a local registry (`registry:2` on localhost:5000) and **Docker Deploy** (stop old container, pull the new image back from the registry, run a fresh container on port 8092, health check). They run only after tests and Tomcat deployment passed.

Deliverable required by the sheet	Shown in this report
Versioned image	Figures 12.1-12.2 (tags 1.0.6, 1.0.7)
Registry evidence	Figure 12.2 (catalog and tag list from the registry API)
End-to-end commit-to-container pipeline	Figures 12.3-12.5 (build #7 was started by a Git commit, no manual click)

Jenkinsfile - Docker stages

```
                    set "BUILD_ID=dontKillMe"
                    set "JENKINS_NODE_COOKIE=dontKillMe"
                    start "tomcat" /B cmd /c ""%TOMCAT_HOME%\\bin\\catalina.bat" run > "%TOMCAT_HOME%\\logs\\jenkins-run.log" 2>&1"
                    exit /b 0
                '''
            }
        }

        stage('Verify') {
            steps {
                // Wait for the app to come up, then check health
                bat '''
                    @echo off
                    set /a tries=0
                    :wait
                    curl -s -f http://localhost:8081/%APP_CONTEXT%/actuator/health && goto ok
                    set /a tries+=1
                    if %tries% GEQ 30 exit /b 1
                    ping -n 4 127.0.0.1 > nul
                    goto wait
                    :ok
                    echo.
                    echo Deployed: http://localhost:8081/%APP_CONTEXT%/ ^(env=%DEPLOY_ENV%^)
                '''
            }
        }

        // ---- Step 12: continuous deployment with Docker (only reached when every stage above passed) ----
        stage('Docker Build') {
            steps {
                // Versioned image: 1.0.<build number>; "latest" always points at the newest good build
                bat '''
                    docker build --label git.commit=%GIT_COMMIT% -t %REGISTRY%/%IMAGE_NAME%:%APP_VERSION% -t %REGISTRY%/%IMAGE_NAM
E%:latest .
                    docker image ls %REGISTRY%/%IMAGE_NAME%
                '''
            }
        }

        stage('Docker Push') {
            steps {
                bat '''
                    docker push %REGISTRY%/%IMAGE_NAME%:%APP_VERSION%
                    docker push %REGISTRY%/%IMAGE_NAME%:latest
                    echo Tags now in the registry:
                    curl -s http://%REGISTRY%/v2/%IMAGE_NAME%/tags/list
                '''
            }
        }

        stage('Docker Deploy') {
            steps {
                // Replace the running container with a fresh one created from the image pulled back out of the registry
```

```
                bat '''
                    docker rm -f %CONTAINER_NAME% 2>nul
                    docker pull %REGISTRY%/%IMAGE_NAME%:%APP_VERSION%
                    docker run -d --name %CONTAINER_NAME% --restart unless-stopped -p %DOCKER_PORT%:8080 -v dcmp-app-data:/data -e
SPRING_PROFILES_ACTIVE=%DEPLOY_ENV% %REGISTRY%/%IMAGE_NAME%:%APP_VERSION%
                    exit /b 0
                '''
                bat '''
                    @echo off
                    set /a tries=0
                    :wait
                    curl -s -f http://localhost:%DOCKER_PORT%/actuator/health && goto ok
                    set /a tries+=1
                    if %tries% GEQ 30 exit /b 1
                    ping -n 4 127.0.0.1 > nul
                    goto wait
                    :ok
                    echo.
                    echo Container %CONTAINER_NAME% is healthy: http://localhost:%DOCKER_PORT%/ ^(image %APP_VERSION%, env=%DEPLOY_
ENV%^)
                    docker ps --filter name=%CONTAINER_NAME%
                '''
            }
```

Docker - registry and running container

```
$ docker image ls localhost:5000/dcmp
IMAGE                        ID             DISK USAGE   CONTENT SIZE   EXTRA
localhost:5000/dcmp:1.0.6    760f63be8673       526MB          146MB
localhost:5000/dcmp:1.0.7    e044aeacc3e2       526MB          146MB   U
localhost:5000/dcmp:latest   e044aeacc3e2       526MB          146MB   U

$ curl http://localhost:5000/v2/_catalog
{"repositories":["dcmp"]}

$ curl http://localhost:5000/v2/dcmp/tags/list
{"name":"dcmp","tags":["1.0.7","latest","1.0.6"]}


$ docker ps --filter name=dcmp-app
NAMES      IMAGE                      STATUS                    PORTS
dcmp-app   localhost:5000/dcmp:1.0.7   Up 15 minutes (healthy)   0.0.0.0:8092->8080/tcp, [::]:8092->8080/tcp

$ docker image inspect localhost:5000/dcmp:1.0.6  (traceability label)
git.commit=192d536acf4ee300388c2038a1a433e0c1a96c7f
```

Figure 12.1/12.2 - Registry catalog, tags, running container and the git.commit label on the image

```
Jenkins - dcmp-pipeline Docker CD builds   [1/2]

== build #6 (commit 192d536): Docker Build -> Push -> Deploy
Started by user Atharva Anand Damale
Checking out Revision 192d536acf4ee300388c2038a1a433e0c1a96c7f (refs/remotes/origin/feature/8-docker-cd)
Commit message: "ci: build versioned Docker image, push to local registry and deploy fresh container"
[Pipeline] { (Checkout)
[Pipeline] { (Build)
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[Pipeline] { (Package)
[Pipeline] { (Selenium Tests)
{"status":"UP"}
Test instance is up on port 8082
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[Pipeline] { (Deploy)
[Pipeline] { (Verify)
Deployed: http://localhost:8081/dcmp/ (env=dev)
[Pipeline] { (Docker Build)
[Pipeline] { (Docker Push)
C:\Users\damal\tools\jenkins_home\workspace\dcmp-pipeline>echo Tags now in the registry:
Tags now in the registry:
{"name":"dcmp","tags":["latest","1.0.6"]}
[Pipeline] { (Docker Deploy)
C:\Users\damal\tools\jenkins_home\workspace\dcmp-pipeline>docker rm -f dcmp-app  2>nul
C:\Users\damal\tools\jenkins_home\workspace\dcmp-pipeline>docker run -d --name dcmp-app --restart unless-stopped -p 80
92:8080 -v dcmp-app-data:/data -e SPRING_PROFIL
Container dcmp-app is healthy: http://localhost:8092/ (image 1.0.6, env=dev)
b0604ebbe695   localhost:5000/dcmp:1.0.6   "java -jar app.war"   7 seconds ago   Up 6 seconds (healthy)   0.0.0.0:8092
->8080/tcp, [::]:8092->8080/tcp   dcmp-app
Finished: SUCCESS

== build #7: started AUTOMATICALLY by an SCM change (docs commit 386d8d0) - no manual click
Started by an SCM change
Checking out Revision 386d8d073a19155ec83049ad28577d94df857637 (refs/remotes/origin/feature/8-docker-cd)
Commit message: "docs: add step 12 Jenkins-Docker continuous deployment evidence"
[Pipeline] { (Checkout)
[Pipeline] { (Build)
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[Pipeline] { (Package)
[Pipeline] { (Selenium Tests)
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[Pipeline] { (Deploy)
[Pipeline] { (Verify)
[Pipeline] { (Docker Build)
[Pipeline] { (Docker Push)
C:\Users\damal\tools\jenkins_home\workspace\dcmp-pipeline>echo Tags now in the registry:
Tags now in the registry:
{"name":"dcmp","tags":["1.0.7","latest","1.0.6"]}
[Pipeline] { (Docker Deploy)
Container dcmp-app is healthy: http://localhost:8092/ (image 1.0.7, env=dev)
```

Figure 12.3 - Build #6 (manual) and build #7 (automatic, started by commit 386d8d0): stages and image tags (part 1)

```
Jenkins - dcmp-pipeline Docker CD builds   [2/2]

Finished: SUCCESS
```

Figure 12.3 - Build #6 (manual) and build #7 (automatic, started by commit 386d8d0): stages and image tags (part 2)

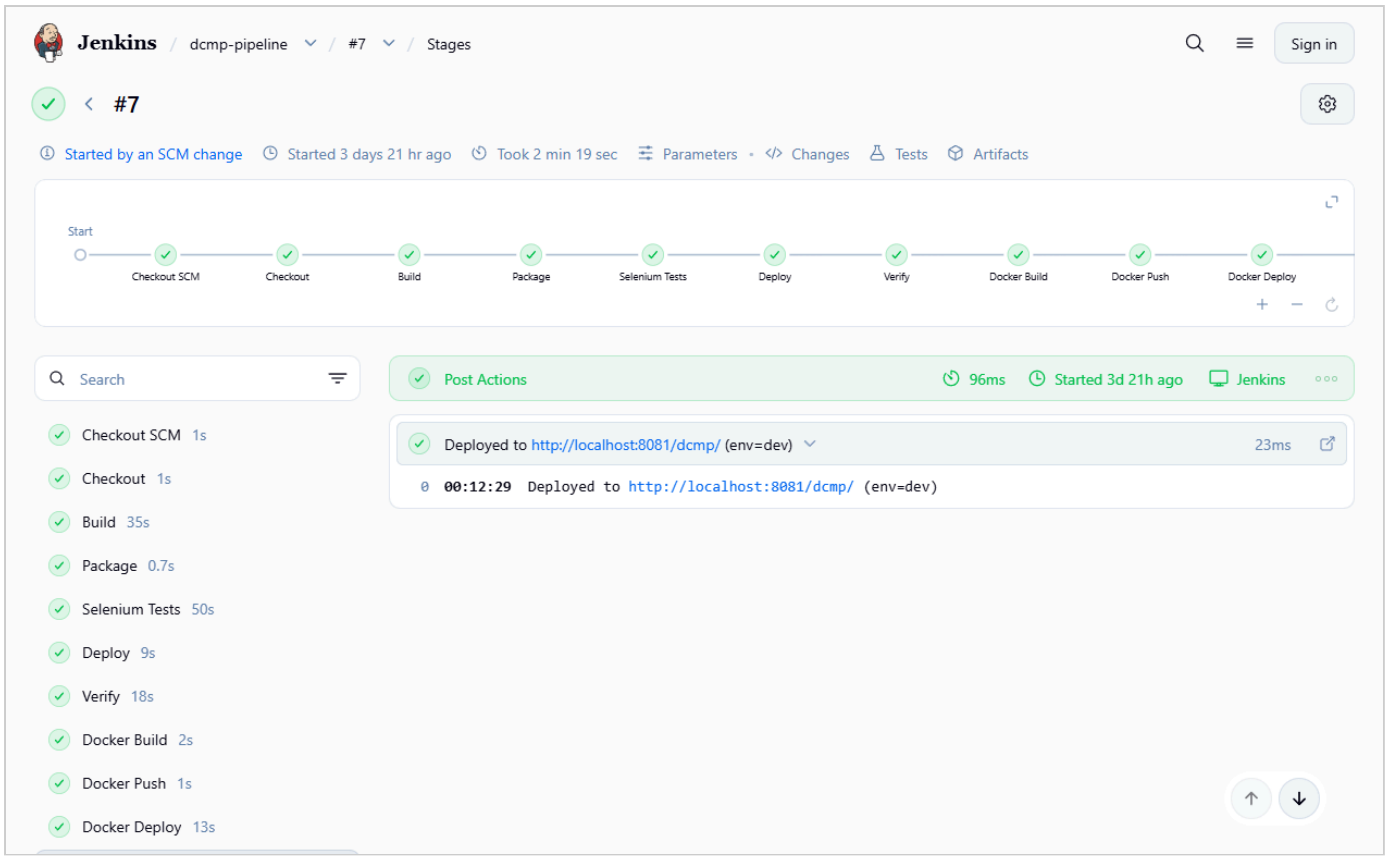


Figure 12.4 - Build #7 stages including Docker Build / Push / Deploy

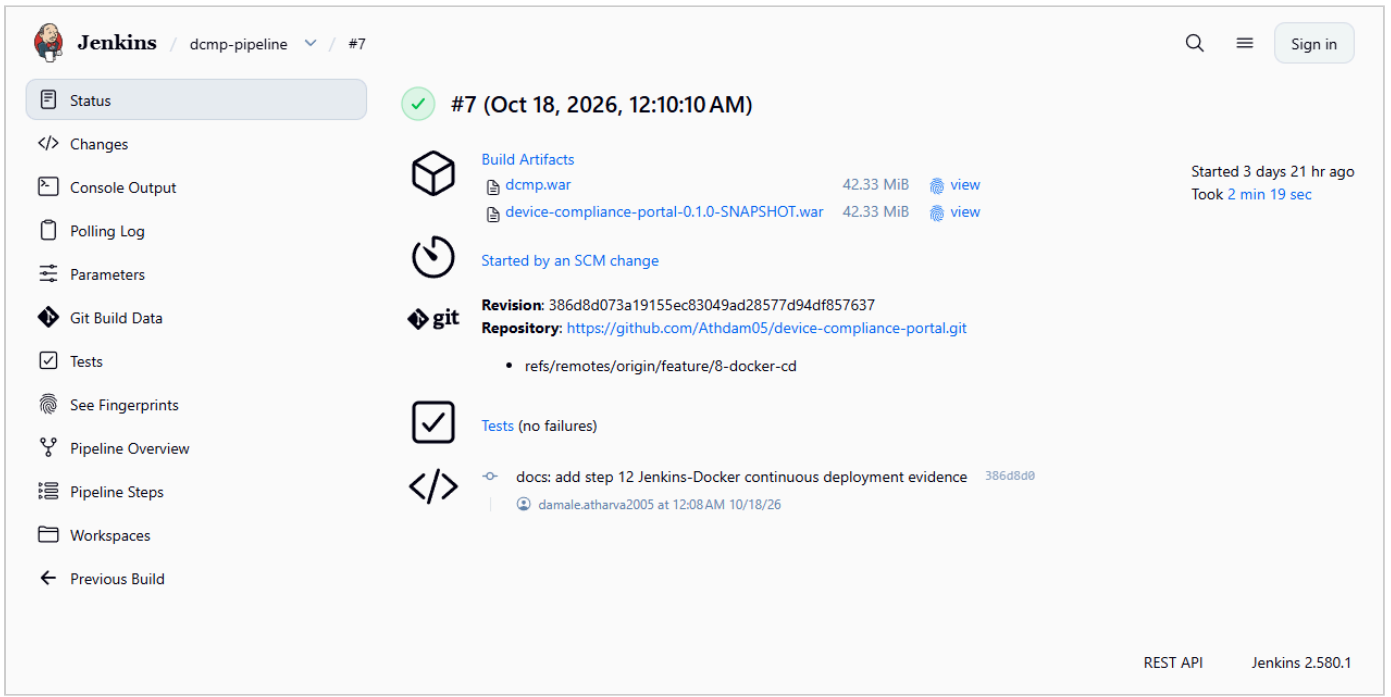


Figure 12.4b - Build #7 status page: 'Started by an SCM change' (automatic commit-to-container run)

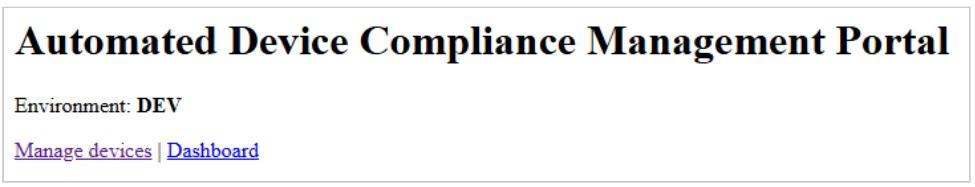


Figure 12.5 - Application running in the freshly deployed container

STEP 13

Configuration Management Script

> **Task given:** Identify all server prerequisites for the Device Compliance Management Portal: packages, users, folders, files, ports and services. Create either a Puppet manifest/modules or an Ansible inventory and YAML playbook to configure the target node.

> **Deliverable required:** Configuration specification, Puppet manifest or Ansible playbook and first execution log.

Ansible (ansible-core 2.17, run from a container) configures a fresh Ubuntu 22.04 node reached over SSH. Source: `docs/13-configuration-management.md`, folder `ansible/`.

Deliverable required by the sheet	Shown in this report
Configuration specification	Table below (packages, user, folders, files, ports, services)
Ansible playbook + inventory	Listings below
First execution log	Figure 13.1 and raw log in repo

1. Configuration specification – what the portal needs on a server

Category	Requirement	Ansible task
Packages	`openjdk-11-jre-headless` (the app is built for Java 11), `curl`, `ca-certificates`, `supervisor` (process manager)	`apt` – Install prerequisite packages
User / group	system group `dcmp` and unprivileged system user `dcmp` (shell `/usr/sbin/nologin`, home `/opt/dcmp`)	`group`, `user`
Folders	`/opt/dcmp` (0755), `/opt/dcmp/releases/<version>` (0755), `/var/lib/dcmp/data` (0750, H2 database), `/var/log/dcmp` (0755) – all owned by `dcmp`	`file` (loop)
Files	`/opt/dcmp/releases/<version>/app.war` (0640, from the Jenkins-built artefact); symlink `/opt/dcmp/current` → active release; `/etc/supervisor/conf.d/dcmp.conf` (service definition from a Jinja2 template)	`copy`, `file state=link`, `template`
Ports	application listens on **8080/tcp** (published as 8093 on the lab host); sshd on 22/tcp for Ansible	`wait_for`
Services	`dcmp` runs under supervisor as user `dcmp`, auto-start and auto-restart, env `SPRING_PROFILES_ACTIVE`, `SPRING_DATASOURCE_URL=jdbc:h2:file:/var/lib/dcmp/data/dcmp`	handler `supervisorctl reread / update / restart`
Verification	`GET /actuator/health` must return 200 and contain `UP`	`uri` with retries

All tunable values (version, profile, port, folders, heap) are in `ansible/group_vars/all.yml`. The deployed layout keeps one folder per version, so a previous release can be re-activated by repointing `current` (used in step 14).

ansible/inventory/hosts.ini

```
# Target node(s) for the Device Compliance Management Portal.
# In this lab the node is a container on the Docker network "dcmp-net"; on a real server only ansible_host changes.
[dcmp_servers]
dcmp-node ansible_host=dcmp-node

[dcmp_servers:vars]
ansible_user=ansible
ansible_ssh_private_key_file=/root/.ssh/id_ed25519
ansible_become=true
```

ansible/group_vars/all.yml (variables)

```
---
# ---- Configuration specification (single place to change settings) ----
app_name: dcmp
```

```
app_user: dcmp
app_group: dcmp
app_version: "1.0.6"                    # release being deployed; each version gets its own folder
app_env: dev                            # Spring profile (dev / qa)
app_port: 8080                          # port the application listens on inside the node
app_home: /opt/dcmp                     # /opt/dcmp/releases/<version>/app.war  +  /opt/dcmp/current -> release
data_dir: /var/lib/dcmp/data            # H2 database files
log_dir: /var/log/dcmp
java_heap: 256m

# WAR built and tested by Jenkins (copied into ansible/artifacts/ by run.sh); path as seen from the control container
war_src: "/work/ansible/artifacts/app-{{ app_version }}.war"

os_packages:
  - openjdk-11-jre-headless
  - curl
  - ca-certificates
  - supervisor
health_wait: 60                         # seconds to wait for the port after a deployment
```

ansible/templates/dcmp.conf.j2 (service definition)

```
; {{ ansible_managed }}
[program:{{ app_name }}]
command=/usr/bin/java -Xmx{{ java_heap }} -jar {{ app_home }}/current/app.war --server.port={{ app_port }}
directory={{ app_home }}
user={{ app_user }}
environment=APP_VERSION="{{ release_label | default(app_version) }}",SPRING_PROFILES_ACTIVE="{{ app_env }}",SPRING_DATASOURCE_URL
="jdbc:h2:file:{{ data_dir }}/dcmp;AUTO_SERVER=TRUE"
autostart=true
autorestart=true
startsecs=20
stdout_logfile={{ log_dir }}/app.log
stderr_logfile={{ log_dir }}/app-error.log
```

ansible/playbook.yml (current version, includes the step-14 rollback logic)

```
---
# Steps 13-14 - configure / provision a server for the Device Compliance Management Portal.
# * Idempotent: running it again on a configured node reports changed=0.
# * Safe releases: each version lives in /opt/dcmp/releases/<version>; /opt/dcmp/current points at the active one.
#   If the new release fails its health check the play restores the previous release automatically and then fails.
- name: Provision Device Compliance Portal server
  hosts: dcmp_servers
  become: true

  tasks:
    # ---------------- packages ----------------
    - name: Refresh apt cache (at most once per hour)
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600

    - name: Install prerequisite packages
      ansible.builtin.apt:
        name: "{{ os_packages }}"
        state: present

    # ---------------- user, folders ----------------
    - name: Create application group
      ansible.builtin.group:
        name: "{{ app_group }}"
        system: true

    - name: Create unprivileged application user
      ansible.builtin.user:
        name: "{{ app_user }}"
        group: "{{ app_group }}"
        system: true
        home: "{{ app_home }}"
        create_home: false
        shell: /usr/sbin/nologin

    - name: Create application, release, data and log folders
      ansible.builtin.file:
        path: "{{ item.path }}"
        state: directory
        owner: "{{ app_user }}"
        group: "{{ app_group }}"
        mode: "{{ item.mode }}"
      loop:
        - { path: "{{ app_home }}", mode: "0755" }
        - { path: "{{ app_home }}/releases", mode: "0755" }
```

```
    - { path: "{{ app_home }}/releases/{{ app_version }}", mode: "0755" }
    - { path: "{{ data_dir | dirname }}", mode: "0755" }
    - { path: "{{ data_dir }}", mode: "0750" }
    - { path: "{{ log_dir }}", mode: "0755" }

# ---------------- remember what is running now (for rollback) ----------------
- name: Look at the currently active release
  ansible.builtin.stat:
    path: "{{ app_home }}/current"
  register: current_link

- name: Remember the previous release
  ansible.builtin.set_fact:
    previous_release: "{{ current_link.stat.lnk_target | default('') if current_link.stat.islnk | default(false) else '' }}"

- name: Show release change
  ansible.builtin.debug:
    msg: "active release: '{{ previous_release | basename }}'  ->  new release: '{{ app_version }}'"

# ---------------- deploy the new release, with automatic rollback ----------------
- name: Deploy and verify release {{ app_version }}
  block:
    - name: Deploy the application WAR into its versioned release folder
      ansible.builtin.copy:
        src: "{{ war_src }}"
        dest: "{{ app_home }}/releases/{{ app_version }}/app.war"
        owner: "{{ app_user }}"
        group: "{{ app_group }}"
        mode: "0640"
      notify: Apply service changes

    - name: Point 'current' at this release
      ansible.builtin.file:
        src: "{{ app_home }}/releases/{{ app_version }}"
        dest: "{{ app_home }}/current"
        state: link
        owner: "{{ app_user }}"
        group: "{{ app_group }}"
      notify: Apply service changes

    - name: Install the supervisor program definition for the application
      ansible.builtin.template:
        src: dcmp.conf.j2
        dest: "/etc/supervisor/conf.d/{{ app_name }}.conf"
        owner: root
        group: root
        mode: "0644"
      notify: Apply service changes

    - name: Apply pending service changes now (so the health check below tests the new release)
      ansible.builtin.meta: flush_handlers

    - name: Make sure the application service is running
      ansible.builtin.command: supervisorctl status {{ app_name }}
      register: svc_status
      changed_when: false
      failed_when: "'RUNNING' not in svc_status.stdout and 'STARTING' not in svc_status.stdout"

    - name: Wait until the application port is listening
      ansible.builtin.wait_for:
        host: 127.0.0.1
        port: "{{ app_port }}"
        timeout: "{{ health_wait }}"

    - name: Health check
      ansible.builtin.uri:
        url: "http://127.0.0.1:{{ app_port }}/actuator/health"
        return_content: true
      register: health
      retries: 6
      delay: 5
      until: health.status == 200 and "UP" in health.content

    - name: Report result
      ansible.builtin.debug:
        msg: "{{ app_name }} {{ app_version }} (env={{ app_env }}) is {{ health.json.status }} on port {{ app_port }}"

  rescue:
    - name: "ROLLBACK - release {{ app_version }} failed its health check"
      ansible.builtin.debug:
        msg: "restoring previous release '{{ previous_release | basename }}'"

    - name: Stop here if there is nothing to roll back to
      ansible.builtin.fail:
        msg: "Release {{ app_version }} failed and there is no previous release to restore."
      when: previous_release == ''
```

```
        - name: Re-point 'current' at the previous release
          ansible.builtin.file:
            src: "{{ previous_release }}"
            dest: "{{ app_home }}/current"
            state: link
            owner: "{{ app_user }}"
            group: "{{ app_group }}"

        - name: Remove the failed release folder
          ansible.builtin.file:
            path: "{{ app_home }}/releases/{{ app_version }}"
            state: absent

        - name: Restore the service definition (so APP_VERSION names the release that is really running)
          ansible.builtin.template:
            src: dcmp.conf.j2
            dest: "/etc/supervisor/conf.d/{{ app_name }}.conf"
            owner: root
            group: root
            mode: "0644"
          vars:
            release_label: "{{ previous_release | basename }}"

        - name: Restart the service on the previous release
          ansible.builtin.shell: |
            supervisorctl reread
            supervisorctl update {{ app_name }}
            supervisorctl restart {{ app_name }}
          changed_when: true

        - name: Health check after rollback
          ansible.builtin.uri:
            url: "http://127.0.0.1:{{ app_port }}/actuator/health"
            return_content: true
          register: recovered
          retries: 12
          delay: 5
          until: recovered.status == 200 and "UP" in recovered.content

        - name: Fail the play so the bad release is reported
          ansible.builtin.fail:
            msg: "Release {{ app_version }} FAILED health check - rolled back to {{ previous_release | basename }} ({{ recovered.js
on.status }})."

  handlers:
    - name: Apply service changes
      ansible.builtin.shell: |
        supervisorctl reread
        supervisorctl update {{ app_name }}
        supervisorctl restart {{ app_name }}
      changed_when: true
```

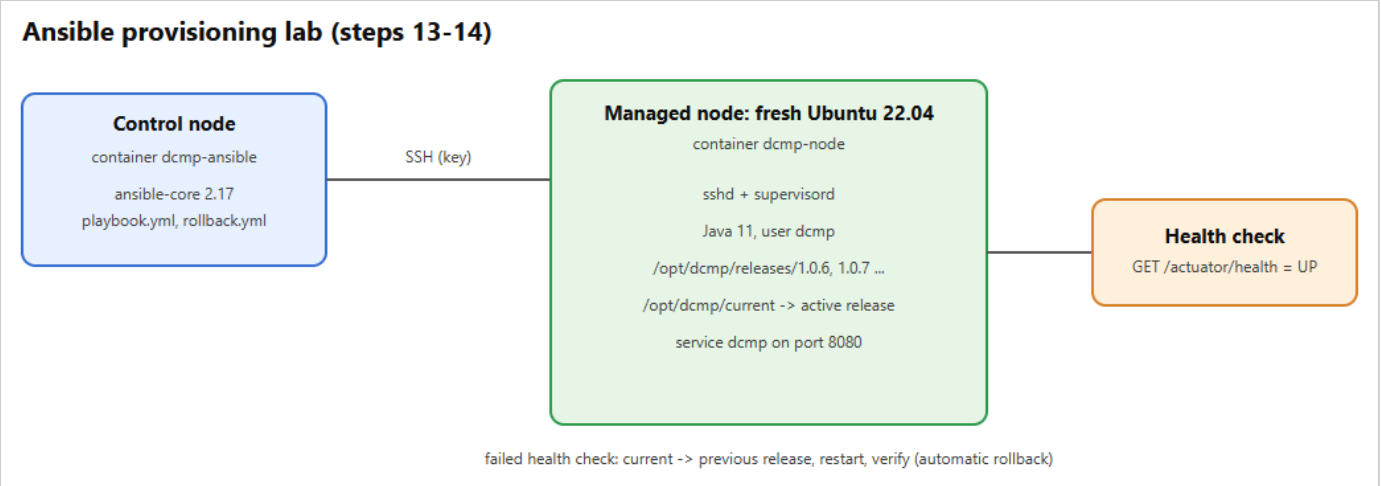


Figure 13.0 - Lab topology

Ansible - first execution (step 13)

```
$ ansible-playbook playbook.yml -e app_version=1.0.6   (step 13 - first execution)
PLAY [Configure Device Compliance Portal server]
TASK [Gathering Facts]
ok: [dcmp-node]
TASK [Refresh apt cache (at most once per hour)]
ok: [dcmp-node]
TASK [Install prerequisite packages]
changed: [dcmp-node]
TASK [Create application group]
changed: [dcmp-node]
TASK [Create unprivileged application user]
changed: [dcmp-node]
TASK [Create application, release, data and log folders]
changed: [dcmp-node] => (item={'path': '/opt/dcmp', 'mode': '0755'})
changed: [dcmp-node] => (item={'path': '/opt/dcmp/releases', 'mode': '0755'})
changed: [dcmp-node] => (item={'path': '/opt/dcmp/releases/1.0.6', 'mode': '0755'})
changed: [dcmp-node] => (item={'path': '/var/lib/dcmp', 'mode': '0755'})
changed: [dcmp-node] => (item={'path': '/var/lib/dcmp/data', 'mode': '0750'})
changed: [dcmp-node] => (item={'path': '/var/log/dcmp', 'mode': '0755'})
TASK [Deploy the application WAR into its versioned release folder]
changed: [dcmp-node]
TASK [Point 'current' at this release]
changed: [dcmp-node]
TASK [Install the supervisor program definition for the application]
changed: [dcmp-node]
TASK [Apply pending service changes now (so the health check below tests the new release)] ***
changed: [dcmp-node]
TASK [Make sure the application service is running]
ok: [dcmp-node]
TASK [Wait until the application port is listening]
ok: [dcmp-node]
TASK [Health check]
ok: [dcmp-node]
TASK [Report result]
ok: [dcmp-node] => {
    "msg": "dcmp 1.0.6 (env=dev) is UP on port 8080"
PLAY RECAP
dcmp-node                  : ok=14   changed=8    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 13.1 - First execution: ok=14 changed=8 failed=0

Ansible - second execution (idempotency preview)

```
$ ansible-playbook playbook.yml   (second run - idempotency preview)
ok: [dcmp-node] => {
    "msg": "dcmp 1.0.6 (env=dev) is UP on port 8080"
PLAY RECAP
dcmp-node                  : ok=13   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 13.2 - Second execution: changed=0

STEP 14

Automated Provisioning and Reliability Validation

Task given: Provision a clean target environment for the Device Compliance Management Portal using Puppet/Ansible. Deploy the application or container, rerun the automation to demonstrate idempotency, perform a health check and test rollback/recovery to the previous stable release.

Deliverable required: Provisioned node, idempotency evidence, health-check result and rollback/recovery demonstration.

A brand-new node was provisioned, the playbook was re-run to prove idempotency, health was checked, and several recovery scenarios were exercised. A bug found during this step (rollback restored the code but left the failed release's label) was fixed and re-tested. Source: `docs/14-provisioning-and-reliability.md`.

Deliverable required by the sheet	Shown in this report
Provisioned node	Figures 14.1-14.2
Idempotency evidence	Figure 14.3 (changed=0)
Health-check result	Figures 14.2, 14.4, 14.7
Rollback / recovery demonstration	Figures 14.5-14.9

Docker / shell - clean managed node before provisioning

```
$ bash ansible/run.sh up   # then verify the node is clean
98f8d311da8dd1fa107ad5f10429fd4160a11f8a39b0747b58398822c66b17a1
63d06afd52a289efe7e4d8e980a04f8d06975e088053c7bc40ac3157135514b2
dcmp-node | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
node: dcmp-node dcmp-node:base Up 5 seconds
java: NOT installed (clean node)
id: 'dcmp': no such user
ls: cannot access '/opt/dcmp': No such file or directory
```

Figure 14.1 - Clean node before provisioning: no Java, no dcmp user, no /opt/dcmp

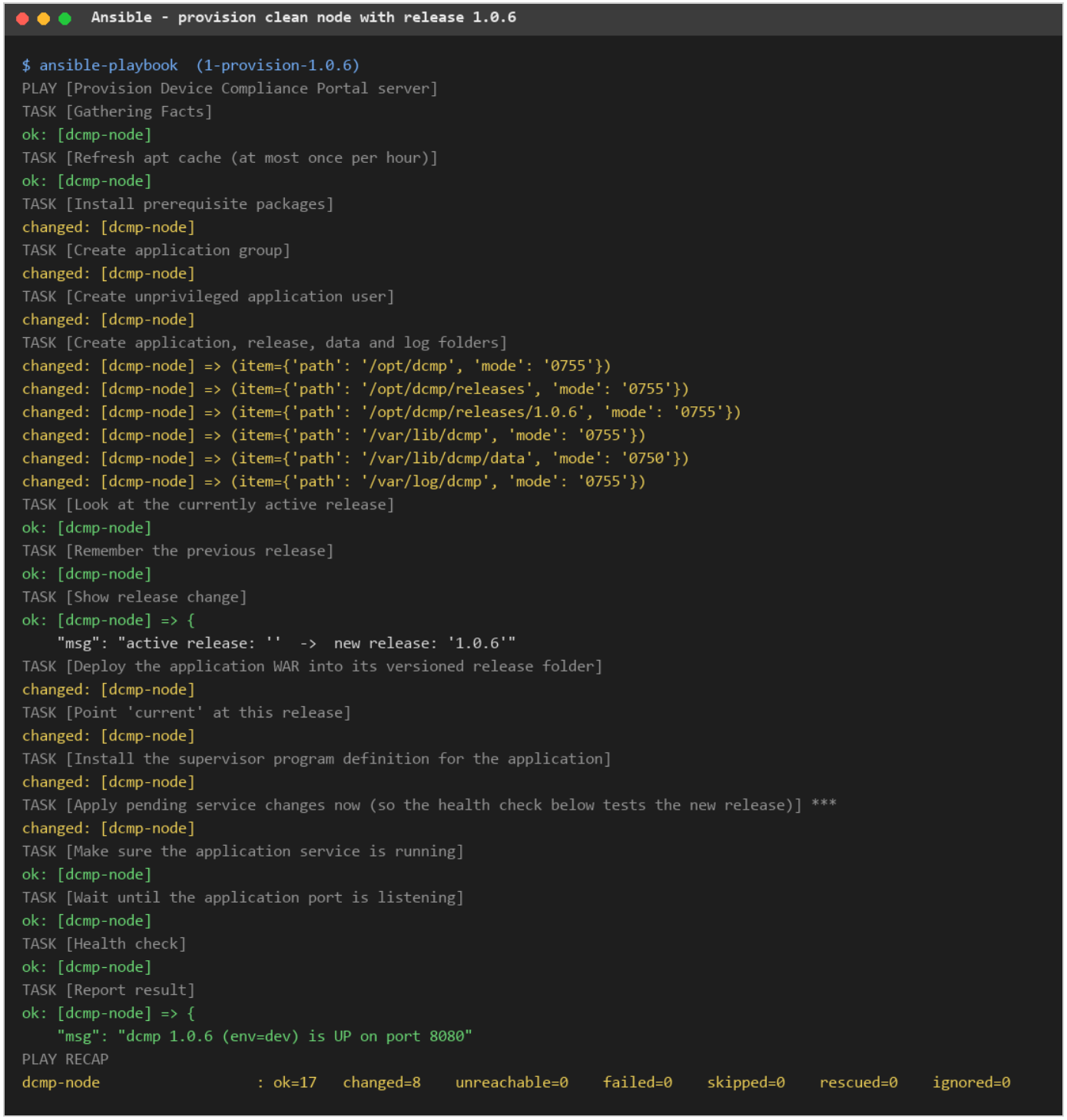


Figure 14.2 - Provisioning release 1.0.6: ok=17 changed=8 failed=0, health UP

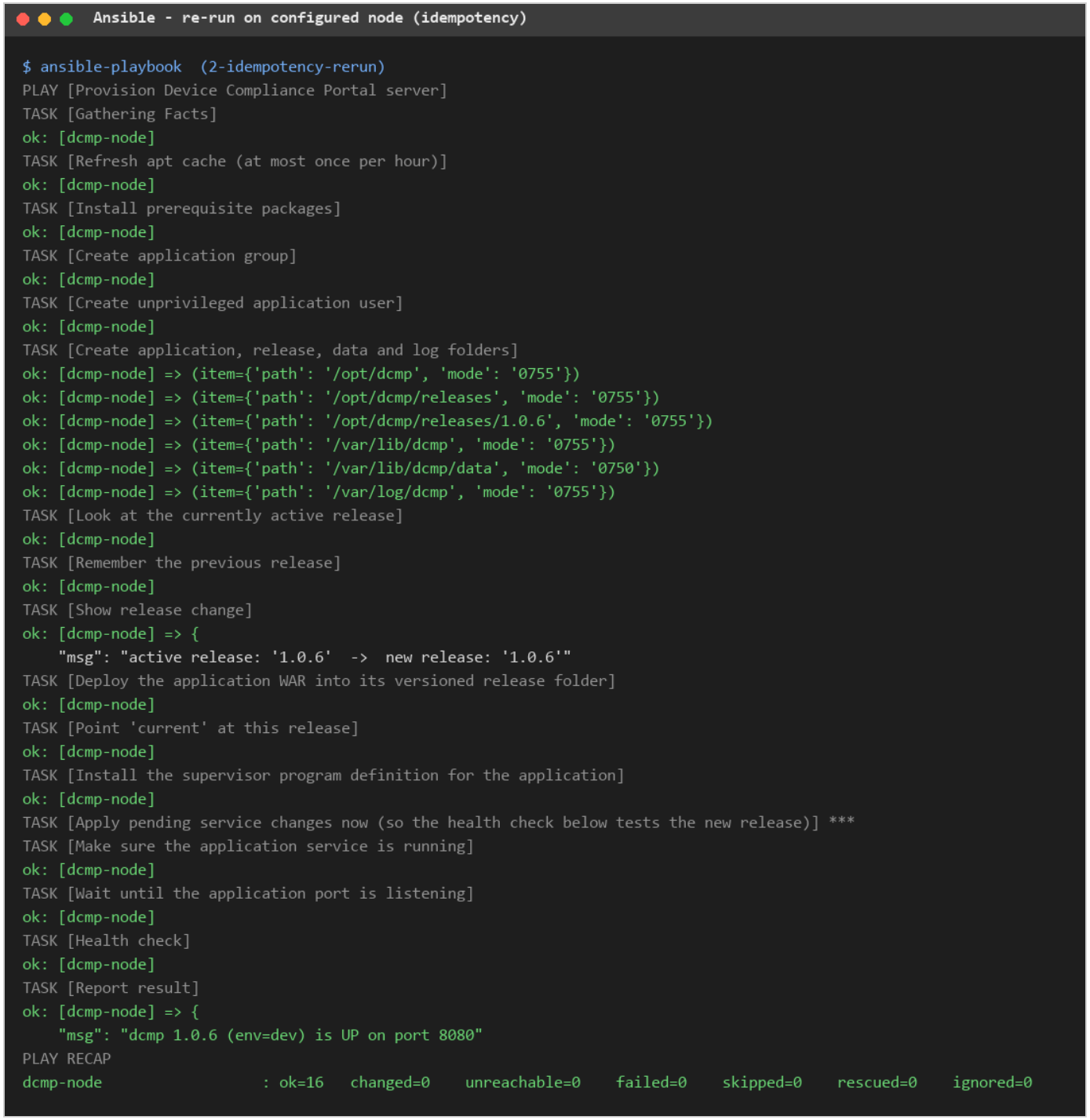


Figure 14.3 - Re-run: ok=16 changed=0 (idempotent)

```
Ansible - upgrade to release 1.0.7

$ ansible-playbook  (3-upgrade-1.0.7)
PLAY [Provision Device Compliance Portal server]
TASK [Gathering Facts]
ok: [dcmp-node]
TASK [Refresh apt cache (at most once per hour)]
ok: [dcmp-node]
TASK [Install prerequisite packages]
ok: [dcmp-node]
TASK [Create application group]
ok: [dcmp-node]
TASK [Create unprivileged application user]
ok: [dcmp-node]
TASK [Create application, release, data and log folders]
ok: [dcmp-node] => (item={'path': '/opt/dcmp', 'mode': '0755'})
ok: [dcmp-node] => (item={'path': '/opt/dcmp/releases', 'mode': '0755'})
changed: [dcmp-node] => (item={'path': '/opt/dcmp/releases/1.0.7', 'mode': '0755'})
ok: [dcmp-node] => (item={'path': '/var/lib/dcmp', 'mode': '0755'})
ok: [dcmp-node] => (item={'path': '/var/lib/dcmp/data', 'mode': '0750'})
ok: [dcmp-node] => (item={'path': '/var/log/dcmp', 'mode': '0755'})
TASK [Look at the currently active release]
ok: [dcmp-node]
TASK [Remember the previous release]
ok: [dcmp-node]
TASK [Show release change]
ok: [dcmp-node] => {
    "msg": "active release: '1.0.6'  ->  new release: '1.0.7'"
TASK [Deploy the application WAR into its versioned release folder]
changed: [dcmp-node]
TASK [Point 'current' at this release]
changed: [dcmp-node]
TASK [Install the supervisor program definition for the application]
changed: [dcmp-node]
TASK [Apply pending service changes now (so the health check below tests the new release)] ***
changed: [dcmp-node]
TASK [Make sure the application service is running]
ok: [dcmp-node]
TASK [Wait until the application port is listening]
ok: [dcmp-node]
TASK [Health check]
ok: [dcmp-node]
TASK [Report result]
ok: [dcmp-node] => {
    "msg": "dcmp 1.0.7 (env=dev) is UP on port 8080"
PLAY RECAP
dcmp-node                  : ok=17   changed=5    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 14.4 - Upgrade to release 1.0.7 (active release 1.0.6 -> 1.0.7), health UP

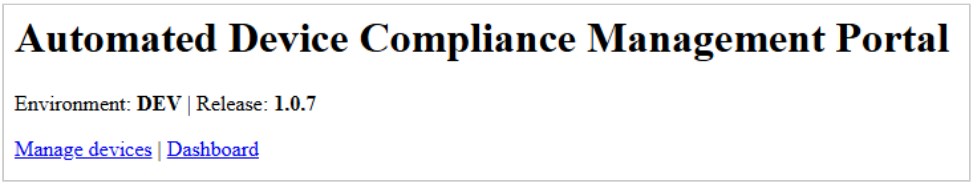


Figure 14.5 - Provisioned node: home page shows Release 1.0.7

```
Ansible - bad release 1.0.8 -> automatic rollback   [1/2]

$ ansible-playbook  (4-bad-release-auto-rollback)
PLAY [Provision Device Compliance Portal server]
TASK [Gathering Facts]
ok: [dcmp-node]
TASK [Refresh apt cache (at most once per hour)]
ok: [dcmp-node]
TASK [Install prerequisite packages]
ok: [dcmp-node]
TASK [Create application group]
ok: [dcmp-node]
TASK [Create unprivileged application user]
ok: [dcmp-node]
TASK [Create application, release, data and log folders]
ok: [dcmp-node] => (item={'path': '/opt/dcmp', 'mode': '0755'})
ok: [dcmp-node] => (item={'path': '/opt/dcmp/releases', 'mode': '0755'})
changed: [dcmp-node] => (item={'path': '/opt/dcmp/releases/1.0.8', 'mode': '0755'})
ok: [dcmp-node] => (item={'path': '/var/lib/dcmp', 'mode': '0755'})
ok: [dcmp-node] => (item={'path': '/var/lib/dcmp/data', 'mode': '0750'})
ok: [dcmp-node] => (item={'path': '/var/log/dcmp', 'mode': '0755'})
TASK [Look at the currently active release]
ok: [dcmp-node]
TASK [Remember the previous release]
ok: [dcmp-node]
TASK [Show release change]
ok: [dcmp-node] => {
    "msg": "active release: '1.0.7'  ->  new release: '1.0.8'"
TASK [Deploy the application WAR into its versioned release folder]
changed: [dcmp-node]
TASK [Point 'current' at this release]
changed: [dcmp-node]
TASK [Install the supervisor program definition for the application]
changed: [dcmp-node]
TASK [Apply pending service changes now (so the health check below tests the new release)] ***
fatal: [dcmp-node]: FAILED! => {"changed": true, "cmd": "supervisorctl reread\nsupervisorctl update dcmp\nsupervisorct
l restart dcmp\n", "delta": "0:00:01.601954", "end": "202
TASK [ROLLBACK - release 1.0.8 failed its health check]
ok: [dcmp-node] => {
    "msg": "restoring previous release '1.0.7'"
TASK [Stop here if there is nothing to roll back to]
skipping: [dcmp-node]
TASK [Re-point 'current' at the previous release]
changed: [dcmp-node]
TASK [Remove the failed release folder]
changed: [dcmp-node]
TASK [Restore the service definition (so APP_VERSION names the release that is really running)] ***
changed: [dcmp-node]
TASK [Restart the service on the previous release]
```

Figure 14.6 - Bad release 1.0.8: health check fails -> automatic rollback to 1.0.7 (part 1)

```
Ansible - bad release 1.0.8 -> automatic rollback   [2/2]

changed: [dcmp-node]
TASK [Health check after rollback]
ok: [dcmp-node]
TASK [Fail the play so the bad release is reported]
fatal: [dcmp-node]: FAILED! => {"changed": false, "msg": "Release 1.0.8 FAILED health check - rolled back to 1.0.7 (UP
)."}
PLAY RECAP
dcmp-node                  : ok=18   changed=8    unreachable=0    failed=2    skipped=1    rescued=0    ignored=0
```

Figure 14.6 - Bad release 1.0.8: health check fails -> automatic rollback to 1.0.7 (part 2)

```
Ansible - manual rollback to 1.0.6

$ ansible-playbook  (5-manual-rollback-1.0.6)
PLAY [Roll back Device Compliance Portal to a previous release]
TASK [Gathering Facts]
ok: [dcmp-node]
TASK [A target version is required]
skipping: [dcmp-node]
TASK [Check that the release folder still exists]
ok: [dcmp-node]
TASK [Currently active release]
ok: [dcmp-node]
TASK [Show rollback]
ok: [dcmp-node] => {
    "msg": "rolling back '1.0.7' -> '1.0.6'"
TASK [Point 'current' at the target release]
changed: [dcmp-node]
TASK [Keep the service definition's APP_VERSION label in step]
changed: [dcmp-node]
TASK [Restart the service]
changed: [dcmp-node]
TASK [Health check]
ok: [dcmp-node]
TASK [Report result]
ok: [dcmp-node] => {
    "msg": "rollback complete: release 1.0.6 is UP"
PLAY RECAP
dcmp-node                  : ok=9    changed=3    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
```

Figure 14.7 - Manual rollback to the earlier stable release 1.0.6

```
Docker - crash and reboot recovery

$ docker exec dcmp-node supervisorctl status dcmp
dcmp                             STARTING

$ docker exec dcmp-node sh -c "kill -9 $(pgrep -u dcmp java)"     # simulate the application crashing
java process killed

$ docker exec dcmp-node supervisorctl status dcmp   # supervisor notices and restarts it
dcmp                             STARTING

$ curl localhost:8093/actuator/health
{"status":"UP"}

$ docker restart dcmp-node        # simulate a node reboot
dcmp-node
$ curl localhost:8093/actuator/health
{"status":"UP"}
$ docker exec dcmp-node supervisorctl status
dcmp                             STARTING
sshd                             RUNNING   pid 7, uptime 0:00:06
```

Figure 14.8 - Process crash (kill -9) and node reboot: service recovers by itself

```
Ansible - promote latest stable release 1.0.7

$ ansible-playbook  (7-final-1.0.7)
PLAY [Provision Device Compliance Portal server]
TASK [Gathering Facts]
ok: [dcmp-node]
TASK [Refresh apt cache (at most once per hour)]
ok: [dcmp-node]
TASK [Install prerequisite packages]
ok: [dcmp-node]
TASK [Create application group]
ok: [dcmp-node]
TASK [Create unprivileged application user]
ok: [dcmp-node]
TASK [Create application, release, data and log folders]
ok: [dcmp-node] => (item={'path': '/opt/dcmp', 'mode': '0755'})
ok: [dcmp-node] => (item={'path': '/opt/dcmp/releases', 'mode': '0755'})
ok: [dcmp-node] => (item={'path': '/opt/dcmp/releases/1.0.7', 'mode': '0755'})
ok: [dcmp-node] => (item={'path': '/var/lib/dcmp', 'mode': '0755'})
ok: [dcmp-node] => (item={'path': '/var/lib/dcmp/data', 'mode': '0750'})
ok: [dcmp-node] => (item={'path': '/var/log/dcmp', 'mode': '0755'})
TASK [Look at the currently active release]
ok: [dcmp-node]
TASK [Remember the previous release]
ok: [dcmp-node]
TASK [Show release change]
ok: [dcmp-node] => {
    "msg": "active release: '1.0.6'  ->  new release: '1.0.7'"
TASK [Deploy the application WAR into its versioned release folder]
ok: [dcmp-node]
TASK [Point 'current' at this release]
changed: [dcmp-node]
TASK [Install the supervisor program definition for the application]
changed: [dcmp-node]
TASK [Apply pending service changes now (so the health check below tests the new release)] ***
changed: [dcmp-node]
TASK [Make sure the application service is running]
ok: [dcmp-node]
TASK [Wait until the application port is listening]
ok: [dcmp-node]
TASK [Health check]
ok: [dcmp-node]
TASK [Report result]
ok: [dcmp-node] => {
    "msg": "dcmp 1.0.7 (env=dev) is UP on port 8080"
PLAY RECAP
dcmp-node                  : ok=17   changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 14.9 - Final promotion of the latest stable release 1.0.7



Figure 14.10 - Health endpoint of the provisioned node (via published port 8093)

6. Problem found and fixed during testing

The first attempt at scenario A (`step14-4a-first-attempt-label-bug.log`) did roll back the code and the app was healthy, but the home page still said `Release: 1.0.8`: the rollback restored the symlink but not the service definition, which still carried the failed release's `APP_VERSION` label. Fix: the `rescue` section now re-renders the service definition with the previous release label (variable `release_label`) before restarting. Re-test: page shows `Release: 1.0.7` after the failed deployment. (Note: Ansible extra-vars `-e app_version=…` always win over task-level `vars`, so a separate variable name was needed.)